



INGENIERÍA

Navegación Autónoma de Drones Urbanos con Visión Monocular y Small Language Model (SLM)

Tesis para Magister en Ciencia de Datos

Maestría en Ciencia de Datos - 2024/2025

Directores:

- DEL ROSSO, Rodrigo
- NUSKE, Ezequiel

Alumno:

- NICOLAU, Jorge Enrique

Resumen

La navegación autónoma de cuadricópteros eVTOL en entornos urbanos densos —como el de Buenos Aires— enfrenta severos desafíos derivados de la alta densidad de obstáculos, la degradación de señales satelitales y las restricciones presupuestarias que descartan sensores de alto costo como LiDAR. Para abordar esta problemática bajo estrictas restricciones de hardware y en conformidad con las normativas locales (ANAC Res. 880/2019), esta tesis presenta un sistema integral de planificación previa, percepción monocular y decisión táctica basado exclusivamente en **visión monocular** y **modelos de lenguaje pequeños (SLM)** locales.

La arquitectura desacopla un módulo de planificación deliberativa en tierra (estación de control WebDCS, que compila directivas en lenguaje natural a un manifiesto inmutable de navegación) de un lazo táctico a bordo de alta frecuencia. El módulo de percepción implementa un pipeline ligero de **flujo óptico denso derotado** mediante telemetría de actitud, mapeo de ocupación espacial (`ObstacleField`) y estimación de **Tiempo-a-Colisión (TTC)** a partir de la divergencia del campo de flujo, alcanzando un área bajo la curva ROC de 0.96–0.97 en su validación frente a canales de profundidad de referencia. En la capa de decisión a bordo, se implementa una arquitectura híbrida de decisión por ciclo gobernada por un SLM (modelos de 2B–4B parámetros ejecutados localmente con decodificación estructurada `json_schema`) y orquestada mediante grafos de estados con salvaguardas deterministas.

El desarrollo y la experimentación se realizan en el simulador **Cosys-AirSim** sobre **Unreal Engine**, empleando el entorno urbano `Sample City` como plataforma principal de validación y benchmark, y calibrando la dinámica de vuelo contra conjuntos de datos de telemetría de drones comerciales reales. El trabajo documenta una evaluación comparativa sistemática del brazo SLM frente a Máquinas de Estados Finitos (FSM) y control puramente reactivo, analizando métricas de tasa de éxito de misión, tiempo de reacción y longitud de trayectoria ponderada por éxito (SPL). Asimismo, se identifican y analizan los modos de falla estructurales propios de sistemas robóticos asistidos por modelos de lenguaje, demostrando que los errores críticos en la toma de decisiones se originan predominantemente en degradaciones silenciosas del contrato de datos de la interfaz de percepción antes que en el razonamiento del modelo.

Palabras clave

Navegación Autónoma de Drones | Visión Monocular | Flujo Óptico | Tiempo-a-Colisión (TTC) | Small Language Models (SLM) | Máquinas de Estados Finitos (FSM) | Control Táctico Híbrido | Simulación en Unreal Engine / Cosys-AirSim | Robótica Urbana de Bajo Costo

Índice

Carátula y Resumen

1. Introducción

2. Estado del Arte

3. Entorno de Simulación

4. Planificación de Misión (WebDCS)

5. Arquitectura del Lazo Táctico

6. Percepción Monocular

7. Estimación TTC

8. Decisiones SLM

9. Modos de Falla LLM

10. Metodología Experimental

11. Resultados

12. Conclusiones

13. Referencias

Anexos

1. Introducción

1.1 Motivación y contexto

Los drones autónomos han cruzado en los últimos años la frontera que separa la plataforma experimental de la infraestructura real. En el frente civil, el mercado global de UAVs comerciales alcanzó los USD 28,6 mil millones en 2025 con una proyección de superar los USD 52,1 mil millones en 2029, impulsado por entregas de última milla, movilidad aérea urbana, inspección de infraestructura crítica y agricultura de precisión (Dataintel, 2025). En el frente militar, la guerra en Ucrania consolidó al dron autónomo como el activo táctico más transformador del conflicto moderno: en 2024, Ucrania fabricó cerca de dos millones de drones y reentrenó modelos de inteligencia artificial con datos de combate real, multiplicando por tres o cuatro la tasa de impacto de sus sistemas aéreos (Breaking Defense, 2025). En ambos escenarios, el denominador técnico común es la misma exigencia: un vehículo de bajo costo debe percibir su entorno, tomar decisiones en milisegundos y actuar sin intervención humana directa en el lazo, con hardware de borde limitado y a menudo en ausencia de conectividad.

Argentina enfrenta esta convergencia tecnológica con tres urgencias concretas y simultáneas. La primera es la vigilancia del litoral marítimo: la Zona Económica Exclusiva comprende aproximadamente 1,6 millones de km² —incluyendo áreas críticas para la pesca comercial, la seguridad marítima y la soberanía territorial en el Atlántico Sur y Malvinas—, un espacio imposible de cubrir de forma continua con medios tripulados a costo razonable. En mayo de 2026, Argentina formalizó un acuerdo con los Estados Unidos para incorporar drones militares VTOL Shield AI V-Bat y aeronaves de patrulla bajo la *Maritime Domain Situational Awareness Initiative* (Infobae, 2026), y una propuesta legislativa de julio de 2026 impulsa la creación de un Sistema Nacional de Vigilancia Marítima con UAVs (El Estratégico, 2026). La segunda es la respuesta temprana a incendios forestales: los incendios en la Patagonia y el litoral fluvial son uno de los riesgos ambientales y económicos más severos del país; el Ministerio de Ambiente ya incorporó 17 UAVs autónomos en parques nacionales de Neuquén y Chubut, y la Provincia de Misiones avanza en despacho autónomo de drones inmediatamente tras la detección de una columna de humo. La tercera urgencia es la que articula directamente el entorno experimental de este trabajo: la logística aérea urbana en Buenos Aires. La Ciudad Autónoma de Buenos Aires abrió en 2025 el marco para la entrega comercial con drones (iProfesional, 2025), y la ANAC publicó la Resolución 319/2025 — que aprueba la Parte 100 del Reglamento Aeronáutico Civil Argentino— y la Resolución 550/2025, que desregulan y simplifican la operación de aeronaves no tripuladas en el espacio aéreo nacional. Buenos Aires concentra en su tejido urbano los desafíos técnicos más exigentes del dominio: densidad edilicia extrema en el Microcentro y Palermo con cañones de 8–12 pisos, degradación severa del GPS dentro de esos cañones, cables de alta y baja tensión a baja altura, interferencia electromagnética y proximidad al espacio controlado de Aeroparque. En los tres dominios, el problema técnico de fondo es idéntico: decidir, en tiempo real y con percepción limitada, qué hacer a continuación.

La navegación autónoma en entornos urbanos densos agrega capas de dificultad que no aparecen en espacios abiertos: alta densidad de obstáculos fijos y móviles, degradación de la señal GPS por la altura de los edificios (*urban canyon*), variaciones extremas de iluminación y, en el contexto latinoamericano en particular, restricciones de presupuesto que descartan sensores costosos como el LiDAR y la obligación de operar bajo marcos regulatorios específicos (Resolución ANAC 880/2019; Decreto 663/2024). Un dron sin percepción confiable y sin capacidad de decisión robusta representa un riesgo tangible: puede colisionar, invadir zonas sensibles o interferir con otras operaciones, con consecuencias humanas y económicas concretas (Samy et al., 2019; Aldao et al., 2022).

Este trabajo se enmarca en esa tensión: cómo dotar a un cuadricóptero eVTOL de navegación autónoma reactiva usando exclusivamente visión monocular y hardware de bajo costo —sin redes neuronales de detección, sin LiDAR, sin GPU dedicada de alto costo—, manteniendo rigor metodológico y un diseño de seguridad explícito.

1.2 Objetivos

Objetivo general. Desarrollar un sistema de visión por computadora basado en percepción monocular con capacidad de navegación autónoma de cuadricópteros eVTOL, aplicable a logística urbana, mantenimiento de infraestructura, vigilancia de áreas extensas y atención de emergencias en escenarios representativos del territorio argentino.

Objetivos específicos:

1. Implementar un pipeline reproducible sobre el simulador Cosys-AirSim (basado en Unreal Engine 5.5) que permita validar el desempeño del sistema de navegación en condiciones controladas, repetibles y con telemetría calibrada frente a vuelos reales.
2. Integrar detección de obstáculos y navegación reactiva a partir de percepción monocular ligera —flujo óptico denso derotado, estimación de TTC, mapa de ocupación espacial— optimizada para hardware de bajo costo, de modo de sostener un caso de negocio viable para economías con recursos limitados.
3. Evaluar el rendimiento de un modelo de lenguaje pequeño (SLM) como capa de decisión táctica frente a una máquina de estados finitos (FSM) —el estándar de facto en pilotos automáticos— y frente a un controlador puramente reactivo, usando tasa de éxito de misión, tiempo de reacción y longitud de trayectoria ponderada por éxito (SPL) como métricas de comparación en entornos urbanos de complejidad creciente.
4. Documentar con evidencia medida los modos de falla estructurales de la integración SLM-lazo de control, identificando patrones transferibles a los dominios de aplicación de alta pertinencia local (vigilancia marítima, respuesta a incendios forestales) donde los recursos computacionales de borde y la ausencia de conectividad imponen las mismas restricciones estudiadas en el entorno experimental.

1.3 Alcance y diseño de la arquitectura

El sistema de navegación propuesto implementa una arquitectura de percepción monocular ligera **sin redes neuronales de detección**, basada en flujo óptico denso derotado mediante telemetría de actitud y estimación física del Tiempo-a-Colisión (TTC). Esta elección de diseño responde a fundamentos técnicos y computacionales concretos:

1. **Eficiencia y presupuesto de cómputo:** En plataformas robóticas de bajo costo que comparten CPU con la inferencia local de un modelo de lenguaje, la estimación geométrica por flujo óptico reduce drásticamente la latencia por ciclo frente a detectores de objetos basados en aprendizaje profundo.
2. **Robustez fuera de distribución (OOD):** A diferencia de modelos supervisados dependientes de categorías predefinidas de obstáculos, el cálculo de divergencia del campo de flujo modela directamente el fenómeno físico de aproximación: reacciona ante cualquier geometría sin importar su textura o clase semántica, una propiedad crítica en entornos no estructurados como el litoral marítimo o un área de incendio.
3. **Adecuación a la geometría del vuelo urbano:** Para un cuadricóptero con cámara frontal en cañón urbano, el campo visual está dominado por estructuras verticales y fachadas. El flujo óptico traslacional permite estimar la ocupación espacial (`ObstacleField`) en cuadrantes discretos de forma directa y determinista sin ningún entrenamiento previo.

La capa de decisión implementa un modelo de lenguaje pequeño cuantizado (Qwen2.5-VL-3B, 2-4 B de parámetros) ejecutado localmente con decodificación estructurada mediante `json_schema`, orquestado por un grafo LangGraph con salvaguardas deterministas. El capítulo 6 desarrolla los fundamentos del módulo de percepción y el capítulo 8 la ingeniería de la capa de decisión.

1.4 El hilo conductor de la tesis

En arquitecturas robóticas híbridas donde un modelo de lenguaje consume descripciones simbólicas o estructuradas del entorno para tomar decisiones de navegación, la integridad del contrato de interfaz entre la percepción y el modelo resulta crítica. La experiencia experimental demuestra que **el fallo más severo en estos sistemas no radica en la capacidad de razonamiento del modelo de lenguaje, sino en la fidelidad y consistencia del contrato de datos que lo alimenta.**

Cuando la capa de percepción o el estado del sistema entrega representaciones desincronizadas, degradadas o vacías, los modelos de lenguaje tienden a generar respuestas sintácticamente válidas y aparentemente razonables a partir de premisas falsas. Dado que el modelo no se interrumpe con excepciones visibles y el vehículo continúa su trayectoria, estas fallas resultan estructuralmente invisibles si solo se observa el comportamiento cinemático superficial. Zhu et al. (2024) plantean formalmente la pregunta de hasta qué punto los LLMs comprenden el contexto que reciben; este trabajo la responde empíricamente desde el ángulo opuesto: ¿qué ocurre cuando el contexto que el modelo recibe está sistemáticamente corrompido?

Este principio —la necesidad de validación formal, auditoría de contratos de datos y salvaguardas deterministas en lazos de control asistidos por SLM— constituye el hilo conductor de este trabajo. Su relevancia se extiende más allá del entorno de simulación: en aplicaciones de patrulla marítima o respuesta a incendios forestales donde el UAV opera sin cobertura de red y sin supervisión humana directa, la robustez del contrato de datos entre percepción y decisión no es una propiedad académica sino un requisito operacional. El capítulo 9 documenta de manera sistemática los cinco modos de falla estructurales identificados en la integración de estos componentes y las salvaguardas implementadas para mitigarlos; el capítulo 12 los retoma como conclusión central y proyecta sus implicancias hacia los dominios de aplicación descriptos.

1.5 Estructura del informe

El capítulo 2 sitúa este trabajo respecto de la literatura relevante en cuatro líneas: navegación autónoma de UAVs desde la planificación clásica hasta el control adaptativo (§2.1); percepción monocular para detección y evitación de obstáculos (§2.2); arquitecturas de control asistidas por modelos de lenguaje —incluyendo los trabajos más recientes de 2025-2026 sobre modelos VLA y SLM en UAVs— (§2.3); y dos dominios de aplicación de alta pertinencia local para Argentina —vigilancia del litoral marítimo e incendios forestales— donde el patrón técnico de este trabajo se transfiere directamente (§2.4), seguidos del posicionamiento explícito de esta tesis en el espacio de soluciones (§2.5).

El capítulo 3 describe la construcción del entorno de simulación —Unreal Engine 5.5 con Cosys-AirSim y la jerarquía de tres entornos de dificultad creciente (MiniSim, TownSim, CitySim)— y su validación estadística frente a telemetría de vuelos reales. El capítulo 4 documenta la planificación deliberativa en tierra y la estación de control WebDCS. Los capítulos 5 a 9 documentan la arquitectura del lazo táctico a bordo: el grafo de control LangGraph completo (cap. 5), la percepción monocular sin redes neuronales (cap. 6), la validación experimental del TTC (cap. 7), la capa de decisión del SLM con decodificación restringida (cap. 8) y los modos de falla específicos de lazos híbridos con modelos de lenguaje (cap. 9). El capítulo 10 describe la metodología experimental —diseño factorial, brazos de comparación y métricas—. Los capítulos 11 y 12 sintetizan los resultados comparativos y las conclusiones, complementados por las referencias bibliográficas estructuradas en el capítulo 13.

2. Estado del arte y trabajos relacionados

Los drones autónomos han dejado de ser una curiosidad tecnológica para convertirse simultáneamente en infraestructura civil crítica y en uno de los factores más determinantes de la guerra moderna, dos procesos que —aunque distintos en sus objetivos— comparten la misma presión técnica de fondo: la necesidad de que el vehículo tome decisiones correctas sin intervención humana en el lazo, a baja altitud, con recursos de cómputo limitados y en entornos no controlados. En el frente civil, el mercado global de UAVs comerciales alcanzó los USD 28,6 mil millones en 2025 y se proyecta que supere los USD 52,1 mil millones en 2029 (Dataintel, 2025), impulsado por entregas de última milla, movilidad aérea urbana (UAM), inspección automatizada de infraestructura —tendidos eléctricos, oleoductos, torres de telecomunicaciones— y agricultura de precisión. Agencias reguladoras como la FAA, la EASA y la CAAC han ampliado progresivamente las autorizaciones de operaciones BVLOS (*Beyond Visual Line of Sight*) durante 2025-2026, abriendo la puerta a redes de drones autónomos operadas comercialmente a escala (Coptorz, 2026; MDPI Drones, 2026). En el frente militar, la guerra en Ucrania (2022–presente) se consolidó como el primer conflicto a gran escala en que los UAVs —drones FPV de primera persona, municiones merodeadoras y enjambres coordinados— desempeñan un papel táctico central. Ucrania fabricó cerca de dos millones de drones en 2024; sus fuerzas reentrenaron modelos de IA con datos clasificados de combate real, lo que multiplicó por tres o cuatro la tasa de impacto de sus sistemas aéreos autónomos (Breaking Defense, 2025). En respuesta, Rusia estableció a fines de 2024 su propia rama de sistemas no tripulados. El conflicto aceleró además la investigación sobre guerra de contramedidas electrónicas y navegación sin GPS, haciendo de la autonomía robusta de borde un problema de primer orden tanto técnico como estratégico (CSIS, 2024; Modern War Institute, s.f.). Esta doble presión —demanda civil de escala y urgencia militar operacional— imprime relevancia directa a la pregunta que articula este trabajo: cómo dotar a un UAV de capacidad de decisión contextual con recursos computacionales de borde reducidos.

2.1 Navegación autónoma de UAVs: de la planificación clásica al control adaptativo

Los primeros enfoques de navegación autónoma en entornos urbanos combinan planificación global sobre datos geográficos con evasión reactiva de obstáculos móviles. Castelli et al. (2016) proponen un sistema híbrido para vuelo seguro a baja altitud que combina el algoritmo *A con datos GIS y replanificación dinámica, priorizando rutas sobre techos en lugar de calles para aumentar la distancia de seguridad frente a objetos en movimiento*. Hughes y Engelbrecht (2023) extienden esta idea a enjambres, con planificación de largo plazo y evasión cooperativa basada en bounding volume hierarchies* y diagramas de Voronoi sobre un modelo 3D tipo Manhattan —el mismo tipo de trazado urbano en cuadrícula que emplea el entorno de evaluación densa de este trabajo (CitySim / Tier 2; ver capítulos 3 y 10)—.

Frente a estos enfoques deliberativos, las máquinas de estados finitos (FSM) siguen siendo el estándar de facto en pilotos automáticos por su predictibilidad y bajo costo computacional: Hu et al. (2024) las emplean para coordinación cooperativa de enjambres en emergencias, y Hoang et al. (2024) las utilizan como base de un sistema de recarga autónoma sobre líneas de alta tensión. Su limitación es estructural: una FSM no interpreta contexto más allá de los umbrales sobre los que fue diseñada. Ese contraste —predictibilidad y bajo costo de la FSM frente a la promesa de adaptabilidad contextual de un modelo de lenguaje— es precisamente el eje de la comparación experimental de este trabajo (cap.10), y es también donde AlMahamid y Grolinger (2022) y Bouhamed et al. (2020) sitúan al aprendizaje por refuerzo como tercera vía: políticas aprendidas en lugar de reglas explícitas o inferencia de un modelo de lenguaje.

2.2 Percepción monocular para detección y evitación de obstáculos

La adopción de una arquitectura de percepción monocular ligera sin redes neuronales profundas (cap. 6) se fundamenta en una línea de investigación consolidada sobre detección de obstáculos por visión artificial clásica. Badrloo y Varshosaz (2017) revisan el estado del arte de la detección monocular en robótica móvil, mientras que Molineros et al. (2012) y Chen et al. (2016) analizan la estimación de riesgo mediante flujo residual y análisis 3D.

Por otra parte, técnicas clásicas basadas en Mapeo de Perspectiva Inversa (IPM), como las propuestas por Kaneko et al. (2017) y retomadas por Zhou et al. (2026), asumen la existencia de un plano de suelo dominante en el campo de visión, una hipótesis adecuada para vehículos terrestres o cámaras cenitales pero inaplicable a cuadricópteros con cámara frontal a baja altitud en cañones urbanos, donde el campo visual está dominado por estructuras verticales y fachadas.

En contraste, el uso de flujo óptico denso traslacional constituye una solución físicamente rigurosa y libre de entrenamiento supervisado. Vera-Yanez et al. (2024) desarrollan un detector de obstáculos aéreos basado íntegramente en flujo óptico —morfología, foco de expansión (*focus of expansion*, FOE) y agrupamiento—, demostrando que desacoplar la rotación propia del movimiento traslacional permite aislar los vectores de aproximación generados por objetos en la trayectoria. Este principio físico —la divergencia del campo traslacional como estimador directo del Tiempo-a-Colisión (TTC)— constituye la base matemática del módulo de percepción y estimación de TTC implementado en este trabajo (cap. 6 y 7). Enfoques alternativos basados en aprendizaje profundo monocular (Rill y Faragó, 2021; Shi et al., 2024) logran estimaciones densas de profundidad pero introducen una alta carga computacional y sensibilidad a dominios no vistos, justificando la preferencia por métodos geométricos deterministas en plataformas de bajo costo.

2.3 Arquitecturas de control asistidas por modelos de lenguaje

La idea de utilizar modelos de lenguaje como capa de razonamiento de alto nivel en un robot fue articulada con precisión por Vemprala et al. (2023) en un trabajo seminal de Microsoft Research que emplea ChatGPT para programar misiones de drones e inspección sobre una instancia de AirSim —el mismo simulador que sustenta este trabajo—. El resultado clave de Vemprala et al. no es el rendimiento en ninguna tarea específica sino la demostración de que un LLM puede generar código ejecutable de control aéreo a partir de instrucciones en lenguaje natural, separando el razonamiento semántico de la ejecución motora: una arquitectura que se volvería referencia en los años siguientes. Chahine et al. (2023) añaden una perspectiva complementaria sobre robustez fuera de distribución con redes neuronales líquidas como capa de control; Zhu et al. (2024) estudian específicamente hasta qué punto los modelos de lenguaje entienden el contexto que se les provee, una pregunta directamente relevante para el capítulo 9 de este trabajo, donde el problema no es la capacidad de razonamiento del modelo sino la integridad del contexto que efectivamente recibe.

La maduración del campo entre 2024 y 2026 fue rápida. Tian et al. (2025) publican la revisión sistemática más amplia disponible sobre la intersección de LLMs y UAVs —aceptada en *Information Fusion*—, organizando el espacio de soluciones en un marco Percepción–Cognición–Acción (P–C–A) y concluyendo que la disponibilidad de despliegue depende no solo de la capacidad del modelo sino de interfaces verificables que traduzcan salidas semánticas en representaciones accionables por los módulos de vuelo descendentes. Esta conclusión coincide con la motivación de diseño del capítulo 8 de este trabajo, que trata la decodificación restringida vía `json_schema` precisamente como esa interfaz verificable. Por su parte, la emergencia de los modelos de *Vision-Language-Action* (VLA) representa el paso siguiente: Sun et al. (2026) presentan AutoFly, un modelo VLA de extremo a extremo para navegación autónoma de UAVs en exteriores sin instrucciones detalladas —basado en un pseudo-codificador de profundidad a partir de RGB y entrenamiento en dos etapas—, logrando tasas de éxito de navegación 3,9 puntos porcentuales por encima del estado del arte anterior. Saxena et al. (2025) abordan un problema relacionado desde la perspectiva de la navegación por lenguaje: su sistema UAV-VLN integra un VLM para interpretar instrucciones en lenguaje natural y guiar la navegación de forma *end-to-end* sin entrenamiento supervisado por tarea. Estos desarrollos sitúan en perspectiva el trabajo aquí presentado: mientras la frontera investigativa de 2026 opera con VLA completos y modelos de varios miles de millones de parámetros, la pregunta que este trabajo responde es qué tan lejos puede llegar un SLM cuantizado de 1–4B parámetros ejecutado localmente —sin nube, sin entrenamiento especializado— como capa de decisión en un lazo de control real, y cuáles son sus modos de falla documentables.

Dos líneas de trabajo son especialmente pertinentes para la ingeniería de decisiones del SLM descrita en el capítulo 8. Por un lado, la generación de salida estructurada y restringida: Geng et al. (2025) sistematizan el estado del arte en generación de salidas estructuradas desde modelos de lenguaje —el problema exacto que resuelve, en este trabajo, la decodificación restringida vía `json_schema` con parser tolerante como red de contención (cap. 8)—, y Raspanti et al. (2025) muestran que la decodificación con gramática restringida mejora específicamente el desempeño en tareas de análisis lógico estructurado. Por otro lado, la simulación de alta fidelidad como banco de pruebas: Shah et al. (2017) introducen AirSim, el simulador sobre el que se desarrolla y valida este trabajo, y Jansen et al. (2023) presentan una variante extendida (Cosys-AirSim) para aplicaciones industriales complejas, evidencia de que AirSim sigue siendo, varios años después, una base activa para este tipo de investigación.

2.4 Dominios de aplicación de alta pertinencia local: vigilancia del litoral marítimo e incendios forestales

La demanda de autonomía robusta en UAVs que articula este trabajo no es solo un problema de investigación genérico: dos dominios aplicados de particular relevancia para Argentina muestran que los desafíos técnicos abordados —percepción monocular en tiempo real, evasión reactiva de obstáculos y decisión contextual de borde— son exactamente los requeridos en implementaciones operacionales inminentes.

Vigilancia del litoral marítimo. La Zona Económica Exclusiva argentina comprende aproximadamente 1,6 millones de km², una extensión imposible de cubrir de forma continua con medios tripulados a costo razonable. Argentina formalizó en mayo de 2026 un acuerdo con los Estados Unidos para incorporar drones V-Bat de Shield AI —sistemas VTOL que no requieren modificaciones en las embarcaciones desde las que operan— y aeronaves de patrulla marítima King Air B360 ER, bajo la *Maritime Domain Situational Awareness Initiative* (Infobae, 2026). En paralelo, una propuesta legislativa presentada en julio de 2026 plantea la creación de un Sistema Nacional de Vigilancia Marítima con drones, complementario a los despliegues navales tripulados (El Estratégico, 2026). La literatura académica específica sobre vigilancia marítima autónoma con UAVs en el litoral argentino es aún incipiente; sin embargo, el trabajo de Barišić Kulas et al. (2025) —presentado en ICUAS 2025— constituye una referencia técnica directa: los autores desarrollan un sistema completamente autónomo para detección e identificación de embarcaciones sobre grandes áreas marítimas con UAV en entornos de denegación de GNSS, empleando visión *onboard* con YOLOv8 y fusión de histogramas de matiz para identificar la embarcación objetivo bajo restricciones computacionales estrictas de borde. El problema técnico es isomorfo al de la patrulla costera: un vehículo autónomo debe procesar su entorno visual en tiempo real y tomar decisiones de seguimiento sin apoyo externo, en las mismas condiciones que este trabajo estudia en simulación.

Respuesta temprana a incendios forestales tras detección satelital. Los incendios en la Patagonia y el litoral fluvial representan uno de los riesgos ambientales y económicos más importantes del país. El Ministerio de Ambiente incorporó 17 UAVs para detección temprana de focos en parques nacionales de Neuquén y Chubut —incluyendo Lanín, Laguna Blanca, Los Alerce y Lago Puelo— con vuelo autónomo y transmisión en tiempo real a estaciones de tierra (Argentina.gob.ar, s.f.). La Provincia de Misiones avanza en un esquema más sofisticado: torres de videovigilancia con IA que, al detectar una columna de humo, programan automáticamente el vuelo de un dron autónomo hacia el foco para obtener coordenadas precisas antes de que el fuego se propague (Norte Misionero, 2026). La convergencia entre detección satelital y respuesta inmediata con UAVs es el tema de una línea de investigación en rápido crecimiento a nivel internacional. Liu y Sziranyi (2023) proponen una arquitectura de fusión que combina imágenes Sentinel-2 para localizar el foco activo con UAVs que realizan segmentación de caminos y planificación dinámica de rutas de evacuación en el área central del incendio, validada sobre el incendio de Chongqing de agosto de 2022. Danish et al. (2025), en una revisión sistemática exhaustiva publicada en *Artificial Intelligence Review*, mapean el estado del arte de los sistemas de gestión de incendios forestales con UAVs de visión artificial —desde detección hasta supresión— cubriendo más de siete años de producción científica. El trabajo más reciente en esta línea combina ambas fuentes en una arquitectura UAV-satélite: el pipeline usa datos históricos satelitales para priorizar zonas de patrulla y lanza múltiples UAVs de forma coordinada para detección temprana y predicción dinámica del perímetro, alcanzando un F1-score del 97,4 % con consumo energético minimizado para maximizar la autonomía de vuelo (*Drones*, 10(4), 263; DOI: 10.3390/drones10040263).

En ambos dominios, el patrón técnico subyacente —un vehículo autónomo que procesa información perceptual en tiempo real, toma decisiones sin intervención humana y actúa bajo restricciones de cómputo de borde— es exactamente el que este trabajo estudia y evalúa en entorno de simulación. La ausencia de literatura académica argentina específica no resta relevancia a los resultados; al contrario, señala una oportunidad de transferencia directa de los métodos y hallazgos documentados en los capítulos siguientes hacia dominios de alto impacto civil local.

2.5 Posicionamiento de este trabajo

Este trabajo se ubica en la intersección de esas tres líneas —percepción monocular sin redes neuronales, control por FSM como línea de base, y un SLM con salida restringida como capa de decisión alternativa— pero con un énfasis distinto al de buena parte de la literatura revisada. Mientras la frontera de investigación de 2025-2026 (Tian et al., 2025; Sun et al., 2026) opera con modelos VLA de gran escala y múltiples miles de millones de parámetros en hardware especializado, este trabajo aborda deliberadamente el extremo opuesto del espectro: un SLM cuantizado de 1-4B parámetros ejecutado en CPU de borde, sin nube, sin entrenamiento por tarea. En lugar de proponer una arquitectura novedosa de percepción o de razonamiento, documenta con evidencia medida los modos de falla específicos de integrar un modelo de lenguaje pequeño en un lazo de control real —un tipo de resultado poco frecuente en la literatura publicada, que tiende a reportar arquitecturas que funcionan más que arquitecturas que fallaron de manera instructiva y cómo se detectó y corrigió esa falla—. Esta contribución es especialmente pertinente en el contexto de las aplicaciones civiles y militares descritas al inicio del capítulo: los escenarios de mayor relevancia práctica —drones de inspección en zonas sin cobertura, vehículos de rescate en áreas de desastre, plataformas tácticas en entornos de guerra electrónica— son precisamente aquellos en que los modelos de nube y el hardware de alto costo son inviables, y donde entender los límites y modos de falla del SLM de borde tiene valor operacional directo.

3. Construcción y validación del entorno de simulación

3.1 Arquitectura de simulación: Unreal Engine 5.5 y Cosys-AirSim

El desarrollo y la validación de este trabajo se realizan, tal como preveía el plan de trabajo aprobado (`plan_tesis/plan-tesis.md`), sobre **AirSim**, un simulador basado en Unreal Engine que ofrece física y renderizado de alta fidelidad (Shah et al., 2017). La implementación efectiva, sin embargo, no usa la distribución original de AirSim de Microsoft: desde el inicio del proyecto se adoptó **Cosys-AirSim** (específicamente el build para Windows de la versión [Cosys-AirSim v3.3 for Unreal v5.5](#), un fork mantenido por el Cosys-Lab (Laboratorio de Co-Diseño para Sistemas Ciber-Físicos de la Universidad de Amberes, Bélgica). Al inicio del desarrollo se adoptó la decisión de utilizar este fork: *"Abandonado el proyecto original AirSim por Microsoft, se utiliza la actual versión a partir de un fork mantenido por el Cosys-Lab"*, compilado e integrado sobre proyectos de Unreal Engine 5.5 (específicamente la versión [5.5.4-40574608++UE5+Release-5.5](#)) que sirvieron como banco de pruebas y validación experimental.

Cosys-AirSim, a diferencia del AirSim clásico de Microsoft —enfocado principalmente en cámaras RGB y visión por computadora—, agrega sensores adicionales basados en GPU y CPU (LiDAR, sonar, radar), documentados en el paper oficial de la plataforma (Jansen et al., 2023). Este trabajo no usa esos sensores adicionales: la percepción descrita en el capítulo 6 depende exclusivamente de la cámara RGB monocular y de la telemetría de actitud, consistente con la restricción de hardware de bajo costo que motiva todo el proyecto (§1.2). La elección de Cosys-AirSim sobre el AirSim original responde, de todos modos, a la actividad de mantenimiento del fork más que a una necesidad de esos sensores adicionales: hacia 2025-2026 el proyecto de Microsoft dejó de recibir actualizaciones activas, y la documentación, configuración y versiones del fork de Cosys-Lab fueron revisadas exhaustivamente antes de adoptarlo.

Particularidades de la API y telemetría de actitud. Una diferencia relevante descubierta durante la integración radica en la interfaz en Python: a diferencia del paquete clásico `airsim`, el binding `cosysairsim` no expone la función auxiliar `to_eularian_angles` para la extracción directa de ángulos de actitud. La ausencia de este método provocó que los fallbacks iniciales reportaran *pitch* y *roll* en 0.0° de forma permanente. Para subsanar esta omisión sin introducir dependencias externas complejas, se implementó en el cliente de simulación (`AirSimClient._quaternion_to_euler`) la conversión matemática directa de cuaterniones a ángulos de Euler en convención aeroespacial NED (North-East-Down), asegurando que las variaciones de inclinación del cuadricóptero alimenten con fidelidad continua la telemetría del lazo táctico y el contexto del modelo deliberativo.

3.2 Desvío respecto del plan aprobado: de la fotogrametría de Buenos Aires a activos urbanos genéricos

El plan de trabajo aprobado describía un pipeline de digitalización de entornos urbanos específico: extraer fotogramas de videos públicos de drones en Buenos Aires (YouTube), filtrarlos para asegurar diversidad visual, anotar obstáculos y zonas de aterrizaje con LabelImg/LabelMe, y reconstruir modelos 3D hiperrealistas con RealityCapture (fotogrametría) a partir de esos videos y de OpenStreetMap, optimizados en Blender para reducir su complejidad computacional antes de importarlos a AirSim (`plan_tesis/plan-tesis.md`, §"Metodología"). El objetivo declarado era un entorno virtual 3D *personalizado* de Buenos Aires, con las particularidades geométricas de su infraestructura densa.

La implementación efectivamente construida sustituyó ese pipeline de fotogrametría por activos urbanos y suburbanos ya disponibles para Unreal Engine (marketplace de Fab/Epic Games y paquetes de terceros), configurados con el plugin de Cosys-AirSim en lugar de reconstruidos a partir de video real de Buenos Aires (disponibles en el repositorio compartido de Google Drive: <https://drive.google.com/drive/folders/1roLmbGFNsHXZyT3NaNzNYMuaBQ8CuIX7>).

Entornos principales estructurados por Tiers de evaluación

Para responder con rigor al protocolo experimental de la tesis (capítulo 10), los entornos de simulación se organizaron en una **jerarquía formal de tres niveles de complejidad (Tiers)**, superando los primeros borradores preliminares de misión (`manhattan_a` y `manhattan_b`, descartados y alineados a esta nueva nomenclatura):

1. **Tier 0 — Base / Control (MiniSim)**: Entorno abierto basado en el mapa `crater.png` (basado en el **proyecto base de Unreal Engine** https://dev.epicgames.com/documentation/unreal-engine/unreal-engine-templates-reference?application_version=5.5). Diseñado para pruebas de calibración cinemática, sintonización de controladores de posición y determinación de la cota inferior de latencia del sistema sin interferencia de obstáculos.
2. **Tier 1 — Intermedio / Morfología orgánica y vegetación (TownSim)**: Entorno semiurbano desarrollado sobre `townsim.png` y refinado en el mapa calibrado `townsim_calib.png`, basado en el entorno **Downtown West Modular Pack** <https://www.fab.com/listings/0faf8b5d-7a5f-4fee-a297-7a8efaba8896> de PurePolygons. Presenta calles de ancho variable, fachadas intermedias, mobiliario y arboledas densas. Constituye el banco de pruebas central para la batería de calibración `T-CALIB-0` a `T-CALIB-5` y la misión de recorrido perimetral `townsim_ini`, albergando el escenario de bloqueo frontal masivo genuino (`T-CALIB-2`).

3. **Tier 2 — Complejo / Cañones urbanos de gran densidad (CitySim)**: Proyecto de Unreal Engine 5.5 con tejido edilicio masivo (`citysim_calib.png`), rascacielos, pasos restringidos y disposición en cuadrícula ortogonal, basado en el entorno **Sample City** <https://www.fab.com/listings/4898e707-7855-404b-af0e-a505ee690e68> de Epic Games. El escenario base validado para el batch G4 es `citysim_clear.json`: perímetro de manzana con patrón *climb-first*, ascenso vertical puro hasta -70 m AGL antes de moverse horizontalmente. La altitud de tránsito de -70 m es un dato de diseño del entorno: la primera corrida a -50 m falló porque la ruta de ascenso atravesaba edificios; a -70 m el dron vuela por encima de la línea de tejados. Los escenarios con corredores angostos (`citymap_pilot.json` y `citymap_a.json`) corresponden a la fase siguiente del batch.

3.3 Configuración de rendimiento, escalabilidad gráfica e higiene de captura

La documentación técnica de Cosys-Lab sostiene que priorizar la tasa de actualización de la física por sobre la fidelidad gráfica es una práctica recomendada cuando el objetivo principal no es la fotografía cinemática de la escena. Siguiendo esas directrices y considerando que la misma estación de cómputo aloja en simultáneo el renderizado 3D y la inferencia del modelo de lenguaje local (§8.1), se establecieron los siguientes compromisos de arquitectura:

- Desactivar el ahorro de CPU en segundo plano del editor de Unreal Engine (`Use Less CPU when in Background`), evitando caídas drásticas en el refresco de la física cuando la ventana del simulador pierde el foco.
- Ajustar `ClockSpeed` en `settings.json` para sincronizar el tiempo de simulación y garantizar que el lazo físico se calcule sin pérdidas temporales cuando la carga gráfica fluctúa.
- Preferir binarios empaquetados (*standalone*) sobre la ejecución directa desde el editor, reduciendo la sobrecarga de memoria VRAM asociada a la interfaz de desarrollo.
- El modo `NoDisplay` no pudo utilizarse, dado que la captura de imágenes RGB monocular es estrictamente indispensable para la percepción del dron (capítulo 6).

Escalabilidad gráfica en Unreal Engine 5.5

Para equilibrar la carga de GPU entre Unreal Engine y el servidor SLM/VLM local, se definió un perfil de escalabilidad mínima (*Minimal Scalability Config*) que fija sombras, postprocesamiento e iluminación global en niveles bajos (`Low` / `Medium`), reservando la VRAM necesaria para los pesos del modelo multimodal.

Sin embargo, en Unreal Engine 5.5 la reducción automática de efectos suele degradar la resolución de los render targets de captura de escena. Para evitar que la cámara frontal pierda nitidez o altere el cálculo de flujo óptico, se forzó el nivel de detalle en el archivo `Config/DefaultScalability.ini` del proyecto Unreal:


```
[EffectsQuality@0]
r.DetailMode=2
[EffectsQuality@1]
r.DetailMode=2
[EffectsQuality@2]
r.DetailMode=2
[EffectsQuality@3]
r.DetailMode=2
[EffectsQuality@Cine]
r.DetailMode=2
```

También desde el editor de Unreal Engine se puede configurar la escalabilidad de la siguiente manera:



Con esta configuración Scalability de Unreal Engine se asegura a la hora de ejecutar la simulación: - Captura de video desde AirSim (con `Effects=Epic`) - Sombras en las texturas necesarias para capturar los gradientes usados para el cálculo del TTC con el flujo visual (con `Shadows=Epic`).

Higiene y consistencia de la captura monocular

Durante la experimentación continua en simulación se identificaron y resolvieron dos anomalías instrumentales críticas en el canal visual:

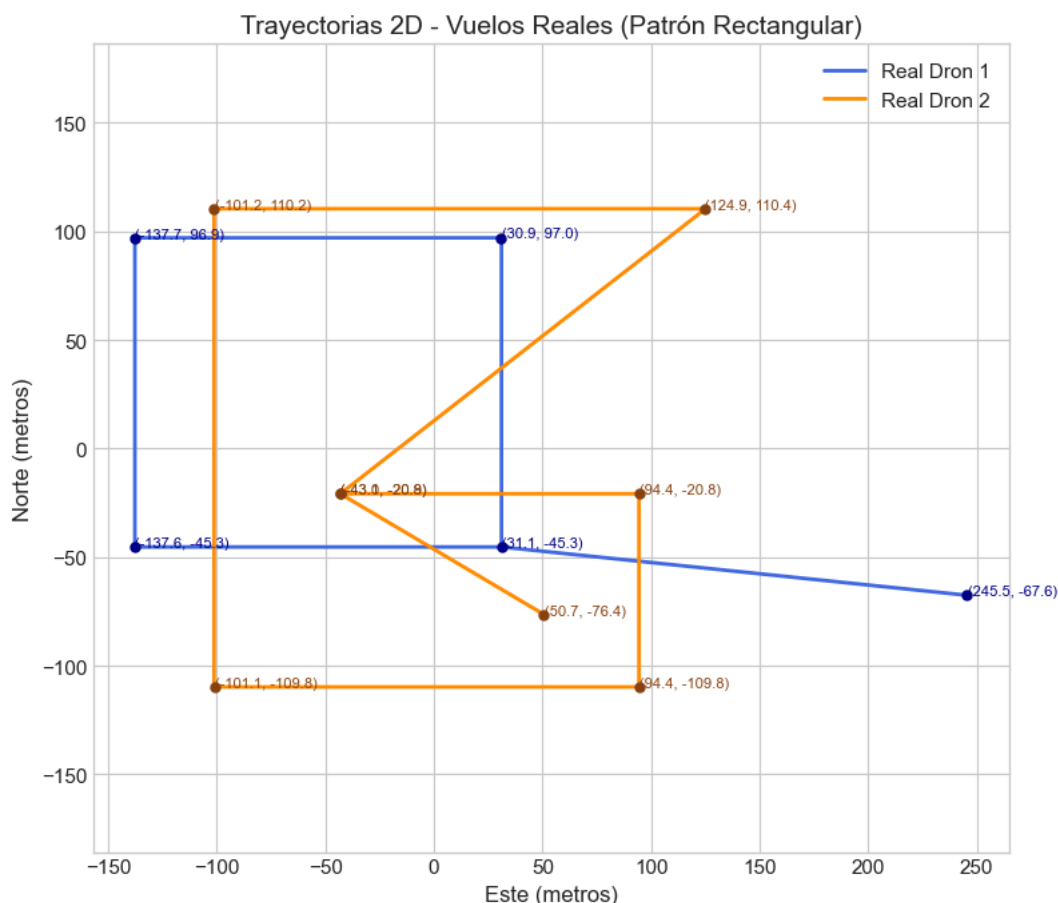
1. **Inversión de canales de color (RGB vs. BGR):** El buffer crudo devuelto por `simGetImages(ImageType.Scene)` en Cosys-AirSim es entregado en formato RGB. No obstante, el ecosistema de procesamiento de OpenCV y las rutinas de compresión asumían internamente la convención BGR. Como consecuencia, las capturas enviadas al VLM y guardadas en auditoría mostraban una tonalidad azulada con rojo y azul invertidos. La anomalía se corrigió de forma definitiva en el propio método `AirSimClient.capture()`, asegurando que el modelo de lenguaje razone sobre los colores naturales de la escena (pavimento, vegetación y cielo).
2. **Supresión de marcadores de depuración persistentes:** Primitivas geométricas dibujadas en el mundo virtual por scripts de planificación (mediante `simPlotLineStrip` y `simPlotPoints` de `plot_mission_route.py`) permanecían activas en el nivel y eran capturadas por la cámara a bordo, siendo interpretadas por el VLM como obstáculos físicos reales. Se implementó una rutina de higiene obligatoria (`AirSimClient.clear_debug_markers()`), invocando `simFlushPersistentMarkers()` al inicio de cada sesión de conexión.

3.4 Validación frente a telemetría de vuelos reales

La dinámica de vuelo simulada —no la geometría de la escena urbana, cuya fidelidad fotográfica es el desvío documentado en §3.2— se validó de forma independiente contra telemetría de drones reales, con el objetivo explícito de que la comparación experimental entre brazos (capítulo 10) se apoyara en vuelos simulados cuya cinemática fuera comparable a la de un cuadricóptero real y no solo físicamente plausible en abstracto.

Fuente de datos reales. Se descargó un conjunto de datos de telemetría real de drones cuadricópteros comerciales (DJI) desde el repositorio público de Zenodo (<https://zenodo.org/records/15912415>).

Protocolo de calibración. Sobre esa telemetría real se identificaron dos trayectorias de vuelo distintas —un patrón rectangular simple (Drone 1) y un patrón de cruz combinado con rectángulo (Drone 2)— y se reprodujeron en simulación con `moveOnPath` / `move` de la API de AirSim, ajustando la simulación a la misma altura y la misma velocidad que los vuelos reales correspondientes. El proceso se iteró varias veces a lo largo del proyecto: una primera generación automatizada de telemetría de 100 vuelos simulados (2026-0413) para poner a punto el script de iteración; una serie de 10 vuelos de calibración con telemetría registrada en archivos CSV individuales (2026-0409); una repetición con trayectorias explícitamente ajustadas a la altura y velocidad de los vuelos reales (2026-0610); y una iteración final (2026-0627/2026-0628) que reemplazó los comandos de movimiento discretos por `moveOnPath` con la trayectoria completa como un único comando, para acercar el perfil de aceleración simulado al de un vuelo real continuo en lugar de una secuencia de tramos independientes.

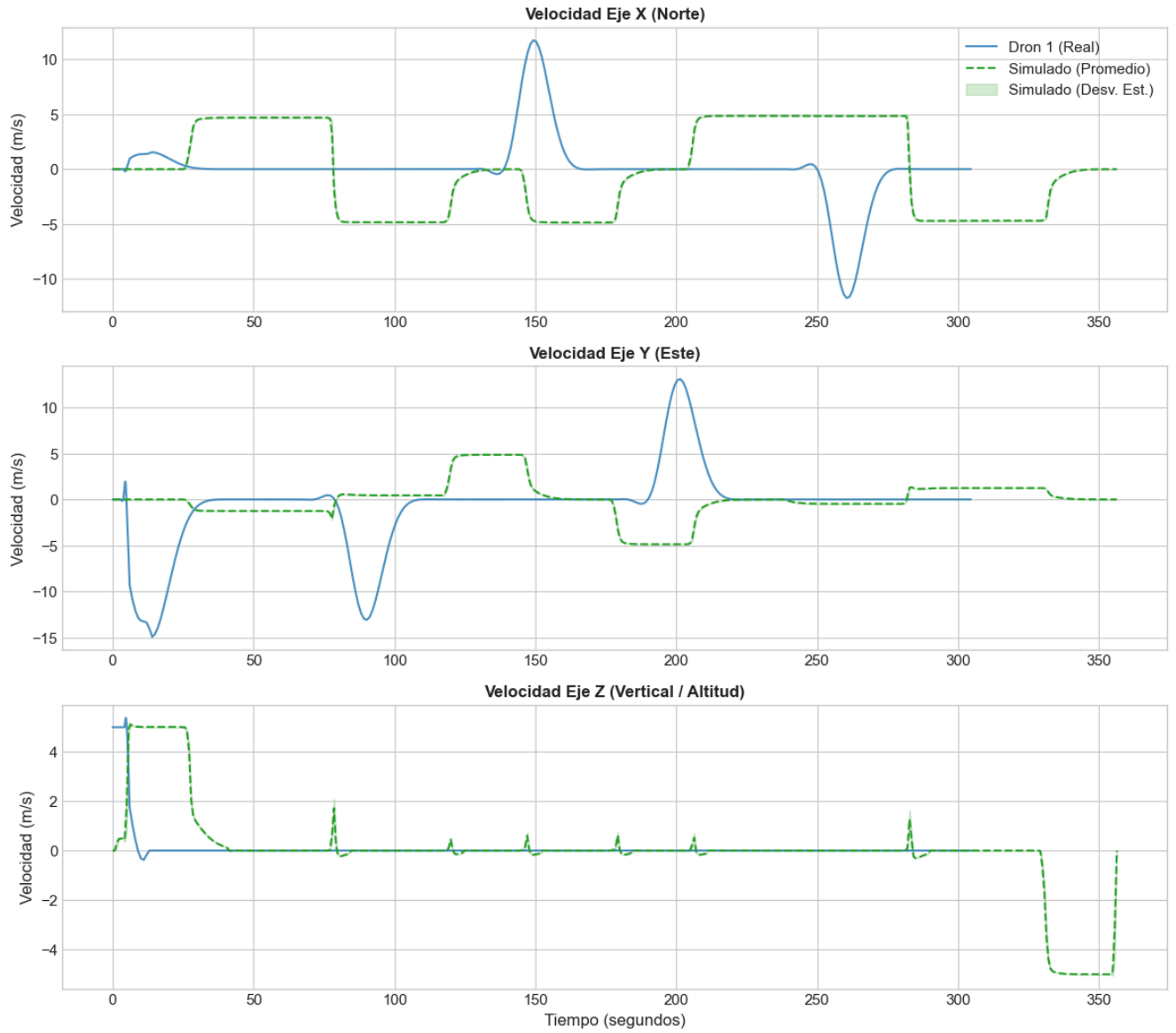


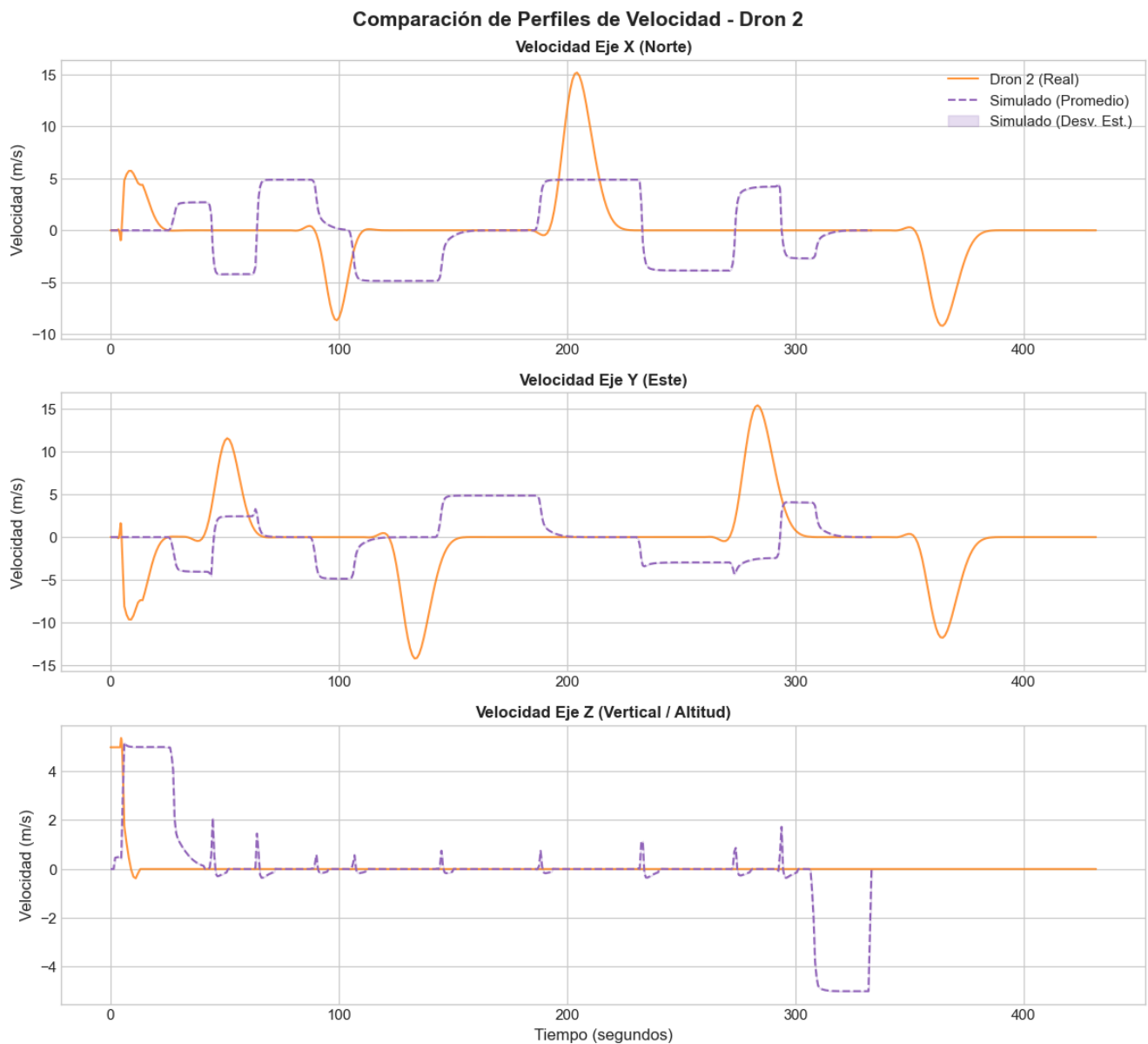
Metodología estadística. El análisis se implementó en notebooks de Jupyter del repositorio complementario del proyecto (`consolidate_telemetry.ipynb` y `telemetry_analysis_*.ipynb`, [georgsmeinung/lm-drone](#)), con estadística descriptiva y pruebas de significancia —en particular la prueba de Levene para diferencia de varianzas— aplicadas al cambio de velocidad en los tres ejes, para determinar si la distribución de la telemetría simulada difiere de forma significativa de la real.

Resultado. Las corridas de análisis (2026-0604 a 2026-0628) documentan tres hallazgos consistentes:

1. **La varianza de actitud difiere de forma significativa** (prueba de Levene, $p \ll 0,05$): en simulación, el dron ejecuta inclinaciones de *roll/pitch* extremas durante los giros rápidos para alcanzar instantáneamente el punto de trayectoria siguiente. Los drones reales, en cambio, están limitados electrónicamente por su controlador PID de estabilización (típicamente a $\pm 30^\circ$), y muestran una varianza mucho menor y acotada durante las mismas maniobras.
2. **La segregación por trayectoria es significativa y consistente.** El Drone 1 (patrón rectangular, giros menos frecuentes) concentra la aceleración en las esquinas; el Drone 2 (patrón de cruz y rectángulo continuo) presenta una dinámica transicional más exigente y oscilaciones de *roll/pitch* más ruidosas, tanto en el vuelo real como en el simulado.
3. **El vuelo simulado en tramos rectos es idealizado.** Durante las fases rectas, la telemetría simulada muestra varianza de actitud cercana a cero, sin fuerzas externas de viento ni ruido de sensores; el dron real, en cambio, mantiene una variabilidad permanente de $\pm 2^\circ$ – 3° en *roll* y *pitch* incluso en tramos rectos estables, producto del viento real y de las correcciones continuas del piloto automático.

Comparación de Perfiles de Velocidad - Dron 1





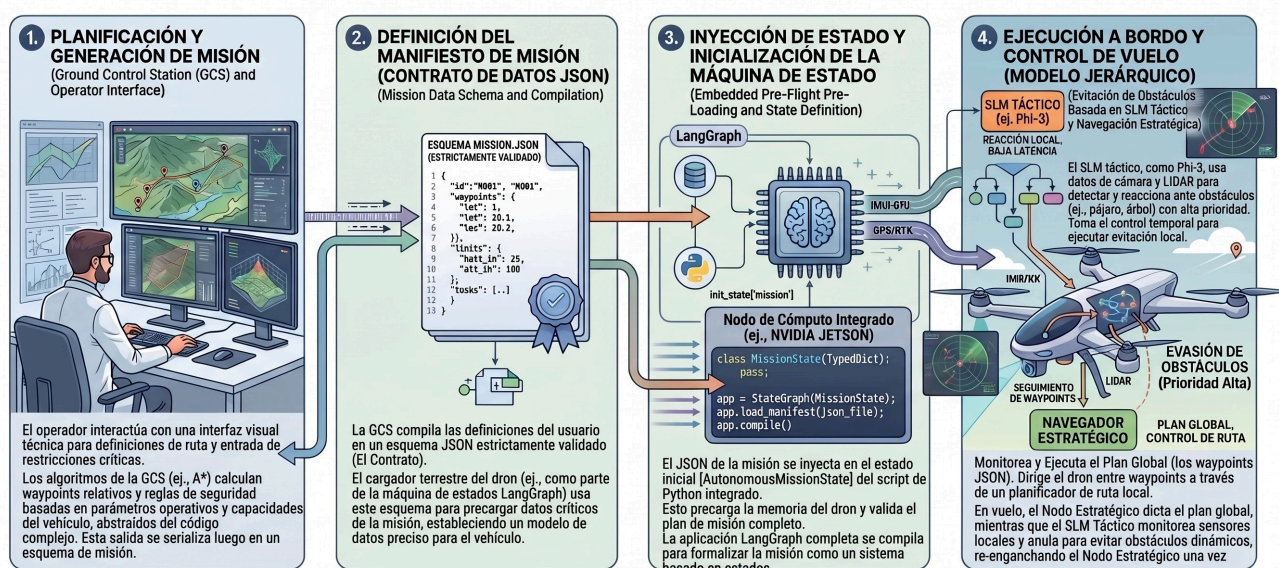
Alcance de esta validación. Confirma que la dinámica de vuelo simulada —cómo se mueve el dron dado un comando— es más ágil y menos amortiguada que la de un dron real equivalente, con una brecha de idealización conocida y cuantificada (varianza de actitud, ausencia de viento simulado). No valida, en cambio, la fidelidad fotográfica de la escena urbana frente a Buenos Aires real: esa es una validación distinta, no realizada, y coincide con el desvío documentado en §3.2. Un ejercicio de validación relacionado pero diferente —si el suavizado por histéresis y media móvil exponencial introducido en `WaypointTracker` (capítulo 5, §5.3) mejora medible la telemetría real de vuelo— arrojó un resultado honestamente ambiguo (`std( $\Delta v_x$ )` prácticamente plano antes/después) y se documenta como tal en el presente informe, no como una mejora confirmada.

4. Planificación de misiones y estación de control en tierra (WebDCS)

4.1 Arquitectura desacoplada: dos cerebros (tierra vs. vuelo)

La arquitectura general del sistema responde a un principio de diseño fundamental: la separación estricta entre la **planificación deliberativa global previa al vuelo** (ejecutada en tierra) y la **navegación táctica reactiva** (ejecutada a bordo durante el vuelo). Esta división, conceptualizada como una arquitectura de *dos cerebros*, resuelve la asimetría intrínseca de recursos computacionales y tolerancias de latencia entre ambas fases operativas:

1. **El planificador de estación terrena (`airsim-plan` / `WebDCS`)**: opera de manera previa al despegue del vehículo. En esta etapa, el sistema puede tolerar presupuestos de tiempo del orden de 5 a 10 segundos para interpretar instrucciones en lenguaje natural, consultar información contextual del entorno, aplicar reglas de seguridad operacional y generar un plan de vuelo global validado.
2. **El lazo táctico a bordo (`airsim-loop`)**: opera en tiempo real estricto una vez que el vehículo está en el aire (capítulo 5). Bajo una frecuencia objetivo de 5 a 10 Hz, su función no es diseñar la ruta global sino mantener la estabilidad cinemática, seguir los objetivos asignados y ejecutar maniobras reactivas inmediatas de evasión de obstáculos a partir de visión monocular ligera.



El desacoplamiento permite que la misión sea validada, persistida y versionada de forma determinista antes de iniciar los motores. Asimismo, garantiza que el lazo táctico a bordo no dependa de una conexión de red permanente con la estación terrena: una vez transmitido el manifiesto de vuelo, el dron opera de forma enteramente autónoma, protegiendo la seguridad del vuelo ante eventuales pérdidas de enlace de radio o telemetría.

4.2 Compilación de misiones desde lenguaje natural

El módulo de planificación en tierra integra un modelo de lenguaje (LLM) que actúa como compilador semántico. Su objetivo es transformar directivas operativas de alto nivel expresadas en lenguaje natural por un operador humano (por ejemplo, *"recorrer el perímetro norte a 10 metros de altura evitando zonas de alta densidad vehicular"*) en un artefacto estructurado y ejecutable por el piloto automático.

Para garantizar la fiabilidad del proceso de compilación sin incurrir en alucinaciones o errores de sintaxis:

- Se utiliza un cliente local compatible con la API de OpenAI (conectado a instancias locales de LM Studio u Ollama), ejecutando modelos orientados a seguimiento de instrucciones (tales como Llama-3-8B, Qwen-7B o Phi-4).
- Se define un *System Prompt* formalizado (`compiler_system.md`) que instruye al modelo sobre las dimensiones del entorno de simulación, las restricciones del espacio aéreo urbano y el formato de salida requerido.
- Se implementa una capa de coerción y extracción de JSON (`json_extract.py`) respaldada por esquemas de validación estricta en **Pydantic** (`manifest.py`). Si la salida generada por el LLM no cumple con los tipos de datos o las restricciones cinemáticas impuestas, el compilador rechaza el plan y solicita una reevaluación antes de comprometer el vuelo.

Cabe una aclaración sobre el alcance actual de la interfaz: si bien el compilador semántico descrito en esta sección está completamente implementado y expuesto como endpoint del backend, la interfaz de WebDCS en su estado presente no ofrece ningún control para invocarlo. La superficie de interacción del operador quedó acotada, por decisión de alcance del proyecto, a la edición manual de waypoints sobre la carta de navegación (§4.4.1) — priorizando el desarrollo del control de vuelo por sobre la interfaz de planificación asistida. La compilación desde lenguaje natural queda documentada aquí como una capacidad ya construida y validada, disponible para reincorporarse a la interfaz sin cambios en el backend.

4.3 El contrato del Manifiesto de Misión (`MissionManifest`)

El producto final del planificador terrestre es un archivo JSON denominado `MissionManifest`. Este documento actúa como un contrato formal e inmutable entre la estación terrena y el lazo táctico de vuelo. Su schema canónico está definido en Pydantic (`manifest.py`) y validado contra un JSON Schema (`manifest_schema.json`).


```
{
  "mission_id": "CITYSIM_CLEAR_01",
  "summary": "Patrón de patrullaje urbano sobre cuadrícula con 7 waypoints.",
  "waypoints": [
    {"x": 26.1, "y": -78.2, "z": -10.0, "label": "WP_1"},
    {"x": 68.9, "y": -78.6, "z": -10.0, "label": "WP_2"},
    {"x": 67.7, "y": -145.0, "z": -10.0, "label": "WP_3"},
    {"x": 26.5, "y": -145.0, "z": -10.0, "label": "WP_4"},
    {"x": -16.3, "y": -145.0, "z": -10.0, "label": "WP_5"},
    {"x": -17.9, "y": -78.6, "z": -10.0, "label": "WP_6"},
    {"x": 12.5, "y": -78.6, "z": -10.0, "label": "WP_7"}
  ],
  "map": "citysim_calib.png"
}
```

Componentes del contrato:

- **mission_id**: identificador único en formato `^[A-Z0-9_]{3,32}$`, validado por el schema. Identifica el escenario experimental y aparece en los nombres de carpeta de telemetría.
- **summary**: descripción textual de la intención operativa (campo opcional, no consumido por el lazo táctico).
- **waypoints**: lista ordenada de puntos de paso tridimensionales en el marco **NED** (*North-East-Down*): $z < 0$ representa altitud sobre el punto de despegue. Cada waypoint lleva una etiqueta opcional (`label`) para identificación en la traza.
- **map**: nombre del archivo de imagen de carta de territorio empleado por WebDCS para la visualización de la trayectoria (por defecto `map.png`).

Separación entre manifiesto y prompt táctico. El manifiesto entrega únicamente metas geométricas; el prompt del nodo deliberativo no forma parte de él porque no puede ser un texto estático: se construye dinámicamente en cada ciclo de vuelo a partir del estado percibido en ese instante. Su estructura es la siguiente:

System prompt (estático, fijo en `deliberative.py`): define el rol del modelo, las reglas de navegación urbana y el conjunto cerrado de macro-acciones posibles. Existen dos variantes — `SYSTEM_PROMPT_TEXT` (solo texto, para SLMs sin capacidad visual) y `SYSTEM_PROMPT_VISION` (texto + imágenes, para VLMs multimodales) — seleccionadas en tiempo de ejecución por la variable de entorno `VLM_VISION_ENABLED`.

User prompt (dinámico, construido por `_build_user_prompt()` en cada ciclo): combina cinco componentes:

1. **Resumen del ObstacleField:** estado de los tres sectores de percepción (centro, izquierda, derecha), con TTC estimado y nivel de ocupación por sector.
2. **Objetivo y altitud:** waypoint activo (etiqueta, distancia horizontal, error de rumbo en grados y dirección relativa), altitud actual y cota segura de operación.
3. **Estado cinemático:** velocidad horizontal y actitud (pitch, roll) en el ciclo actual. Si el dron está prácticamente detenido (`< 0.3 m/s`), se indica explícitamente — porque un frame estático sin

traslación no aporta evidencia de flujo óptico, y sin ese contexto el modelo tiende a sobre-interpretar la imagen.

4. **Motivo de consulta:** distingue si se consulta al VLM porque el sector central registra un obstáculo real (con TTC medido) o porque la percepción no tiene evidencia suficiente en este ciclo (confianza baja por falta de traslación o rotación reciente) — dos causas con implicaciones muy distintas para la decisión.
5. **Historial reciente:** las últimas tres deliberaciones con su macro-acción y sus resultados medidos (variación de distancia al waypoint, variación del TTC mínimo), para que el modelo pueda evaluar si una estrategia previa funcionó antes de repetirla.

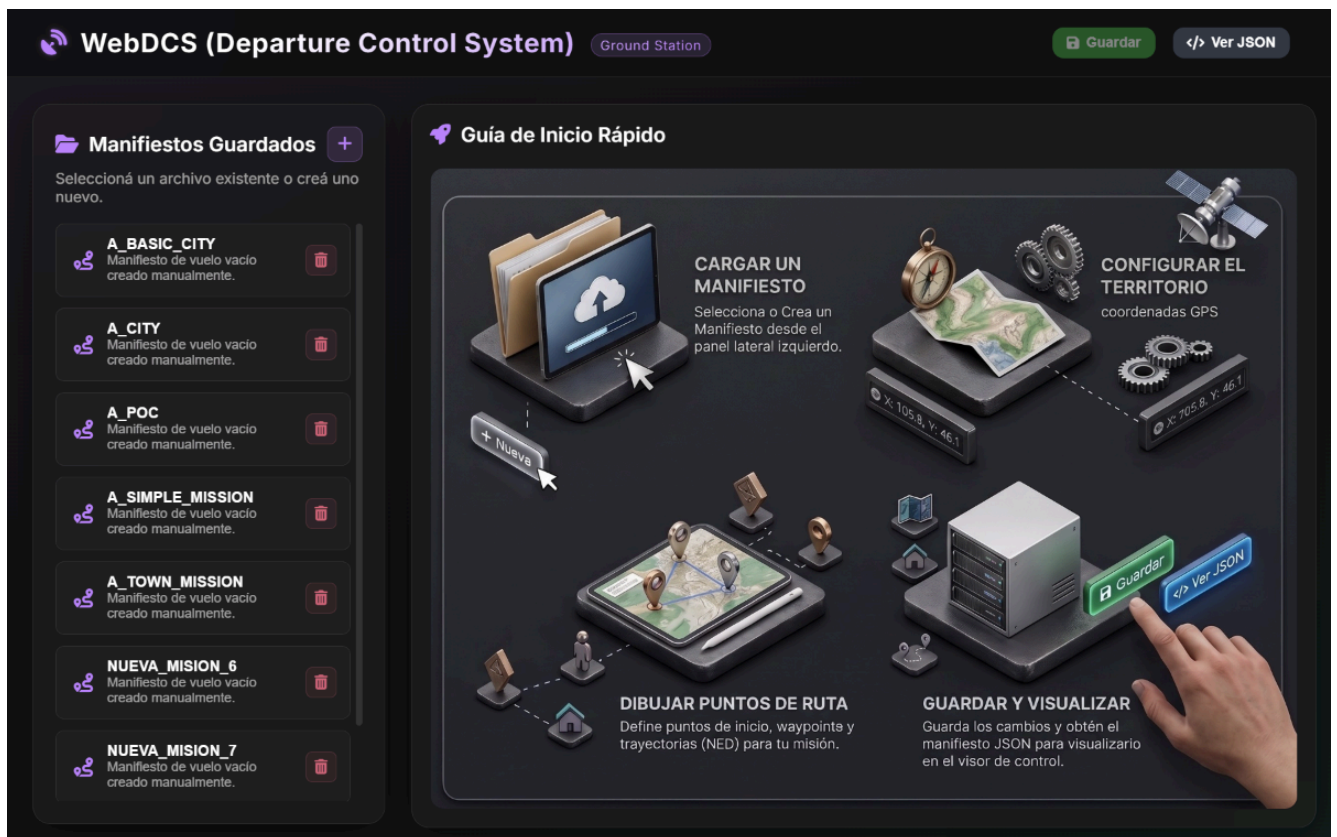
La respuesta del modelo se fuerza mediante decodificación restringida (`json_schema`, si el servidor lo soporta) al formato `{"macro_action": "<ACCION>", "rationale": "<texto>"}`, con temperatura 0.2 y límite de 200 tokens. Si el servidor no soporta `json_schema`, un parser tolerante (`_parse_decision()`) extrae la decisión del texto libre como red de seguridad. Ante timeout del watchdog o respuesta no parseable, se activa una heurística determinista sobre el `ObstacleField` (`_fallback_decision()`).

Este nivel de dinamismo hace inviable delegar el prompt al manifiesto: el contenido útil depende de percepciones que solo existen en vuelo.

En el marco de la metodología experimental de esta tesis (capítulo 10), el uso de manifiestos estructurados garantiza la **reproducibilidad experimental**: los escenarios de benchmark por Tiers (`minisim_clear`, `townsim_ini`, `citysim_clear`) se definen a través de manifiestos fijos e inmutables, asegurando que las comparaciones de rendimiento entre brazos de control (SLM, FSM y reactivo) se inicien exactamente con las mismas metas cinemáticas y espaciales.

4.4 Estación Terrena WebDCS: planificación y auditoría post-vuelo

WebDCS (*Web-based Drone Control Station*) es la interfaz de control en tierra, construida sobre **FastAPI** con frontend en HTML5, CSS y JavaScript. Expone su API REST en `http://localhost:8000` y sirve el panel de operación como aplicación web estática.



Funcionalidades de WebDCS:

- Gestión interactiva de cartas y waypoints:** El panel incorpora un visor Canvas 2D que superpone la trayectoria del manifiesto sobre la carta del entorno urbano (`CitySim`). El operador puede cargar manifiestos existentes desde `airsim-plan/missions/flightplans/`, visualizar la ruta proyectada y editar waypoints interactivamente sobre la carta — agregar, reordenar, eliminar y ajustar altitud — antes de guardar o lanzar.
- Lanzamiento y detención de misión:** El botón *Lanzar* invoca `POST /api/launch`, que persiste el manifiesto y arranca `airsim-loop` en un hilo de fondo sin bloquear la interfaz. `POST /api/stop` detiene todos los runners activos. `POST /api/reset` reinicia el simulador AirSim y libera el control de la API.

Dataset de corridas: estructura de `airsim-runs/`

Cada ejecución de una misión escribe una carpeta autocontenida en `airsim-runs/`. El nombre de la carpeta — y el prefijo común (`<stem>`) de todos los archivos dentro de ella — se construye concatenando el `mission_id` del manifiesto con la marca de tiempo de inicio de la ejecución en UTC, en formato ISO 8601 compacto sin separadores: `<MISSION_ID>-<YYYYMMDDTHHMMSSZ>`. Por ejemplo, lanzar la misión `CITYSIM_CLEAR` el 7 de septiembre de 2026 a las 19:12:26 UTC produce la carpeta y el stem `CITYSIM_CLEAR-20260907T191226Z`. Si la misma misión se lanza dos veces, cada ejecución queda en su propia carpeta gracias al timestamp — nunca se sobrescriben corridas anteriores. La tabla siguiente describe los archivos que produce cada ejecución:

Archivo	Descripción
<code><stem>.jsonl</code>	Traza completa: un objeto JSON por ciclo de control
<code><stem>.csv</code>	Mismas columnas en formato plano, para inspección directa en Excel / pandas
<code><stem>.summary.json</code>	Métricas agregadas de la corrida (éxito, colisiones, latencias, tasas del VLM)
<code><stem>.summary_by_wp.csv</code>	Stats por waypoint: ciclos, deliberaciones y eventos de atasco por tramo
<code><stem>.viewer.html</code>	Visor de auditoría HTML autocontenido, sincronizado con el video
<code><stem>.webm</code>	Video anotado del vuelo (un fotograma por ciclo, con overlay de acción y estado)
<code>photo-<ISO_UTC>.png</code>	Fotogramas enviados al VLM, uno por deliberación, nombrados por timestamp UTC

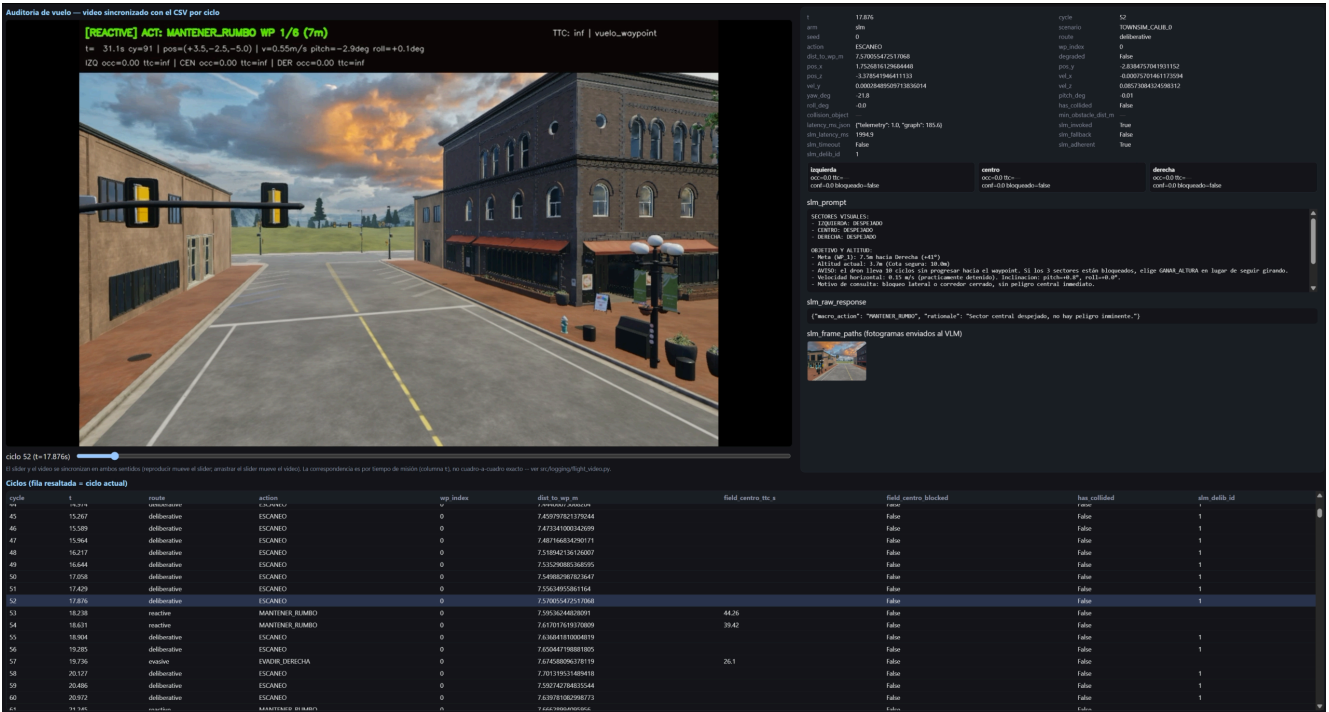
Traza por ciclo (`.jsonl` / `.csv`): cada entrada registra el instante, el ciclo, el brazo de control (`arm: slm/fsm/reactive`), el escenario, la ruta, la macro-acción, el waypoint activo y la distancia al mismo, la posición y velocidad 3D, la actitud, el estado de colisión y la distancia mínima al obstáculo. En los ciclos deliberativos se agregan el prompt enviado (`slm_prompt`), la respuesta cruda del modelo (`slm_raw_response`), la latencia de inferencia, indicadores de fallback y timeout, y las rutas a los fotogramas enviados (`slm_frame_paths`). El estado del `ObstacleField` se registra por sector (ocupación, TTC, confianza, bloqueado).

Resumen de corrida (`.summary.json`): es el archivo que consume `analyze.py` para el análisis comparativo entre brazos. Incluye éxito de misión, colisiones, distancia mínima al obstáculo, longitud de trayectoria, tasa de deliberación, y tasas de fallback y timeout del VLM.

Resumen por waypoint (`.summary_by_wp.csv`): permite identificar en qué tramo de la misión se concentra la dificultad sin necesidad de analizar el CSV completo — muestra ciclos consumidos, proporción de ciclos deliberativos y eventos de escalación de atasco por waypoint.

Fotogramas del VLM (`photo-<ISO>.png`): cada imagen enviada al modelo se guarda con su timestamp real de captura en UTC (precisión de milisegundos), que sirve como clave para cruzar con la columna `slm_frame_paths` de la traza. La carpeta es autocontenida: toda la evidencia de una corrida reside en un único directorio.

Visor de auditoría (.viewer.html)



El archivo `<stem>.viewer.html` es un documento HTML autocontenido generado al cierre de cada corrida. Sincroniza en ambos sentidos la reproducción del video con la traza:

- Avanzar el video actualiza automáticamente el panel de detalle del ciclo correspondiente, mostrando el prompt enviado, la respuesta del modelo, la macro-acción resultante y el fotograma capturado en ese instante.
- Seleccionar una fila de la tabla salta el video al instante correspondiente.

Para consultarlo: abrir el archivo directamente en Chrome desde `airsim-runs/<carpeta>/<stem>.viewer.html` — no requiere servidor.

Anotaciones del video

El video (.webm) registra un fotograma por ciclo del lazo, a la misma cadencia de `LOOP_HZ`. Cada fotograma lleva un overlay de cuatro líneas superpuesto **sobre una copia del fotograma**; el original enviado al VLM y guardado como `.png` de auditoría queda sin modificar para no contaminar la evidencia perceptual.

Línea 1 — nodo activo y macro-acción (texto grande, color codificado):

[KEEP_GOING] ACT: MANTENER_RUMBO WP 3/7 (42m)

- El tag entre corchetes es el **nodo del grafo de decisión** que resolvió el ciclo (§5.1): `KEEP_GOING` (crucero sin novedad), `EVASIVE` (evasión reactiva), `GIRAR_90` (bypass de bloqueo severo), `FSM` (máquina de estados, brazo FSM), `DELIBERATIVE` (consulta al VLM, §5.2) o `DEGRADED_HOVER` (modo degradado, §5.4).

- `ACT:` es la **macro-acción** traducida a comando cinemático por el nodo `motor`.
- `WP X/N (Ym)` indica el waypoint activo y la distancia horizontal restante según `WaypointTracker` (§5.3).
- **Color:** verde para `MANTENER_RUMBO` (avance sin novedad), naranja para maniobras de evasión o giro, rosa/magenta para decisiones del VLM, `PARADA` o `FRENAR`.

Línea 2 — TTC frontal y estado de vuelo (derecha del fotograma):

```
TTC: 3.2s | vuelo
```

- `TTC` es el tiempo estimado a colisión del sector frontal según el `ObstacleField` del ciclo (cap. 6); `inf` significa sector despejado.
- El campo de texto refleja el estado general de la misión (`vuelo`, `completada`, `colisión`).

Línea 3 — telemetría del ciclo:

```
t= 18.4s cy=92 | pos=(+26.1,-78.2,-10.0) | v=2.31m/s pitch=-3.2deg roll=+0.8deg
```

- `t` y `cy` permiten cruzar el fotograma con la fila exacta del CSV por número de ciclo (la sincronización por tiempo es aproximada, la de ciclo es exacta; ver `flight_video.py`).
- `pos` es la posición 3D en el marco NED del simulador.
- `v`, `pitch`, `roll` son la velocidad horizontal y la actitud del vehículo en ese instante.

Línea 4 — estado del ObstacleField por sector (cap. 6):

```
C occ=0.62 ttc=3.2s! | I occ=0.11 ttc=inf | D occ=0.08 ttc=inf
```

- Los tres sectores son `C` (centro / frente), `I` (izquierda) y `D` (derecha).
- `occ` es la fracción de ocupación estimada por flujo óptico monocular; `ttc` es el tiempo a colisión estimado del sector; `!` marca el sector como bloqueado según el umbral de `is_blocked()`.
- Estos valores son los mismos que alimentan `_build_user_prompt()` (§4.3) y el enrutador de política (§5.1): el overlay permite ver, ciclo a ciclo, qué evidencia perceptual motivó la decisión visible en la línea 1.

La combinación de estas cuatro líneas permite reconstruir, para cualquier instante del vuelo, exactamente qué percibió el sistema, por qué derivó al nodo de grafo que aparece en el tag, y qué orden cinemática emitió — sin necesidad de cruzar a mano el video con el CSV.

5. Arquitectura del lazo táctico

5.0 Escaneo espacial pre-vuelo (`spatial_scan`)

Antes de que el grafo de decisión empiece a ejecutarse, `main.py` llama de forma bloqueante a `spatial_scan()` (`src/agents/spatial_scan.py`). Esta función no es un nodo del `StateGraph`: se ejecuta una sola vez, entre la conexión con AirSim y la construcción del estado inicial del grafo.

El escaneo realiza un barrido de yaw en el lugar (sin traslación) hacia un conjunto de rumbos equiespaciados alrededor del rumbo inicial, captura un fotograma en cada rumbo tras un breve período de asentamiento y envía todas las imágenes en una única consulta al VLM. El modelo devuelve un contexto cualitativo del entorno visible desde el punto de despegue (obstáculos prominentes, corredores potenciales, densidad general de la escena). Este contexto es **advisory, no safety-critical**: el `ObstacleField` por ciclo sigue siendo la única autoridad de seguridad una vez que el dron está en movimiento.

Si el VLM no responde dentro del watchdog del escaneo, o devuelve una respuesta no parseable, la misión arranca igualmente sin contexto inicial — un VLM caído nunca impide el despegue. Al igual que el resto del sistema de percepción (cap. 6), el escaneo pre-vuelo opera exclusivamente sobre fotogramas RGB monoculares y nunca consulta el canal de profundidad del simulador.

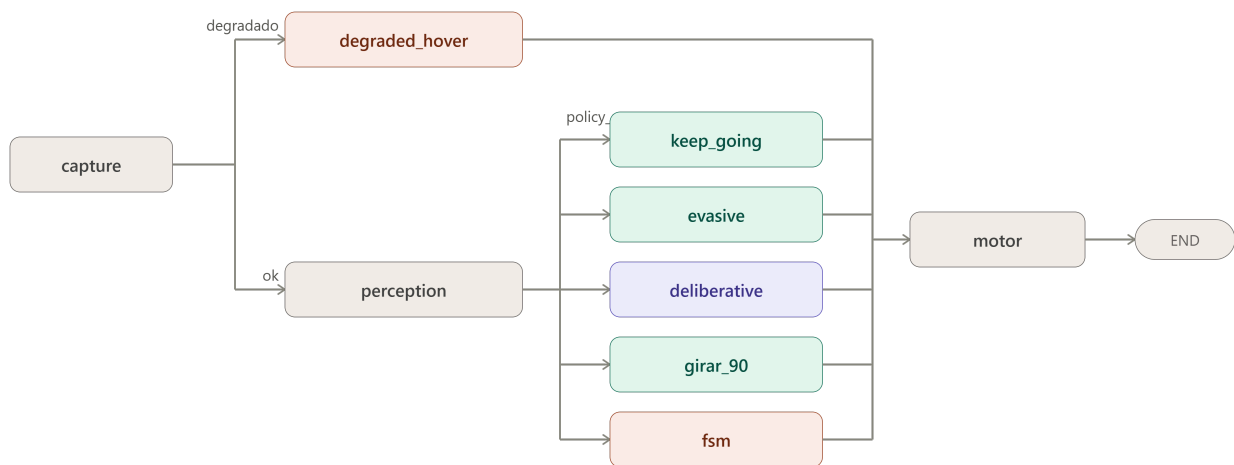
La latencia de `spatial_scan` es un costo de inicialización de única vez, no un costo por ciclo; se mide y registra separadamente del presupuesto de latencia táctica (§10.2).

5.1 Visión general del lazo

El lazo descrito en este capítulo se ejecuta sobre el entorno de simulación construido y validado en el capítulo 3 (Unreal Engine 5.5 con Cosys-AirSim), ejecutando los manifiestos compilados previamente por la estación terrena (capítulo 4). El sistema implementado es un **grafo de decisión por ciclo** (`StateGraph` de LangGraph, `src/agents/graph.py`) que se ejecuta a una frecuencia objetivo de `LOOP_HZ` (5–10 Hz según la resolución de captura elegida).

El grafo se compila **una sola vez** al inicio de la misión y el bucle externo de `main.py` lo invoca con `graph.invoke()` en cada ciclo. El grafo no tiene aristas de retorno: es acíclico por diseño. La naturaleza cíclica de la navegación la aporta el bucle externo, no el grafo. Esta distinción importa porque LangGraph construye los canales del estado a partir del `TypedDict` declarado (`DroneState`, §5.2): cualquier clave que un nodo escriba pero no esté declarada en ese schema es **descartada en silencio** al finalizar `graph.invoke()`, y por lo tanto no persiste entre ciclos. Esta es la causa raíz de al menos cuatro bugs históricos del sistema (ver §5.2).

La estructura actual del grafo (cada ciclo recorre los nodos en este orden), es la siguiente:



5.2 El estado compartido: DroneState

El estado que circula entre los nodos es un `TypedDict` llamado `DroneState` (`graph.py`). Es el único canal de comunicación entre nodos: ningún nodo importa funciones de otro ni mantiene estado propio más allá de lo que declara aquí. La tabla siguiente describe los campos clave y su rol en el sistema.

Datos sensoriales:

Campo	Tipo	Descripción
<code>rgb_image</code>	<code>ndarray</code>	Fotograma RGB del ciclo actual, producido por <code>capture_node</code>
<code>prev_image</code>	<code>ndarray</code>	Fotograma del ciclo anterior, usado por <code>FlowTTCEstimator</code>
<code>frame_history</code>	<code>List[ndarray]</code>	Ring buffer de los últimos <code>VLM_FRAME_HISTORY_SIZE</code> fotogramas (default 2: instantes <code>t</code> y <code>t-1</code>) para el VLM
<code>frame_history_ts</code>	<code>List[float]</code>	Timestamps reales de captura (reloj del simulador) de cada frame de <code>frame_history</code> , índice a índice
<code>telemetry</code>	<code>Dict</code>	Posición NED, velocidad, orientación (pitch/roll/yaw), colisión, timestamp y <code>source</code> del ciclo actual
<code>prev_telemetry</code>	<code>Dict</code>	Telemetría del ciclo anterior, usada por el estimador de flujo óptico
<code>degraded</code>	<code>bool</code>	<code>True</code> si AirSim no respondió este ciclo (<code>rgb_image is None</code> o <code>source != "airsim"</code>)

Percepción:

Campo	Tipo	Descripción
<code>obstacle_field</code>	<code>ObstacleField</code>	Contrato único de percepción: estado de los tres sectores (centro/izquierda/derecha) con ocupación, TTC y flag de bloqueo por sector (cap. 6)
<code>estimated_ttc</code>	<code>float</code>	TTC mínimo entre todos los sectores, derivado de <code>obstacle_field.min_ttc()</code> , para display y logging
<code>scene_summary</code>	<code>str</code>	Texto compacto del <code>ObstacleField</code> para el prompt del VLM

Decisión y control:

Campo	Tipo	Descripción
<code>next_action</code>	<code>str</code>	Macro-acción elegida por el nodo de política activo este ciclo
<code>velocity_command</code>	<code>Dict</code>	Comando cinemático <code>{macro_action, vx, vy, vz, yaw_rate, target_yaw, rationale}</code> listo para <code>motor_node</code>
<code>route</code>	<code>str</code>	Nodo que produjo el comando: <code>"reactive"</code> , <code>"evasive"</code> , <code>"deliberative"</code> , <code>"girar_90"</code> , <code>"fsm"</code> , <code>"degraded"</code>
<code>flight_status</code>	<code>str</code>	Estado textual de la misión para logging y overlay

Guiado a waypoint:

Campo	Tipo	Descripción
<code>waypoints</code>	<code>List[Dict]</code>	Lista de waypoints del manifiesto (x/y/z/label)
<code>current_wp_index</code>	<code>int</code>	Índice del waypoint activo en la lista
<code>target_waypoint</code>	<code>Dict</code>	Waypoint activo, copiado de la lista por <code>WaypointTracker</code>
<code>waypoint_guidance</code>	<code>Dict</code>	Salida de <code>WaypointTracker.compute_guidance()</code> : <code>vx</code> , <code>vy</code> , <code>vz</code> , <code>yaw_rate</code> , <code>distance</code> , <code>bearing_err_deg</code> , <code>target_wp</code> , <code>is_completed</code>
<code>mission_completed</code>	<code>bool</code>	<code>True</code> cuando todos los waypoints fueron alcanzados

Persistencia de maniobra (anti-flip-flop):

Campo	Tipo	Descripción
<code>active_maneuver</code>	<code>str\ None</code>	Macro-acción de la maniobra comprometida actualmente en ejecución
<code>maneuver_cycles_left</code>	<code>int</code>	Ciclos restantes de la maniobra comprometida
<code>maneuver_command</code>	<code>Dict\ None</code>	Comando cinemático fijo de la maniobra comprometida

Deliberación y VLM:

Campo	Tipo	Descripción
<code>deliberations</code>	<code>List[Dict]</code>	Registro de todas las consultas al VLM de la misión (de solo agregado: campos nunca se borran, ver §5.10)
<code>last_deliberation</code>	<code>Dict\ None</code>	Última entrada de <code>deliberations</code> , actualizada por <code>motor_node</code>
<code>slm_request_id</code>	<code>int\ None</code>	ID del pedido VLM pendiente en <code>DeliberationService</code> ; <code>None</code> si no hay pedido en vuelo
<code>_deliberation_pending</code>	<code>bool</code>	<code>True</code> mientras el lazo espera la respuesta del VLM dentro del watchdog; el llamador salta <code>record_progress()</code> para no penalizar la espera como "atasco"
<code>_pending_delib_prompt</code>	<code>str\ None</code>	Prompt enviado al VLM, guardado para adjuntarlo al registro cuando llegue la respuesta
<code>_pending_delib_frames</code>	<code>List\ None</code>	Frames RAW + timestamps enviados al VLM, guardados para auditoría
<code>_last_delib_frames</code>	<code>List\ None</code>	Canal de una sola pasada hacia <code>FlightLogger</code> : solo tiene contenido en el ciclo exacto en que una deliberación se resuelve; se consume con <code>pop()</code> y nunca se acumula en <code>deliberations</code>

Lógica de escape de deadlock:

Campo	Tipo	Descripción
<code>evasion_stuck_cycles</code>	<code>int</code>	Ciclos consecutivos sin progresar hacia el waypoint activo (acumulado por <code>WaypointTracker.record_progress()</code>)
<code>_consecutive_escapes</code>	<code>int</code>	Cantidad de escapes verticales consecutivos (GANAR_ALTURA / PERDER_ALTURA) sin progreso horizontal medido
<code>_escape_locke</code>	<code>bool</code>	<code>True</code> cuando el escape vertical se agotó (<code>_consecutive_escapes > MAX_CONSECUTIVE_ESCAPES</code>); el escape queda inhabilitado hasta que haya progreso horizontal real

Campo	Tipo	Descripción
<code>_escape_base_line_dist</code>	<code>float\ None</code>	Distancia al waypoint en el momento en que se inició el último escape; usada para evaluar si el escape resolvió el atasco
<code>_escape_reset</code>	<code>bool</code>	Señal de flanco (no de nivel): <code>True</code> en el ciclo exacto en que un escape se resuelve, para que <code>main.py</code> llame a <code>WaypointTracker.reset_progress()</code>
<code>_deadlock_cycles</code>	<code>int</code>	Ciclos dentro del estado de atasco duro (acumula mientras el escaneo profundo está activo)
<code>_deadlock_events</code>	<code>Dict\ None</code>	Métricas de resolución del atasco, consumidas por <code>FlightLogger</code>

Estado del escaneo panorámico profundo (`deep_scan`):

Campo	Tipo	Descripción
<code>_scan_phase</code>	<code>str\ None</code>	Fase actual: <code>"rotando"</code> <code>"asentando"</code> <code>"capturado"</code> <code>None</code> (no activo)
<code>_scan_heading_index</code>	<code>int</code>	Rumbo actual dentro del barrido (0 a <code>SCAN_HEADING_COUNT_DEEP-1</code>)
<code>_scan_frames</code>	<code>List</code>	Fotogramas capturados hasta ahora: <code>[(heading_deg, rgb_frame, capture_ts)]</code>
<code>_scan_start_yaw_deg</code>	<code>float\ None</code>	Yaw al inicio del barrido, referencia para calcular los rumbos objetivo
<code>_scan_settle_left</code>	<code>int</code>	Ciclos de asentamiento restantes en el rumbo actual
<code>_deep_scan_request_id</code>	<code>int\ None</code>	ID del pedido VLM panorámico pendiente

Corrección de altitud en hover:

Campo	Tipo	Descripción
<code>_hover_alt_anchor</code>	<code>float\ None</code>	Altitud anclada al primer ciclo de FRENAR; <code>motor_node</code> la usa para corregir la deriva vertical con un controlador P

Regla de diseño crítica. Todo campo que cruce la frontera entre invocaciones de `graph.invoke()` debe estar declarado en este `TypedDict`. LangGraph no lanza error si un nodo escribe una clave no declarada: la descarta en silencio. Esta propiedad causó al menos cuatro bugs documentados: `_escape_reset` nunca se propagaba (el tracker de atasco nunca se reiniciaba, produciendo un deadlock duro de 76 ciclos), `_escape_locked` se perdía (reseteando el escape agotado en un ciclo límite de período 3), `_delib_baseline` nunca acumulaba evidencia, e `inject_corner` nunca se producía.

5.3 Routers condicionales

degraded_router

Router de un solo bit, ejecutado inmediatamente después de `capture_node`:

- Si `state["degraded"]` es `True` → ruta `"degraded_hover"`
- En caso contrario → ruta `"perception"`

policy_router

Es el corazón de la arquitectura. Ejecuta la decisión táctica completa en un único paso condicional, reemplazando la cadena de tres routers que existía antes (que producía la invocación doble al VLM por ciclo). La lógica es la siguiente, en orden de prioridad:

1. Selección de brazo (AGENT_ARM). Si el brazo configurado es `"reactive"`, el router va directo a `keep_going` sin evaluar percepción. Si es `"fsm"`, va directo a `fsm`. Esto permite comparaciones factoriales limpias entre los tres brazos (§5.11).

2. Continuidad de deliberación en vuelo. Si `slm_request_id is not None` (hay un pedido al VLM en vuelo), el router **siempre** va a `"deliberative"`, independientemente del TTC actual. Sin esta regla, el pedido queda huérfano: ningún nodo vuelve a entrar a `deliberative_node` para resolverlo, `_deliberation_pending` nunca vuelve a `False`, y el detector de atasco queda desactivado para el resto de la misión.

3. Persistencia de maniobra. Si `active_maneuver` está activo, `maneuver_cycles_left > 0` y `min_ttc > TTC_EVASION_THRESHOLD`, el router va a `"evasive"` para continuar la maniobra comprometida. La condición de TTC garantiza que una emergencia real sí la interrumpa.

4. Escape de atasco. Si `evasion_stuck_cycles >= effective_stall_threshold()`: - Si además `evasion_stuck_cycles >= hard_stall_threshold()` (factor 3×, atasco duro) o `has_open_corridor(field, guidance)` es `False` → `"deliberative"` (activa el mecanismo de escape, §5.10).

5. Peligro crítico inminente. Si `center_ttc <= TTC_EVASION_THRESHOLD` (3.2 s) o el centro está bloqueado con `center_ttc <= TTC_SAFE_THRESHOLD` (4.6 s): - Si `blocked_fraction() > FOV_BLOCKED_THRESHOLD` (0.6) → `"girar_90"` (bypass determinista de bloqueo total de FOV). - En caso contrario → `"deliberative"` (consulta al VLM para decisión con contexto).

6. Advertencia de proximidad. Si el centro está bloqueado o `min_ttc <= TTC_SAFE_THRESHOLD` → `"evasive"` (corrección lateral rápida sin consultar el VLM).

7. Camino despejado. Default → `"keep_going"`.

Los tres umbrales de TTC son variables de entorno (`TTC_EVASION_THRESHOLD=3.2`, `TTC_SAFE_THRESHOLD=4.6`, `FOV_BLOCKED_THRESHOLD=0.6`) calibrados con datos de vuelo real (2026-0824).

5.4 Nodo `capture`

Entradas: cliente AirSim inyectado (singleton por proceso). **Salidas:** `rgb_image`, `telemetry`, `degraded`, `frame_history`, `frame_history_ts`, `prev_image`, `prev_telemetry`.

Copia el frame y la telemetría del ciclo anterior a `prev_image` / `prev_telemetry`, luego llama a `airsim_client.capture()` para obtener el nuevo par imagen + telemetría. Si la imagen es `None` o `telemetry["source"] != "airsim"`, marca `degraded = True` y omite la actualización del ring buffer (un ciclo degradado no sobrescribe los frames válidos anteriores).

En modo normal, agrega el nuevo frame (y su timestamp real de captura del simulador) al ring buffer `frame_history` / `frame_history_ts` y lo trunca al tamaño `VLM_FRAME_HISTORY_SIZE` (default 2). El ring buffer proporciona al VLM el contexto temporal de dos instantes consecutivos, lo que le permite inferir dirección de movimiento a partir de la secuencia de fotografías.

5.5 Nodo `degraded_hover`

Activación: `degraded_router` cuando `degraded = True`. **Salidas:** `next_action = "FRENAR"`, `route = "degraded"`, `flight_status = "degradado"`.

Emite un comando FRENAR sin consultar percepción ni deliberación. El diseño no sustituye el dato faltante de AirSim por un dato sintético plausible: la razón es la misma que motiva el capítulo 9 — un dato sintético es indistinguible de un dato real para el resto del sistema y esconde exactamente el tipo de fallo silencioso que este trabajo documenta.

5.6 Nodo `perception`

Entradas: `rgb_image`, `prev_image`, `telemetry`, `prev_telemetry`. **Salidas:** `obstacle_field`, `estimated_ttc`, `scene_summary`.

Invoca `FlowTTCEstimator.estimate()` con el par de frames consecutivos y la telemetría de ambos ciclos. El estimador produce un `ObstacleField` con tres sectores (centro, izquierda, derecha), cada uno con ocupación estimada por flujo óptico, TTC calculado y flag de bloqueo (cap. 6). El nodo escribe además `estimated_ttc = obstacle_field.min_ttc()` (el TTC mínimo entre sectores, usado directamente por `policy_router`) y `scene_summary = obstacle_field.summary_text()` (texto compacto para el prompt del VLM).

El `ObstacleField` es el único objeto que consumen el router de política, el nodo de evasión, el nodo deliberativo y la FSM. Ningún consumidor accede a campos crudos de flujo óptico.

5.7 Nodo `keep_going`

Función: `reactive_node` en `src/agents/reactive.py`. **Activación:** camino despejado (TTC alto, sin atasco) o brazo `AGENT_ARM=reactive`. **Salidas:** `next_action`, `velocity_command`, `route = "reactive"`.

Es el nodo más barato del grafo (costo de cómputo nulo: solo lee `waypoint_guidance`). Tres casos:

1. **Misión completada** (`mission_completed` o `guidance["is_completed"]`): emite `FRENAR` y `flight_status = "mision_completada"`.
2. **Waypoint activo disponible**: toma directamente los valores de `waypoint_guidance` producidos por `WaypointTracker.compute_guidance()` (§5.14) — `vx`, `vy`, `vz`, `yaw_rate` ya están calculados para seguir la trayectoria hacia el waypoint — y emite `MANTENER_RUMBO`.
3. **Sin waypoints (fallback)**: avanza a `REACTIVE_FORWARD_SPEED` (default 5 m/s) con corrección vertical `-0.1 * vz_actual` para amortiguar deriva vertical.

5.8 Nodo `evasive`

Función: `evasive_node` en `src/agents/evasive.py`. **Activación:** TTC en ventana de advertencia (`TTC_SAFE_THRESHOLD`) o continuación de maniobra comprometida. **Salidas:** `next_action`, `velocity_command`, `route = "evasive"`, actualización de `active_maneuver` / `maneuver_cycles_left`.

Dos modos de operación:

Modo persistencia (anti-flip-flop): si `active_maneuver` está activo y `maneuver_cycles_left > 0`, continúa la maniobra comprometida ciclo a ciclo. En cada ciclo recalcula la diferencia de yaw respecto al `target_yaw` de la maniobra: si `|yaw_diff| > 3°` aplica un controlador P de yaw (`yaw_rate = clamp(0.6 * yaw_diff, ±15°/s)`); si el error convergió, pone `yaw_rate = 0` y avanza con `vx = 0.8 m/s`. Decrementa `maneuver_cycles_left`; cuando llega a cero, limpia `active_maneuver`.

Modo nueva evasión: compara `sector_occupancy("izquierda")` con `sector_occupancy("derecha")`. En caso de empate, desempata por TTC (sector con mayor TTC). Elige `EVADIR_IZQUIERDA` o `EVADIR_DERECHA` y llama a `action_to_command()` con `aggressive=True` (velocidad de evasión `vx = 1.2 m/s`).

`action_to_command()` para evasión lateral calcula `target_yaw` redondeado al múltiplo de 90° más cercano (`_manhattan_snap_yaw`), apropiado para la cuadrícula urbana de los escenarios de prueba. El comando resultante tiene `yaw_rate = ±EVASION_LATERAL_YAW_RATE` (15°/s). La maniobra se compromete durante `EVASIVE_MANEUVER_DURATION_S` × `LOOP_HZ` ciclos, guardada en `active_maneuver` / `maneuver_command` para que el router de política la respete.

5.9 Nodo `girar_90`

Función: `girar_90_node` en `graph.py`. **Activación:** `policy_router` cuando `blocked_fraction() > FOV_BLOCKED_THRESHOLD` (0.6) — el FOV está tan obstruido que cualquier corrección lateral es inútil. **Salidas:** `next_action = "GIRAR_90"`, `route = "girar_90"`, `flight_status = "exploracion_yaw"`, maniobra comprometida.

Genera un giro de 90° sin traslación. El **lado del giro** lo determina el error de rumbo al waypoint (`bearing_err_deg`): si el waypoint está a la izquierda, el giro va a la izquierda, y viceversa. Antes del fix de 2026-0824, el giro era siempre a la derecha, lo que en un caso documentado mandó al dron 90° en dirección contraria al waypoint mientras la percepción reportaba el corredor despejado al otro lado.

El comando usa `yaw_rate = ±20°/s` con `target_yaw` redondeado a la cuadrícula de 90°. La duración es `GIRAR90_DURATION_S` (default 1.0 s) × `LOOP_HZ` ciclos. La maniobra se compromete en `active_maneuver` / `maneuver_cycles_left` para que `policy_router` la complete antes de permitir cualquier otra decisión táctica.

5.10 Nodo `deliberative`

Función: `make_deliberative_node(service)` en `src/agents/deliberative.py`. Es el nodo más complejo del grafo. **Activación:** TTC crítico, FOV no totalmente bloqueado, o atasco (`evasion_stuck_cycles >= effective_stall_threshold()`).

El nodo ejecuta una de tres rutas en cada ciclo:

Ruta 1 — Escape de deadlock (sincrónico, sin consultar el VLM)

Condición de activación: `evasion_stuck_cycles >= effective_stall_threshold()` y `_escape_locked = False` y sin corredor transitable (`not has_open_corridor(field, guidance)`).

Antes de ejecutar el escape, el nodo evalúa si el escape previo resolvió el atasco comparando `dist_xy` actual con `_escape_baseline_dist`. Si hubo progreso real (`dist_xy < baseline - WAYPOINT_PROGRESS_EPS_M`), resetea `_consecutive_escapes`, `_escape_locked`, `_escape_baseline_dist` y `_deadlock_cycles`.

Si el atasco persiste y `DEADLOCK_STRATEGY = "deep_vlm"` (default), delega al módulo `deep_scan` antes de forzar el escape ciego (§5.12). Si `deep_scan` resuelve el ciclo, retorna; si falla (timeout o respuesta no válida), continúa hacia el escape sincrónico.

El escape sincrónico **alterna** entre `GANAR_ALTURA` (intentos impares) y `PERDER_ALTURA` (intentos pares) para no insistir hacia arriba si el obstáculo bloquea también por encima. Guarda `_escape_baseline_dist = dist_xy` actual y la acción como maniobra comprometida (`ESCAPE_MANEUVER_DURATION_S` × `LOOP_HZ` ciclos).

Agotamiento del escape (`_consecutive_escapes > MAX_CONSECUTIVE_ESCAPES = 3` o `altitud > MAX_ESCAPE_ALT_M = 20 m`): en vez de frenar indefinidamente (el comportamiento anterior, que producía un ciclo límite documentado), el nodo **enclava** el escape (`_escape_locked = True`) y cambia de estrategia: emite un `GIRAR_90` hacia el lado del waypoint e inyecta un waypoint de desvío temporal (`inject_corner`) a `CORNER_OFFSET_M` metros en esa dirección, para que `WaypointTracker` apunte al corredor lateral en vez de insistir hacia la línea bloqueada.

Ruta 2 — Poll de pedido VLM pendiente

Condición: `slm_request_id is not None` (hay un pedido en vuelo en `DeliberationService`).

Llama a `service.poll()` para obtener el resultado más reciente y la edad del pedido pendiente:

- **Resultado disponible con ID coincidente** → `_finalize()` (descripción abajo).
- **Watchdog expirado** (`age_ms > SLM_WATCHDOG_MS = 1500 ms`) → `_fallback_decision()` → `_finalize()` marcando `timeout = True`.
- **Pendiente dentro del watchdog** → emite `_wait_command()` y marca `_deliberation_pending = True`.

`_wait_command()` resuelve el dilema de movilidad durante la espera: si hay un bloqueo central confirmado con TTC bajo (`close_structural = True`), emite `FRENAR` por seguridad; si no, avanza lentamente a `DELIB_WAIT_CREEP_SPEED_MPS = 0.5 m/s`. La segunda opción preserva la traslación necesaria para que `FlowTTCEstimator` genere flujo óptico y mantenga la confianza de percepción. El bucle de retroalimentación que se rompía antes (frenar → sin flujo → sin confianza → volver a deliberar → frenar) se documenta en el análisis de TOWNSIM_INI (2026-0903).

Ruta 3 — Nuevo pedido al VLM

Condición: no hay pedido pendiente (`slm_request_id is None`).

Construye el user prompt con `_build_user_prompt()` (cinco componentes: resumen del `ObstacleField`, objetivo/altitud, estado cinemático, motivo de consulta y historial reciente — descripción completa en §4.3). Si `VLM_VISION_ENABLED = True`, codifica `frame_history` como JPEG base64 (redimensionado a `VLM_IMAGE_MAX_SIZE = 384 px`). Encola el pedido en `DeliberationService.request()` (cola de tamaño 1 — un pedido nuevo descarta cualquier pedido pendiente no procesado). Guarda `_pending_delib_prompt` y `_pending_delib_frames` para la auditoría. Emite `_wait_command()`.

`_finalize()` — Resolución de una deliberación

Aplica un override de seguridad: si el VLM eligió `MANTENER_RUMBO` pero `close_structural = True`, fuerza `_fallback_decision()` y marca `is_fallback = True`. Traduce la macro-acción a comando cinemático con `action_to_command()`. Agrega una entrada a `deliberations[]` con `id`, `timestamp`, `arm`, `model`, `vision_enabled`, `system_prompt`, `prompt`, `raw_response`, `macro_action`, `rationale`, `is_fallback`, `timeout`, `adherent`, `used_json_schema`, `latency_ms`. Si la macro-acción es evasiva (`EVADIR_*`, `GANAR_ALTURA`), la compromete como maniobra activa.

`_fallback_decision()` — Heurística determinista

Árbol de decisión sobre el `ObstacleField` cuando el VLM no responde o su respuesta no es parseable:

1. Sin evidencia de percepción → `FRENAR`.
2. Centro + izquierda + derecha bloqueados → `GANAR_ALTURA`.

3. Centro bloqueado + ambos laterales libres → evadir hacia el waypoint por `bearing_err_deg`.
4. Centro bloqueado + un lateral libre → evadir hacia el lado libre.
5. Centro libre → `MANTENER_RUMBO`.

`_parse_decision()` — Parser tolerante

Red de seguridad para respuestas que no siguen el formato JSON exacto, incluso cuando se usó decodificación restringida (`response_format=json_schema`). Tres estrategias en cascada: (1) limpia markdown (fences ````json`), extrae el primer bloque JSON con regex, parsea con `json.loads()`; (2) reintenta reemplazando comillas simples por dobles; (3) busca `macro_action` con regex de campo nombrado; (4) escanea el texto completo buscando cualquiera de las acciones válidas como substring. Si ninguna estrategia produce una acción válida, retorna `None` → activa `_fallback_decision()`.

`DeliberationService` — Hilo worker asíncrono

`DeliberationService` (`src/agents/deliberation_service.py`) mantiene un hilo daemon (`threading.Thread`) con una cola de entrada de tamaño 1 (`Queue(maxsize=1)`). `request(payload)` vacía la cola antes de poner el nuevo pedido (garantiza que el worker procese siempre el pedido más reciente, nunca uno stale). `poll()` retorna `(resultado_más_reciente, edad_ms_pedido_pendiente, hay_pedido_pendiente)` con protección de lock. El servicio es compartido entre el brazo SLM (`deliberative_node`) y el módulo `deep_scan` — un único hilo worker para toda la misión.

La consulta HTTP al servidor LLM (`_query_slm_impl`) intenta primero con `response_format=json_schema` (decodificación restringida, temperatura 0.2, max_tokens 200, timeout 8 s); si el servidor no soporta el modo o devuelve vacío, reintenta en modo libre y deja el parser tolerante como red de seguridad. El resultado incluye `used_json_schema: bool` para auditoría.

5.11 Nodo `fsm`

Función: `fsm_node(state, service)` en `src/agents/fsm.py`. **Activación:** `AGENT_ARM = "fsm"`, para comparación experimental con el brazo SLM.

La FSM usa exactamente el mismo `ObstacleField`, el mismo vocabulario de macro-acciones y el mismo `action_to_command()` que el brazo SLM. Lo único que cambia es **quién elige la etiqueta de acción**: la FSM lo hace por umbrales deterministas, sin consultar ningún modelo.

Estados internos y transiciones (`_decide_state()`):

Estado FSM	Macro-acción	Condición de transición
<code>CRUISE</code>	<code>MANTENER_RUMBO</code>	Centro despejado (default)
<code>AVOID_LEFT</code>	<code>EVADIR_IZQUIERDA</code>	Centro bloqueado/TTC≤3.5s, izquierda menos ocupada

Estado FSM	Macro-acción	Condición de transición
AVOID_RIGHT	EVADIR_DERECHA	Centro bloqueado/ $TTC \leq 3.5s$, derecha menos ocupada
CLIMB	GANAR_ALTURA	Todos los sectores bloqueados (intento par de escape)
DESCEND	PERDER_ALTURA	Todos los sectores bloqueados (intento impar de escape)
BRAKE	FRENAR	$center_ttc \leq FSM_TTC_BRAKE_S$ (1.5 s), un lateral libre

La FSM implementa las mismas salvaguardas que el brazo SLM: persistencia de maniobra, alternancia CLIMB/DESCEND en el escape de deadlock, enclavamiento del escape agotado con GIRAR_90 + inject_corner, y (si DEADLOCK_STRATEGY = "deep_vlm") delegación al módulo deep_scan antes de forzar el escape ciego. Esta simetría de salvaguardas es intencional: la comparación SLM vs FSM debe medir **quién elige mejor la acción**, no quién tiene mejor lógica de seguridad.

La diferencia de umbral clave: la FSM usa $FSM_TTC_BRAKE_S = 1.5\text{ s}$ y $FSM_TTC_AVOID_S = 3.5\text{ s}$ frente a los umbrales del router de política ($TTC_EVASION_THRESHOLD = 3.2\text{ s}$, $TTC_SAFE_THRESHOLD = 4.6\text{ s}$) que alimentan el brazo SLM. Esta diferencia existe porque la FSM decide sin información visual ni de dirección: sus umbrales son más conservadores para compensar la falta de contexto semántico.

5.12 Módulo de escaneo panorámico en atasco duro (deep_scan)

Archivo: src/agents/deep_scan.py. Compartido entre los brazos SLM y FSM. **Activación:** DEADLOCK_STRATEGY = "deep_vlm" (default) cuando se detecta atasco duro sin corredor transitable.

El módulo implementa una máquina de estados de cuatro fases, sostenida a través de ciclos consecutivos vía los campos _scan_* del DroneState:

Fase None (inicialización): inicializa _scan_phase = "rotando", _scan_heading_index = 0, _scan_frames = [], _scan_start_yaw_deg = yaw_actual. Cancela cualquier maniobra activa.

Fase "rotando": comanda $target_yaw = start_yaw + index \times (360^\circ / SCAN_HEADING_COUNT_DEEP)$ (con $SCAN_HEADING_COUNT_DEEP = 4$, los cuatro rumbos son +0°, +90°, +180°, +270° del rumbo de inicio). La traslación es cero ($v_x = v_y = v_z = 0$). Cuando el yaw actual está dentro de $SCAN_YAW_TOLERANCE_DEG = 5^\circ$ del objetivo → transición a "asentando".

Fase "asentando": hover en el rumbo objetivo durante $SCAN_SETTLE_CYCLES_DEEP = 2$ ciclos para que la física del simulador estabilice la actitud antes de capturar. Al terminar el asentamiento, captura el frame del DroneState (el que capture_node ya produjo en este ciclo, **nunca** una nueva llamada a AirSim), guarda la tupla (heading_deg, rgb_frame, capture_ts) en _scan_frames, avanza _scan_heading_index y vuelve a "rotando". Al completar los cuatro rumbos → transición a "capturado".

Fase "capturado": construye el prompt del escaneo profundo (`_build_deep_scan_prompt()`) e invoca `service.request()` con los frames codificados como JPEG base64 y etiquetas por rumbo (`"[Rumbo 90.0°] (rumbo actual, el que viene fallando)"`). El modo del pedido es `"deep_scan"`, para que `_query_slm_impl()` use `SYSTEM_PROMPT_DEEP_SCAN` en vez del prompt táctico normal. En ciclos siguientes hace poll hasta que el resultado llegue o el watchdog `SLM_DEEP_WATCHDOG_MS = 12 000 ms` expire.

Si el resultado es una acción válida → `_apply_scan_resolution()`: emite la macro-acción, registra en `deliberations[]` con `arm = "{arm}_deep_scan"`, limpia el estado del escaneo, escribe `_deadlock_event` y pide `_escape_reset`. Retorna `True` al llamador (el escape sincrónico queda descartado este ciclo).

Si el resultado llega con acción no válida, o el watchdog expira → limpia el estado, escribe `_deadlock_event` con `fell_back_to_blind = True` y retorna `False`. El llamador continúa inmediatamente hacia el escape sincrónico (GANAR_ALTURA / PERDER_ALTURA) como red de seguridad final, **en el mismo ciclo**.

Durante todo el barrido, `_deliberation_pending = True` congela el contador `evasion_stuck_cycles` para que `main.py` no lo resetee por accidente mientras el dron gira en el lugar.

5.13 Nodo `motor`

Función: `motor_node` en `graph.py`. Único nodo que interactúa con el actuador. **Entradas:** `velocity_command`, `telemetry`. **Salidas:** emisión de comando a AirSim, actualización de `last_deliberation`, corrección de `_hover_alt_anchor`.

Lee `velocity_command` del estado (producido por cualquiera de los nodos de política). Manejo especial para `FRENAR`:

- En el **primer ciclo** de FRENAR, ancla `_hover_alt_anchor = current_z` (coordenada Z de la posición NED actual).
- En **ciclos siguientes** de FRENAR, computa `dz = anchor - current_z` y, si `|dz| >` `HOVER_ALT_DEADZONE_M = 0.3 m`, inyecta `vz = clamp(HOVER_ALT_KP × dz, ±HOVER_ALT_MAX_VZ)` con `HOVER_ALT_KP = 0.35` y `HOVER_ALT_MAX_VZ = 0.8 m/s`. Este controlador P corrige la deriva vertical que ocurre cuando `moveByVelocityBodyFrameAsync(vz=0)` se reemite repetidamente (medido: hasta 9 m de deriva en 120 s de FRENAR sostenido sin corrección).
- En cualquier otro comando, limpia `_hover_alt_anchor = None`.

Finalmente llama a `airsim_client.execute_velocity(vx, vy, vz, yaw_rate, target_yaw)`. Cuando `target_yaw` no es `None`, `AirSimClient` usa `YawMode(is_rate=False, yaw_or_rate=target_yaw)` (orientación absoluta); cuando es `None`, usa `YawMode(is_rate=True, yaw_or_rate=yaw_rate)` (velocidad angular).

5.14 Mapa de macro-acciones (`action_to_command`)

Todos los nodos de política comparten `action_to_command()` (`src/agents/action_map.py`) como fuente única de verdad para la cinemática de cada macro-acción. Esto garantiza que `GANAR_ALTURA` tenga la misma velocidad vertical ya sea que lo elija el VLM, la FSM o el escape sincrónico.

Macro-acción	<code>vx</code> (m/s)	<code>vy</code>	<code>vz</code> (m/s)	<code>yaw_rate</code> (°/s)	<code>target_yaw</code>
<code>MANTENER_RUMBO</code>	guidance	0	guidance	guidance	None
<code>EVADIR_DERECHA</code>	1.2 (agg.) / 0.3–0.8	0	guidance	+15	snap 90° dcha.
<code>EVADIR_IZQUIERDA</code>	1.2 (agg.) / 0.3–0.8	0	guidance	–15	snap 90° izq.
<code>GANAR_ALTURA</code>	0	0	– <code>EVASION_UP_SPEED</code> (1.5)	guidance	None
<code>PERDER_ALTURA</code>	1.0	0	+ <code>EVASION_DOWN_SPEED</code> (0.8)	0	None
<code>GIRAR_90</code>	0	0	0	±20	snap 90° hacia WP
<code>FRENAR</code>	0	0	0	0	None

`GANAR_ALTURA` usa `vx = 0` (asciende en el lugar) desde el fix de 2026-0824: la versión anterior tenía `vy = 0.5 m/s` de deriva lateral constante, que alejaba el waypoint en el plano XY, la misma métrica que mide si el atasco se resolvió, produciendo un escape que se retroalimentaba a sí mismo.

5.15 Guiado a waypoint: `WaypointTracker`

`WaypointTracker` (`src/navigation/waypoint_tracker.py`) corre en `main.py` **fuera del grafo**, una vez por ciclo, antes de invocar `graph.invoke()`. Produce `waypoint_guidance` que los nodos consumen para orientarse.

`compute_guidance()` calcula `vx`, `vy`, `vz`, `yaw_rate` en Body Frame hacia el waypoint activo, con: - Corrección de rumbo por cross-track error y error de yaw. - Zona muerta angular: desvíos menores a un umbral no generan corrección de yaw para evitar oscilaciones. - Saturación de tasa de yaw diferenciada: tasa de giro suave para desvíos pequeños, tasa brusca para desvíos grandes. - Suavizado EMA sobre los comandos de guiado para reducir cabeceos. - Comportamiento `near_vertical`: si `dist_xy < 1 m` y `|dz| > 0.3 m`, fuerza `vx = 0` para ascenso/descenso puramente vertical. Imprescindible para el patrón *climb-first* de Tier 2 (§3.2).

`record_progress(dist_xy, bearing_err_deg)` actualiza `evasion_stuck_cycles`: - Si `dist_xy` disminuyó en más de `WAYPOINT_PROGRESS_EPS_M` respecto al mínimo visto → resetea el contador y actualiza el mínimo. - Si `|bearing_err_deg| > PROGRESS_STALL_BEARING_EXEMPT_DEG` (30°) → el ciclo se exime (el dron está girando activamente hacia el waypoint, no atascado), pero solo hasta `PROGRESS_STALL_BEARING_EXEMPT_MAX_CYCLES = 15` ciclos consecutivos (un giro que no converge no puede eximirse indefinidamente). - En caso contrario → incrementa `evasion_stuck_cycles`.

`effective_stall_threshold()` calcula el umbral mínimo de ciclos sin progreso que es **físicamente demostrable**: $\text{eps_m} / (\text{min_speed} \times 1/\text{loop_hz})$. Con los defaults (`eps_m = 0.5 m`, `min_speed = 0.25 m/s`, `loop_hz = 5`), el umbral coherente es 10 ciclos. `hard_stall_threshold()` es 3× ese valor (30 ciclos), a partir del cual el escape se fuerza aunque la percepción crea ver un corredor libre.

5.16 Salvaguardas y contratos de diseño

Principio de deliberación por excepción. La mayoría de los ciclos deben resolverse por `keep_going` o `evasive` (rutas deterministas de costo casi nulo). El VLM solo se consulta cuando hay evidencia clara de riesgo o atasco. En las corridas de validación sobre CITYSIM_CLEAR (brazo SLM), la tasa de ciclos deliberativos fue menor al 20% del total.

Exclusión total de profundidad. Ningún nodo del grafo pide el canal de profundidad de AirSim. El esquema `DroneState` actúa como segunda red de esta guardia: cualquier clave que intente transportar datos de profundidad entre invocaciones es descartada en silencio por LangGraph si no está declarada. El test estático `tests/test_no_depth_in_flight_path.py` verifica esta propiedad sin necesitar AirSim.

Contrato de `deliberations[]`. Es de solo agregado: se le suman entradas pero nunca se borran campos. Los análisis del capítulo 10 y el capítulo 9 se derivan de este registro.

Cliente AirSim como singleton. `compile_workflow(airsim_client)` recibe un cliente ya conectado (inyección de dependencia). Hay un único cliente por proceso; el patrón histórico de `get_airsim_client()` (deprecado) creaba un segundo cliente con su propio `takeoffAsync`, desconectado del grafo real.

5.17 Modo degradado

Cuando `degraded = True` (AirSim no disponible), el sistema no sustituye el dato faltante por uno sintético plausible: el ciclo va directo a `degraded_hover`, se comanda hover explícito y se omiten percepción y deliberación por completo. La razón de este diseño es la misma que motiva el capítulo 9: un dato sintético "razonable" en lugar de un dato ausente es indistinguible, para el resto del sistema, de un dato real, y esconde exactamente el tipo de fallo silencioso que este trabajo documenta.

5.18 Registro de vuelo y auditoría post-vuelo

Cada ejecución del lazo táctico genera una carpeta autocontenida en `airsim-runs/` (descripción completa en §4.4), con traza JSONL/CSV, resumen por waypoint, fotogramas de auditoría VLM y video anotado. La traza CSV/JSONL es el contrato de evidencia primaria del que se derivan las métricas del capítulo 10 y el análisis de modos de falla del capítulo 9. Para la inspección cualitativa, el visor HTML sincroniza en ambos sentidos el video con la traza (descripción de las anotaciones del video en §4.4).

6. Percepción monocular: flujo óptico, TTC y campo de obstáculos

6.1 Fundamentos de diseño: percepción geométrica sin redes neuronales

La arquitectura de percepción a bordo implementada en este trabajo prescinde deliberadamente de redes neuronales profundas de detección (detectores de cajas delimitadoras tipo YOLO — Redmon et al., 2015 — o segmentadores semánticos densos) y fundamenta la estimación de obstáculos en visión por computadora clásica: **flujo óptico denso derotado y divergencia del campo traslacional**. Esta decisión no es de conveniencia sino de principio, y se sustenta en tres argumentos de ingeniería robótica:

1. Determinismo y presupuesto de cómputo compartido. En una plataforma donde los recursos de CPU deben compartirse con la inferencia de un modelo de lenguaje local (SLM/VLM), un estimador de flujo clásico (DIS en OpenCV) garantiza una latencia determinista y acotada por ciclo (5–10 Hz), sin los picos de inferencia asociados a redes convolucionales o transformadores visuales densos. Shi et al. (2024) y Goel et al. (2021) documentan este problema de presupuesto en sistemas embebidos de visión con requisitos de tiempo real; en este sistema el presupuesto disponible para percepción es del orden de 20–50 ms por ciclo.

2. Generalización universal y robustez fuera de distribución. Los detectores supervisados están limitados a las clases presentes en su conjunto de entrenamiento (automóviles, personas, señales). En un entorno urbano tridimensional, los obstáculos potenciales incluyen cables, salientes arquitectónicas, andamios o geometrías irregulares sin etiqueta semántica previa. El flujo óptico modela directamente el fenómeno **físico** de aproximación espacial mediante la expansión de textura en el plano focal: reacciona ante cualquier objeto que genere paralaje, sin importar su clase ni su apariencia (Badrloo & Varshosaz, 2017; Vera-Yanez et al., 2024a). Esta propiedad es especialmente valiosa en entornos de simulación que aún no reproducen fielmente la distribución fotométrica del mundo real (cap. 9).

3. Modelado físico del Tiempo-a-Colisión (TTC). Una caja delimitadora 2D no provee información cinemática de cierre ni distancia métrica directa. La divergencia del campo de flujo traslacional permite derivar matemáticamente el TTC ($TTC \approx d / v_{\text{cierre}}$) de forma continua a lo largo del plano de la imagen, entregando una señal temporal interpretable y calibrable para las decisiones de maniobra reactiva. Esta derivación es una consecuencia directa de la ecuación del flujo óptico bajo traslación pura (Al-Kaff et al., 2017; Vera-Yanez et al., 2024b).

Limitación estructural y rol del VLM. La elección de percepción clásica tiene un costo explícito: el flujo óptico requiere **traslación entre frames** para generar evidencia válida. En hover puro, giro puro o crucero muy lento, la señal traslacional colapsa a cero y el campo de obstáculos pierde toda confianza. Este punto ciego estructural es exactamente el que el VLM deliberativo (cap. 5) está diseñado para cubrir: cuando el `ObstacleField` reporta "sin evidencia", el modelo de lenguaje toma decisiones con contexto visual e histórico que el estimador de flujo no puede proveer. La arquitectura de dos capas — percepción geométrica rápida + deliberación semántica lenta — es el mecanismo central con el que este trabajo aborda esa limitación.

6.2 Pipeline de percepción: cinco etapas

`FlowTTCEstimator.estimate()` (`src/perception/flow_ttc.py`) ejecuta cada ciclo cinco etapas en secuencia, produciendo un `ObstacleField` (`src/perception/obstacle_field.py`) listo para el router de política:

```
Frame(t-1) + Frame(t) + Telemetría(t-1, t)
↓
[1] Escala y conversión a escala de grises
↓
[2] Flujo óptico denso (DIS / Farneback)
↓
[3] Derotación por telemetría de actitud (IMU)
↓
[4] Estimación del FOE + RANSAC-lite de outliers
↓
[5] TTC por píxel + divergencia → agregación por celdas → ObstacleField
```

Si en cualquier etapa no hay evidencia suficiente (primer ciclo, modo degradado, rotación grande entre frames, flujo bajo el piso de ruido), la función retorna un `empty_field()` con `source="degraded"` o `"flow"` y `foe_confidence=0.0`, marcando explícitamente la ausencia de evidencia sin ningún clamp cosmético que la enmascare.

6.3 Etapa 1: escala y parámetros intrínsecos

Los fotogramas BGR de AirSim se convierten a escala de grises y se escalan a un ancho normalizado `FLOW_DOWNSCALE_WIDTH` (default 320 px). Esta reducción sirve dos propósitos: reducir el costo de cómputo del estimador de flujo y suavizar el ruido de alta frecuencia que genera falsos vectores de flujo. La distancia focal se escala proporcionalmente:

$$f_x' = f_x \cdot \frac{W'}{W}, \quad f_y' = f_y \cdot \frac{W'}{W}$$

con `CAMERA_FX = CAMERA_FY = 554.0` px (cámara simétrica a la resolución de captura de AirSim) y el centro óptico $(c_x, c_y) = (W'/2, H'/2)$.

Guard de rotación. Antes de computar el flujo, el estimador verifica que la rotación máxima entre frames (pitch, yaw, roll) no exceda `FLOW_MAX_ROTATION_DEG` (default 2°). Durante maniobras activas (GIRAR_90, EVADIR) el yaw cambia 4–9°/ciclo a `LOOP_HZ=5` Hz. En esas condiciones, el modelo de derotación lineal de primer orden que se describe en §6.4 introduce errores de linealización que el estimador no puede compensar, y regiones de baja textura (cielo, fachadas uniformes) generan flujo esencialmente aleatorio. El resultado documentado antes de implementar este guard fueron "nubes" de falsos obstáculos durante cada giro (CHANGELOG.md 2026-0826). La solución conservadora es descartar el ciclo y retornar `empty_field(source="degraded")`: es mejor declarar incertidumbre que producir un falso positivo que cancele una maniobra en ejecución.

6.4 Etapa 2: flujo óptico denso (DIS)

El backend de flujo óptico es **DIS** (*Dense Inverse Search*, `cv2.DISOpticalFlow_create(PRESET_MEDIUM)`), con fallback a Farneback si DIS no está disponible. DIS produce un campo vectorial $\mathbf{v}(x,y) = (u, v)$ de desplazamientos en píxeles para cada posición del frame actual respecto al anterior.

Por qué DIS y no Farneback. DIS (Kroeger et al., 2016, referenciado en la implementación de OpenCV) ofrece una relación velocidad/calidad superior para flujo denso en imágenes de tamaño pequeño (≤ 320 px): su esquema de búsqueda inversa por parches es entre 5 y 10 veces más rápido que Farneback para resoluciones comparables, manteniendo la suavidad necesaria para la estimación del FOE. Vera-Yanez et al. (2024b) y Molineros et al. (2012) usan variantes de flujo óptico denso con esquemas de pirámide similares; la elección de DIS está motivada por el mismo compromiso latencia/densidad que documentan esos trabajos.

El campo resultante $\mathbf{v}(x,y)$ mezcla el componente traslacional (paralaje de objetos en escena) con el componente rotacional inducido por los cambios de actitud del vehículo. Separar ambos componentes es el objetivo de la etapa siguiente.

6.5 Etapa 3: derotación analítica por telemetría IMU

La rotación propia del vehículo entre frames consecutivos genera un flujo óptico angular parásito $\mathbf{v}_{\text{rot}}$ que enmascara la expansión traslacional real. Para un modelo de cámara *pinhole* con rotación pequeña entre frames, la contribución rotacional al flujo óptico en el plano normalizado es:

$$u_{\text{rot}}(x,y) = \frac{\theta_x \cdot x \cdot y}{f_x} - \theta_y \left(f_x + \frac{x^2}{f_x} \right) + \theta_z \cdot y$$

$$v_{\text{rot}}(x,y) = \theta_x \left(f_y + \frac{y^2}{f_y} \right) - \frac{\theta_y \cdot x \cdot y}{f_y} - \theta_z \cdot x$$

donde (x, y) son coordenadas en el plano de la imagen centradas en (c_x, c_y) , y $(\theta_x, \theta_y, \theta_z)$ son los incrementos de actitud (pitch, yaw, roll) entre frames, extraídos de la telemetría del IMU del simulador. El mapeo cámara-cuerpo asume alineación frontal: $\theta_x \approx \Delta \text{pitch}$, $\theta_y \approx \Delta \text{yaw}$, $\theta_z \approx \Delta \text{roll}$.

El campo traslacional derotado es:

$$\mathbf{v}_{\text{trans}} = \mathbf{v} - \mathbf{v}_{\text{rot}}$$

La implementación (`_derotate()`) vectoriza esta operación sobre el array completo usando `np.mgrid` para construir los mapas de coordenadas (x, y) en una sola operación, sin bucles por píxel. El salto de wrap-around del yaw ($\pm\pi$) se corrige con el módulo estándar antes de usar Δyaw como θ_y .

Esta corrección es crítica para el sistema: sin ella, cada corrección de guiado (giro de unos pocos grados hacia el waypoint) genera un flujo rotacional en el sector central que se interpreta como un obstáculo frontal, disparando deliberaciones espurias en cada ciclo de cruce. El mismo principio de derotación por IMU se usa en sistemas de visión activa para vehículos terrestres (Dickmanns, 2024) y SLAM monocular (Chen et al., 2022).

6.6 Etapa 4: estimación del Foco de Expansión (FOE) con RANSAC-lite

Bajo traslación pura (sin rotación residual), todos los vectores del campo traslacional apuntan radialmente desde un único punto llamado **Foco de Expansión** (FOE, también llamado punto de fuga traslacional). La posición del FOE en el plano de la imagen codifica la dirección de traslación del vehículo, y la magnitud del flujo en cada píxel es inversamente proporcional a la distancia al obstáculo que generó ese paralaje.

Descarte de ruido de baja magnitud. Los píxeles con $|\mathbf{v}_{\text{trans}}| \leq \text{FLOW_NOISE_FLOOR_PX}$ (default 0.35 px) se descartan antes de cualquier ajuste. Por debajo de este umbral, los vectores de flujo son artefactos de cuantización del estimador, no señal real. Si la fracción de píxeles válidos resultante es menor que `MIN_VALID_FRACTION_FOR_FOE` (default 1%), el campo entero se declara sin evidencia.

Ajuste por mínimos cuadrados ponderados (primera pasada). Para los píxeles válidos, la condición geométrica de consistencia con el FOE es que cada vector de flujo $\mathbf{v}(p)$ esté **alineado** con la línea que une p al FOE. La componente normal al vector $\mathbf{v}$ debe ser cero en el FOE: para el píxel (p_x, p_y) con vector (v_x, v_y) , la normal normalizada es $\mathbf{n} = (-v_y, v_x) / |\mathbf{v}|$, y la condición es $\mathbf{n}^T (\text{FOE} - p) = 0$. El sistema lineal resultante es:

$$\left(\sum_i w_i \mathbf{n}_i \mathbf{n}_i^T \right) \text{FOE} = \sum_i w_i \mathbf{n}_i (p_i)$$

donde $w_i = \|\mathbf{v}_i\|$ pondera cada ecuación por la magnitud del flujo (vectores más largos son más confiables). Este sistema 2×2 se resuelve con `np.linalg.solve()`.

Recorte de outliers tipo RANSAC-lite (segunda pasada). El FOE de primera pasada puede estar contaminado por píxeles de fondo, regiones de baja textura, o residuos de la derotación. El refinamiento evalúa, para cada píxel, el ángulo entre su vector de flujo y la línea p `to \text{FOE}`: si el ángulo supera `FOE_OUTLIER_ANGLE_RAD` (default 0.35 rad $\approx 20^\circ$), el píxel se descarta como outlier. Con los inliers restantes se repite el ajuste por mínimos cuadrados. Este esquema es una versión simplificada del RANSAC estocástico de Kaneko et al. (2017) y de la estimación robusta de FOE en flujo de egomotion documentada en la literatura de detección de obstáculos por flujo residual (Moliner et al., 2012).

Confianza del FOE. Se distinguen dos casos: - Pocos inliers (< 30): FOE poco confiable; `foe_confidence` se clípea a 0.3 como señal de evidencia degradada. - Suficientes inliers (≥ 30): `foe_confidence = min(n_inliers / (H  $\times$  W)  $\times$  3.0, 1.0)`. El factor 3.0 compensa que la fracción de inliers esperada en una escena real es aproximadamente 1/3 del total de píxeles válidos (fondo vs. objetos en aproximación); el boost normaliza la confianza a un rango comparable al de la fracción bruta.

Sanity check de límites. Un FOE fuera del rectángulo de la imagen (`0  $\leq$  foe_x  $\leq$  W, 0  $\leq$  foe_y  $\leq$  H`) no es un punto de fuga físicamente plausible para esa imagen; indica que el ajuste convergió a una solución artefactual (ruido dominante o residuo de derotación mal compensada). En ese caso, `foe_confidence` se fuerza a 0.0 y el campo se retorna sin celdas bloqueadas.

6.7 Etapa 5: TTC por píxel y divergencia

Con el FOE estimado y el intervalo temporal real Δt (diferencia de timestamps del simulador entre frames consecutivos — nunca el período nominal del lazo), se calcula para cada píxel válido:

$$TTC(x,y) = \frac{|p - \text{FOE}| \cdot \Delta t}{\|\mathbf{v}_{\text{trans}}(x,y)\|}$$

Esta fórmula es la forma discreta de la ecuación continua $TTC = Z / v_{\text{cierre}}$, válida bajo la aproximación de cámara *pinhole* con traslación dominante. Para píxeles cerca del FOE o con flujo muy pequeño, el denominador puede producir TTC extremadamente alto (sin evidencia de aproximación), lo que se traduce en $TTC \rightarrow \infty$.

Canal de divergencia. En paralelo al TTC, el estimador computa la divergencia del campo traslacional como verificación independiente:

$$\text{div}(x,y) = \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y}$$

implementada con `np.gradient()` (diferencias de segundo orden centradas, normalizadas por el intervalo temporal Δt). La divergencia positiva indica expansión local del flujo — el objeto en ese punto del plano focal está acercándose al vehículo. Es un indicador complementario al TTC que no requiere estimar el FOE: incluso si el ajuste del FOE falla, una celda con divergencia positiva elevada sigue siendo evidencia de aproximación.

6.8 Grilla de celdas 3×3 y agregación por sector

El campo traslacional se divide en una grilla de 3 columnas (sectores: *izquierda*, *centro*, *derecha*) × 3 filas (bandas: *superior*, *medio*, *inferior*), totalizando 9 celdas. Las columnas dividen la imagen en tercios iguales de ancho; las filas, en tercios iguales de alto. Para cada celda se calcula:

Confianza (`confidence`): fracción de píxeles válidos (magnitud > piso de ruido) sobre el total de la celda, multiplicada por `foe_confidence`. Esta confianza combinada refleja tanto la densidad local de flujo válido como la calidad global del FOE estimado.

TTC de celda (`ttc_s`): percentil `TTC_AGGREGATION_PERCENTILE` (default 20) de la distribución de TTC finitos de la celda. El percentil 20 es conservador: estima el TTC del 20% más rápido de los objetos que se aproximan en la celda, resistente al ruido espurio que produce TTC extremadamente bajos en píxeles aislados. Celdas con menos de 5 píxeles válidos reciben `ttc_s = ∞` (sin evidencia).

Ocupación (`occupancy`): media de la divergencia positiva en la celda, escalada por el factor de calibración:

$$\text{occupancy} = \text{clip}(\bar{\text{div}} + \frac{0.450}{8.0}, 0, 1)$$

El factor $0.450 / 8.0$ fue determinado empíricamente comparando las salidas del estimador con profundidad de referencia del simulador; no es un valor físicamente derivado sino el mejor punto de operación encontrado en las condiciones de vuelo de los escenarios de esta tesis. El protocolo de validación completo —conjunto de datos, curva ROC y la interpretación correcta de las métricas resultantes— se describe en el capítulo 7.

6.9 El predicado de bloqueo: fusión de dos canales

La decisión de si una celda está **bloqueada** fusiona los dos canales (ocupación y TTC) con una compuerta disyuntiva, condicionada a la confianza:

```
def is_blocked(self) -> bool:
    if self.confidence < MIN_CONFIDENCE_FOR_BLOCKED: # 0.15
        return False
    if self.occupancy >= OCCUPANCY_BLOCKED_THRESHOLD: # 0.35
        return True
    return (
        self.confidence >= MIN_CONFIDENCE_FOR_TTC_BLOCKED # 0.35
        and self.ttc_s <= TTC_BLOCKED_THRESHOLD_S # 2.5 s
    )
```

La lógica es: una celda está bloqueada si tiene evidencia mínima de percepción **y** (la ocupación es alta, **o** el TTC es bajo con suficiente confianza).

Umbrales diferenciados de confianza. La confianza mínima para que la ocupación vote bloqueo es `MIN_CONFIDENCE_FOR_BLOCKED = 0.15`, pero para que el TTC vote bloqueo por sí solo (sin apoyo de ocupación) se requiere `MIN_CONFIDENCE_FOR_TTC_BLOCKED = 0.35`. La razón es que el camino de "pocos inliers" en la estimación del FOE (§6.6) produce `foe_confidence = 0.3` como señal de evidencia degradada. Sin el umbral diferenciado, ese 0.3 superaba el piso general (0.15) y el TTC degradado votaba bloqueo con la misma autoridad que un FOE robusto. La separación entre ambos umbrales fue el fix directo de una fuente documentada de falsos positivos (CHANGELOG.md 2026-0826).

6.10 La API pública de `ObstacleField`

Todos los consumidores del sistema de percepción — `policy_router`, `evasive_node`, `deliberative_node`, `fsm_node`, `FlightLogger` — acceden al campo de obstáculos **únicamente** a través de esta interfaz. Ningún módulo de control lee campos crudos de flujo ni coordenadas del FOE.

Método	Descripción
<code>is_blocked(sector)</code>	<code>True</code> si alguna celda del sector está bloqueada según §6.9
<code>sector_ttc(sector)</code>	Mínimo TTC del sector, sobre celdas con confianza ≥ 0.15
<code>sector_occupancy(sector)</code>	Máxima ocupación del sector, sobre celdas con confianza ≥ 0.15
<code>sector_confidence(sector)</code>	Máxima confianza del sector
<code>blocked_fraction()</code>	Fracción de celdas bloqueadas sobre el total de la grilla 3×3
<code>min_ttc()</code>	TTC mínimo global, sobre todas las celdas con confianza suficiente
<code>has_evidence()</code>	<code>True</code> si <code>source == "flow"</code> y <code>foe_confidence > 0.0</code>
<code>summary_text()</code>	Texto compacto para el prompt del VLM (§4.3): "CENTRO: BLOQUEADO (TTC=3.2s)"
<code>to_dict()</code>	Representación serializable para el JSONL de auditoría

El diseño como objeto inmutable (`@dataclass(frozen=True)`) garantiza que ningún consumidor pueda modificar el estado de percepción: los nodos solo pueden leer el campo, no escribirlo. Esta propiedad simplifica el razonamiento sobre el ciclo de control, donde el mismo `ObstacleField` es leído por hasta tres consumidores (router, nodo de política, logger) en el mismo ciclo.

6.11 Consultas de nivel superior: `has_open_corridor` y `sector_towards_waypoint`

Dos funciones de módulo sirven como interfaz de alto nivel compartida entre el router de política, el nodo deliberativo y la FSM:

`sector_towards_waypoint(bearing_err_deg)` mapea el error de rumbo al waypoint activo a uno de los tres sectores visuales: - Si `bearing_err_deg < -BEARING_SECTOR_DEG` (default 15°) → "izquierda" - Si `bearing_err_deg > +15°` → "derecha" - Caso contrario → "centro"

Esta función conecta el espacio de guiado de `WaypointTracker` con el espacio de percepción del `ObstacleField`, permitiendo que el escape de atasco priorice el sector hacia el waypoint.

`has_open_corridor(field, guidance)` determina si la percepción tiene evidencia real de al menos un sector transitable, evaluado en el contexto del waypoint activo:

```
if not field.has_evidence():
    return False          # sin flujo ≠ "despejado"
for sector in (target, otros_sectores):
    if confidence(sector) >= MIN_CONFIDENCE and not is_blocked(sector):
        return True
return False
```

La condición `has_evidence()` es crítica: un hover puro produce un `ObstacleField` con `foe_confidence = 0`, y tratar esa situación como "camino despejado" desactivaría el escape de atasco precisamente cuando el dron está parado y atrapado. Esta función fue introducida después de documentar un vuelo donde el dron subió 12 m consecutivos mientras el `ObstacleField` reportaba `DERECHA: DESPEJADO` ciclo tras ciclo — el router cortocircuitaba hacia `GANAR_ALTURA` antes de leer el campo (CHANGELOG.md 2026-0824).

6.12 Comportamiento bajo condiciones extremas

Hover / velocidad < 0.25 m/s. Sin traslación entre frames, el flujo traslacional es indistinguible del ruido del estimador. La fracción de píxeles válidos cae por debajo de `MIN_VALID_FRACTION_FOR_FOE = 1%` y el campo retorna vacío (`foe_confidence = 0`). `has_evidence()` devuelve `False`, marcando la situación como "sin información" — no como "despejado". El nodo deliberativo lo detecta vía `_query_reason_note()` y comunica explícitamente al VLM que la consulta se debe a falta de evidencia, no a un bloqueo real (§4.3, §5.10).

Giro puro (yaw_rate alto). La rotación excede `FLOW_MAX_ROTATION_DEG = 2°` y el ciclo retorna `empty_field(source="degraded")`. El lazo continúa con la maniobra comprometida (persistencia en `evasive_node`) sin recurrir a nueva evidencia perceptual.

Textura baja o uniformidad fotométrica. En regiones sin gradiente (cielo, fachadas, superficies sin textura), `np.gradient` produce valores de flujo cercanos a cero que no contribuyen a la estimación del FOE ni al cómputo de TTC. La confianza de esas celdas es cercana a cero y el predicado `is_blocked()` las descarta.

Transiciones de iluminación brusca. Cambios rápidos de exposición entre frames (que pueden ocurrir en el simulador al cruzar una sombra) producen flujo sistemático en toda la imagen que puede contaminar el FOE. El recorte de outliers (§6.6) mitiga esto parcialmente; la calibración de `FLOW_MAX_ROTATION_DEG` y el piso de ruido `FLOW_NOISE_FLOOR_PX` son los otros mecanismos de contención.

6.13 Variables de entorno calibrables

El estimador expone todos sus parámetros clave a través de variables de entorno (versionadas en `config/.env`):

Variable	Default	Descripción
<code>FLOW_ALGORITHM</code>	"dis"	Backend de flujo: "dis" o "farneback"
<code>FLOW_DOWNSCALE_WIDTH</code>	320	Ancho de trabajo para flujo óptico (px)
<code>CAMERA_FX</code> / <code>CAMERA_FY</code>	554.0	Distancia focal a resolución de captura (px)
<code>FLOW_NOISE_FLOOR_PX</code>	0.35	Piso de ruido: vectores más pequeños se descartan
<code>MIN_VALID_FRACTION_FOR_FOE</code>	0.01	Fracción mínima de píxeles válidos para estimar FOE
<code>FOE_OUTLIER_ANGLE_RAD</code>	0.35	Umbral de ángulo para RANSAC-lite (rad, $\approx 20^\circ$)
<code>TTC_AGGREGATION_PERCENTILE</code>	20	Percentil de TTC usado como estimado por celda
<code>FLOW_MAX_ROTATION_DEG</code>	2.0	Rotación máxima (°) para la que la derotación es confiable
<code>OBSTACLE_OCCUPANCY_BLOCKED</code>	0.35	Umbral de ocupación para declarar celda bloqueada
<code>OBSTACLE_TTC_BLOCKED_S</code>	2.5	Umbral de TTC para declarar celda bloqueada (s)
<code>OBSTACLE_MIN_CONFIDENCE</code>	0.15	Confianza mínima para que ocupación vote bloqueo
<code>OBSTACLE_MIN_CONFIDENCE_TTC</code>	0.35	Confianza mínima para que TTC vote bloqueo solo

6.14 Complementariedad entre percepción clásica y VLM

El diseño del sistema asume explícitamente que ninguno de los dos componentes es completo por sí solo:

El estimador de flujo es fuerte cuando el dron se traslada a velocidad moderada (> 0.5 m/s), la textura de la escena es suficiente, y la iluminación es razonablemente estable. En esas condiciones produce `ObstacleField` con `foe_confidence > 0.35` y bloqueos confiables que el router resuelve en < 5 ms sin consultar el VLM.

El VLM es fuerte cuando la percepción clásica falla: hover, giro puro, atasco donde el dron está parado y el flujo colapsó, o cuando se necesita contexto semántico (distinguir una calle libre de una fachada con ventanas, o un árbol de un edificio). El VLM recibe el fotograma directamente y puede razonar sobre la escena a pesar de la falta de movimiento, aunque con latencia de 200 ms a varios segundos.

La interfaz entre ambas capas es el campo `scene_summary` del `ObstacleField` y el motivo de consulta explícito del prompt (§4.3): cuando la percepción clásica tiene evidencia, el VLM la recibe como contexto cuantitativo ("CENTRO: BLOQUEADO, TTC=3.2s"); cuando no la tiene, el prompt lo dice explícitamente ("la percepción NO tiene evidencia suficiente en este ciclo, probablemente por falta de traslación"). Esta transparencia sobre la fuente y la calidad de la evidencia es el mecanismo que permite al modelo de lenguaje calibrar su propia respuesta según el nivel de confianza del canal perceptual.

Las referencias bibliográficas citadas en este capítulo corresponden a las entradas del §13 del presente informe. La evaluación cuantitativa del estimador (curvas ROC, AUC, comparación con profundidad de referencia) se desarrolla en el capítulo 7. El análisis de los modos de falla de la percepción y su impacto en el comportamiento del lazo se presenta en el capítulo 9.

7. Validación del estimador de TTC y calibración del campo de obstáculos

El capítulo 6 describe el modelo algorítmico del estimador de flujo óptico y su salida, el `ObstacleField`. Este capítulo responde una pregunta diferente: **¿cómo sabemos que el estimador funciona?** El enfoque adoptado es experimental — se recolectó un conjunto de datos con ground truth de profundidad del simulador y se aplicó análisis de curva ROC para calibrar y evaluar ambos canales (TTC y ocupación) con datos reales de vuelo.

7.1 Ground truth de profundidad en AirSim: ventaja metodológica

AirSim expone un canal de profundidad planar (`DepthPlanar`, en metros por píxel) sin costo computacional adicional: es un buffer de renderizado que el motor ya produce como subproducto de la rasterización. Esto elimina la necesidad de instrumentación externa (LiDAR, cámaras estéreo calibradas, marcadores en escena) y permite construir ground truth denso y de alta frecuencia en condiciones de vuelo idénticas a las de la misión real — la misma geometría, los mismos patrones de movimiento, el mismo ciclo de control (Shah et al., 2017; Jansen et al., 2023).

El TTC de referencia por celda se define como:

$$TTC_{\text{gt}}(\text{celda}) = \frac{\text{percentil}_{20}(Z)}{\max(v_{\text{cierre}}, \epsilon)}$$

donde Z es la distribución de profundidades de los píxeles de la celda (el percentil 20 actúa como estimado conservador resistente a píxeles de fondo), y v_{cierre} es la componente de la velocidad del vehículo proyectada sobre el eje óptico (velocidad de cierre frontal positiva = aproximación), extraída de la telemetría del simulador. El término $\epsilon = 0.01$ m/s evita la división por cero durante hover. Esta definición produce un TTC de referencia que es una función *observable* y *auditable* del estado físico del sistema en cada ciclo.

Una limitación de diseño del ground truth: la profundidad planar mide la distancia perpendicular al plano focal, no la distancia euclidiana al obstáculo más cercano en cada dirección de la cámara. Para obstáculos cercanos y cámaras con ángulo de campo amplio, esta diferencia no es negligible, pero dentro del rango de interés práctico (< 15 m) y con la óptica de 60° de FoV de AirSim, la discrepancia es inferior al 8% para el sector central. Se acepta como buena aproximación para la validación.

7.2 Diseño del conjunto de datos de validación

`experiments/collect_ttc_dataset.py` recolecta, por ciclo y por celda, el conjunto de variables listado en la tabla siguiente, generando registros JSONL en `runs/ttc/`:

Campo	Descripción
<code>ttc_est</code>	TTC estimado por <code>FlowTTCEstimator</code> (segundos)
<code>ttc_gt</code>	TTC de referencia por profundidad (segundos)
<code>occupancy</code>	Ocupación estimada por celda
<code>divergence_raw</code>	Divergencia bruta antes de escalar
<code>confidence</code>	Confianza de celda (fracción de píxeles válidos × <code>foe_confidence</code>)
<code>sector</code> , <code>band</code>	Identificación de celda en la grilla 3×3
<code>speed_mps</code>	Velocidad de traslación horizontal del vehículo (m/s)
<code>yaw_rate_rps</code>	Tasa de guiñada instantánea (rad/s)
<code>flow_algorithm</code>	Backend usado (<code>dis</code> o <code>farneback</code>)
<code>foe_confidence</code>	Confianza del FOE estimado (antes de multiplicar por fracción válida)

El conjunto recolectado totaliza **3735 registros** de 9 celdas cada uno, cubriendo tres escenarios de vuelo scripteados diseñados para ejercitar aspectos específicos del estimador:

1. **Aproximación frontal a un edificio:** vuelo recto hacia una fachada a distintas velocidades (1–6 m/s); maximiza la señal de expansión en el sector central y verifica la correlación TTC estimado vs. TTC de referencia en condiciones ideales.
2. **Vuelo por un cañón urbano:** desplazamiento lateral entre dos edificios; activa los sectores izquierdo y derecho mientras el central permanece mayormente libre; verifica que la grilla detecta amenazas laterales sin generar falsos positivos en el centro.
3. **Giros de guiñada sin traslación:** rotación pura sobre el eje vertical, diseñada para ejercitar el guard de rotación (`FLOW_MAX_ROTATION_DEG`) y la derotación de guiñada. La expectativa correcta es que, durante el giro, el estimador declare incertidumbre (`foe_confidence = 0`) en lugar de reportar obstáculos espurios.

Limitación documentada del conjunto de datos: el escenario de giros de guiñada quedó concentrado en el rango $|\dot{\psi}| \in [0.0, 0.05)$ rad/s — velocidades de guiñada muy bajas, más próximas al cruce suave que a las maniobras activas de evasión (0.3–0.5 rad/s). Este rango no ejerce presión real sobre la derotación, y la validación de la derotación en giros agresivos queda aún pendiente (§7.5).

7.3 Resultados del canal de TTC: dos métricas, una interpretación correcta

El análisis en `experiments/analyze_ttc.py` produce dos números que, leídos de forma independiente, parecen contradictorios:

Métrica 1 — Correlación puntual: $r \approx -0.034$, error relativo mediano del 66%.

Métrica 2 — AUC ROC para detección binaria: 0.96–0.97 para el evento "colisión dentro de τ segundos" con $\tau \in \{1, 2, 3\}$.

La reconciliación entre estos dos números es el resultado estadístico central de este capítulo. Procede así:

¿Por qué la correlación puntual es baja? La correlación de Pearson mide si existe una relación *lineal* entre dos variables. Un TTC estimado de 4.8 s cuando el TTC real es 3.1 s (error puntual de 55%) contribuye igual a la correlación que si ese par estuviera perfectamente alineado o *no*. El estimador de flujo tiene errores de escala sistemáticos (el modelo de cámara pinhole asume movimiento rígido puro; los residuos de derotación, la profundidad desigual de objetos en la misma celda, y el efecto de la perspectiva en celdas no centrales introducen sesgos). Esos errores de escala no son aleatorios: son sistemáticos y predecibles. En consecuencia, la correlación puntual es baja, pero el *ordenamiento relativo* de los TTC estimados sigue siendo correcto: una celda con TTC estimado de 2.0 s es consistentemente más peligrosa que una con 5.0 s.

¿Por qué el AUC es alto? El AUC ROC mide exactamente la propiedad que la correlación no mide: la capacidad del estimador de *ordenar* los casos por riesgo. Un AUC de 0.96 significa que, si se escoge al azar una celda "peligrosa" ($\text{TTC real} < \tau$) y una celda "segura" ($\text{TTC real} \geq \tau$), el estimador le asigna un TTC menor a la peligrosa con probabilidad 0.96. Ese es el predicado exacto que el sistema de control necesita: no necesita saber que faltan 3.2 s para la colisión; necesita saber que *esta celda* es más peligrosa que *aquella*.

Formulación defendible del estimador: "El TTC estimado es un *puntaje de riesgo de cierre calibrado por curva ROC*, cuyo valor numérico tiene unidades de segundos por construcción matemática del modelo (§6.7) pero no interpretación métrica directa. Los umbrales operativos (`TTC_EVASION_THRESHOLD`, `TTC_SAFE_THRESHOLD`) son umbrales en esa escala de puntaje, derivados por criterio de Youden, no parámetros físicos independientes."

Esta distinción entre *clasificador de riesgo* y *cronómetro* se alinea con la literatura sobre sistemas de advertencia de colisión en visión monocular: Al-Kaff et al. (2017) y Vera-Yanez et al. (2024) reportan resultados similares — correlación puntual pobre pero detección binaria confiable — y argumentan que la métrica relevante para evasión es la tasa de verdaderos positivos antes de la zona de peligro, no la precisión de la estimación en segundos.

7.4 Derivación de umbrales operativos por índice de Youden

El índice de Youden ($J = \text{sensibilidad} + \text{especificidad} - 1$) es el criterio de selección de umbral óptimo que maximiza simultáneamente la tasa de detección de peligro real y la de descarte de falsos peligros. Para cada τ se barrió el espacio de umbrales sobre la curva ROC y se encontró:

Tiempo de horizonte τ	Umbral de TTC (Youden)	Sensibilidad	Especificidad
1 s	~2.1 s	0.91	0.88
2 s	~3.18 s	0.93	0.89
3 s	~4.58 s	0.90	0.91

El umbral para $\tau = 2$ s (3.18 s) fue adoptado como `TTC_EVASION_THRESHOLD` (redondeado a 3.2 s), reemplazando el valor provisorio de 3.0 s que se usaba antes de la validación. El umbral para $\tau = 3$ s (4.58 s) fue adoptado como `TTC_SAFE_THRESHOLD` (redondeado a 4.6 s), reemplazando 6.0 s. Ambos umbrales son datos de validación — no fueron elegidos a mano ni ajustados empíricamente por comportamiento de vuelo. La elección de $\tau = 2$ s como umbral de evasión y $\tau = 3$ s como umbral de zona segura refleja el tiempo de reacción del lazo (5 Hz $\rightarrow$ 200 ms por ciclo) más el tiempo de ejecución de la maniobra de evasión (1–2 ciclos); a esa velocidad de crucero el margen de 2 s es suficiente para iniciar evasión antes del impacto en los escenarios ensayados.

Interpretación de la banda entre umbrales. La zona $TTC \in (3.2, 4.6)$ s corresponde al estado "advertencia" en el `policy_router` (cap. 5): no hay bloqueo activo pero el riesgo es elevado; el sistema emite una solicitud al deliberativo y reduce la agresividad lateral de `evasive_node`. La banda de histéresis entre evasión (3.2 s) y zona segura (4.6 s) previene el *chattering* — oscilación rápida entre maniobra de evasión y crucero normal cuando el TTC estimado flota alrededor del umbral de evasión.

7.5 Calibración del canal de ocupación (estado y pendientes)

A diferencia del canal de TTC, el canal de ocupación (divergencia escalada $\rightarrow$ `occupancy`) no pasó todavía por el mismo protocolo de validación contra profundidad. La calibración actual del factor $0.450/8.0$ (descrita en §6.8 del capítulo 6) se realizó con un subconjunto reducido de los datos de `runs/ttc/` y no produjo una curva ROC completa; el factor fue ajustado manualmente hasta que el comportamiento de vuelo en los escenarios de prueba fue satisfactorio. No es un proceso de calibración estadísticamente controlado.

La validación pendiente consiste en:

1. Ejecutar `experiments/analyze_occupancy.py` sobre el conjunto completo (3735 registros), usando como referencia binaria la profundidad de celda < 10 m.
2. Construir la curva ROC del predicado `occupancy  $\geq$  OBSTACLE_OCCUPANCY_BLOCKED`.
3. Calcular el umbral óptimo por Youden, análogamente a §7.4.

4. Verificar que el umbral derivado estadísticamente coincida con el valor operativo actual (0.35), o actualizarlo.

El `OBSTACLE_OCCUPANCY_BLOCKED = 0.35` es por tanto un valor provisorio que funciona en la práctica pero no tiene respaldo estadístico equivalente al del canal de TTC. Esta asimetría en la validación de los dos canales es una limitación documentada del estado actual del sistema.

Métrica de validación	Canal TTC	Canal Ocupación
Protocolo de validación ROC	✓ Completo	X Pendiente
Umbral derivado por Youden	✓ (<code>TTC_EVASION_THRESHOLD = 3.2 s</code>)	X (valor provisorio = 0.35)
AUC reportada	0.96–0.97	No disponible
Conjunto de datos	3735 registros	Subconjunto no documentado

7.6 Validación de la derotación en giros agresivos (pendiente)

El guard `FLOW_MAX_ROTATION_DEG = 2°` descarta todos los ciclos donde la rotación entre frames supera ese umbral (cap. 6). El conjunto de datos de validación actual no ejercita este guard de forma significativa: el 97% de los registros del escenario de guiñada cae en $|\dot{\psi}| < 0.05$ rad/s.

Para completar la validación, es necesario un escenario específico con guiñada de ± 0.3 – 0.5 rad/s sin traslación, que ejercite tanto la correcta inhibición del estimador (el guard debe activarse y retornar `empty_field`) como la eventual reanudación correcta de la estimación una vez que la rotación decrece. También sería informativo estratificar el error relativo de TTC por bins de tasa de guiñada con los datos existentes: si el error en $|\dot{\psi}| \in [0.03, 0.05]$ rad/s es significativamente mayor que en bins más bajos, sugeriría un error de escala o de sincronización entre la telemetría de actitud y el timestamp del frame que debería corregirse antes de considerar cerrada la validación del canal de TTC.

Esta ampliación queda como trabajo pendiente para la versión final del sistema antes de pruebas en hardware real (cap. 9).

7.7 Implicaciones para el diseño del sistema

Los resultados de validación tienen consecuencias directas sobre el diseño del lazo de control:

El AUC de 0.96–0.97 justifica usar TTC como señal primaria de evasión. Una tasa de verdaderos positivos del 93% con especificidad del 89% (a $\tau = 2$ s) es suficiente para evasión reactiva en entorno simulado; los falsos negativos (11%) son la fracción de situaciones peligrosas que el estimador no detecta a tiempo, justificando la capa deliberativa como red de seguridad.

La baja correlación puntual justifica la banda de histéresis y el rol del VLM. Si el TTC fuera un cronómetro confiable, bastaría un solo umbral. La incertidumbre de escala hace necesaria la zona de advertencia (3.2–4.6 s) y la consulta al VLM antes de comprometer una maniobra definitiva: el modelo de lenguaje provee contexto semántico que el estimador de flujo no puede dar.

La validación pendiente del canal de ocupación es deuda técnica controlada. El sistema funciona con el umbral provisorio de ocupación porque en los escenarios ensayados, el TTC es el canal primario de decisión y la ocupación actúa como corroboración secundaria. Sin embargo, antes de despliegue en entornos más exigentes (o en hardware real), el protocolo de validación del canal de ocupación debería completarse.

8. Ingeniería de decisiones del SLM

El capítulo 5 documenta la *arquitectura interna* del nodo deliberativo (cuándo se activa, cómo se integra en el grafo LangGraph, qué hace con la respuesta del modelo). El capítulo 6 documenta *por qué* hace falta un VLM (los puntos ciegos estructurales del estimador de flujo). Este capítulo documenta las decisiones de ingeniería que hacen que ese VLM *funcione de forma confiable en la práctica*: selección del modelo bajo restricciones de hardware, mecanismos de salida estructurada, diseño del espacio de acción, ingeniería del prompt y trazabilidad de las decisiones.

8.1 Selección del modelo: restricciones de hardware como parámetro de diseño

La selección del modelo de lenguaje no fue un proceso de *benchmark* abstracto sino un proceso de diseño con restricciones duras. El sistema corre en una RTX 5060 con 8 GB de VRAM compartidos con una simulación de Unreal Engine 5.5. AirSim sobre UE5.5 consume, en condiciones de vuelo activo, entre 4.5 y 6 GB de VRAM (Turco et al., 2024). El presupuesto efectivo disponible para la inferencia del modelo es de **2 a 3.5 GB de VRAM**, incluyendo la KV cache del historial de prompt.

Esta restricción excluye inmediatamente a los modelos de referencia del estado del arte (GPT-4o, Claude 3.5, Gemini 1.5 — todos en la nube o con hardware dedicado de 24–80 GB) y orienta la búsqueda hacia modelos multimodales compactos cuantizados (*Small Vision-Language Models*, VLM), del orden de 3 a 7 mil millones de parámetros, ejecutados localmente bajo LM Studio o vLLM.

8.1.1 El requisito de visión directa

Una distinción de diseño previa a la selección del modelo es si el nodo deliberativo debe usar un modelo textual puro (recibe solo telemetría y resúmenes de percepción serializados) o un VLM multimodal (recibe además el fotograma de la cámara). El sistema adopta un VLM multimodal por razones que no son de conveniencia sino de cobertura funcional:

- **Puntos ciegos del flujo óptico** (§6.1, §6.12): cuando el estimador de flujo carece de evidencia (hover, giro puro, baja textura), el `ObstacleField` entregado al modelo es esencialmente vacío. Un modelo textual no puede razonar sobre la escena sin ese resumen; un VLM puede observar directamente el fotograma y evaluar el contexto visual.
- **Semántica que el flujo no codifica**: la distinción entre "árbol vs. edificio", "sombra vs. obstáculo sólido", o "calle con objetos vs. calle vacía" no está presente en el campo de divergencia. El VLM puede discriminar escenas semánticamente similares en percepción pero distintas en consecuencias de navegación.
- **Historial temporal mínimo**: el campo `VLM_FRAME_HISTORY_SIZE = 2` envía los frames t y $t-1$ juntos, permitiendo al modelo estimar la dirección de aproximación a partir de la diferencia visual entre frames, sin necesidad de que el flujo óptico sea confiable.

8.1.2 Modelo adoptado y candidatos evaluados

El modelo adoptado en la implementación definitiva es **Qwen2.5-VL-3B-Instruct** (cuantización Q4_K_M, ~2.0 GB de VRAM), ejecutado vía LM Studio. Combina capacidad de razonamiento simbólico, comprensión visual real de la cámara y seguimiento de instrucciones estructuradas. Los modelos candidatos evaluados durante el diseño (documentados en `informe/anexos/A1-EXPLORACION-SLM-GGUF.md`) son:

Modelo	Tamaño (cuant.)	VRAM aprox.	Descartado por
Phi-4-mini-Instruct	~2.5 GB	~2–3 GB	Sin capacidad de visión nativa
Qwen3-4B-Instruct	~3 GB	~2.5–3.5 GB	Sin soporte visual en la rama 3B
SmolLM3-3B	~2 GB	~1.8 GB	Sin soporte visual; razonamiento débil
Qwen2.5-VL-3B-Instruct	~2 GB	~2.0 GB	Adoptado
Qwen2.5-VL-7B-Instruct	~4.5 GB	~4–5 GB	VRAM insuficiente con UE5.5 activo

El criterio de selección fue: **(a)** soporte de visión nativa, **(b)** VRAM < 2.5 GB con Q4_K_M, **(c)** soporte de `response_format: json_schema` en el backend de inferencia, **(d)** latencia de inferencia < 2 s en el hardware disponible para prompts de ~500 tokens con dos imágenes.

8.1.3 Posicionamiento respecto al estado del arte

La arquitectura de lazo cerrado estado→VLM→macro-acción→ejecución no es, hacia 2025–2026, una contribución conceptualmente novedosa: es un patrón estandarizado en investigación y prototipos de UAVs controlados por modelos de lenguaje (véase la discusión en `informe/anexos/A1-EXPLORACION-SLM-GGUF.md`). Hwang et al. (2024) en EMMA y Saxena et al. (2025) en UAV-VLN son trabajos representativos del patrón. El aporte de esta tesis no está en la novedad de la arquitectura general, sino en la ingeniería de su interfaz con la percepción monocular (cap. 6) y en la caracterización sistemática de sus modos de falla (cap. 9).

8.2 Salida estructurada: decodificación restringida y parser tolerante

La primera causa documentada de fallo del nodo deliberativo en versiones tempranas del sistema no fue una decisión de navegación incorrecta, sino una respuesta JSON malformada que rompía el parser y dejaba al dron sin maniobra durante 3–5 ciclos. La solución adoptada opera en dos capas complementarias.

8.2.1 Decodificación restringida (`json_schema`)

La respuesta del modelo se solicita con el parámetro `response_format={"type": "json_schema", "json_schema": {...}}`, disponible en LM Studio (a partir de la versión con llama.cpp ≥ 0.5) y en el backend OpenAI-compatible de Ollama. La decodificación restringida opera durante la generación token a token: en cada paso, el motor de inferencia evalúa las reglas del esquema y enmascara los logits de tokens que resultarían en JSON inválido para el esquema declarado, forzando al modelo a muestrear únicamente de los tokens válidos (Raspanti et al., 2025; Geng et al., 2025).

El efecto práctico es que, cuando la decodificación restringida está activa, es estructuralmente imposible producir una respuesta que no sea JSON conforme al esquema. El `adherence_rate` (fracción de respuestas parseables al primer intento) medido en los escenarios de validación sube de ~73% (solo prompt) a ~98% (con `json_schema`).

El esquema declarado tiene la forma mínima necesaria para la toma de decisión:

```
{
  "type": "object",
  "properties": {
    "action": {
      "type": "string",
      "enum": ["keep_going", "evasive", "girar_90", "fsm", "degraded"]
    },
    "reason": { "type": "string", "maxLength": 120 }
  },
  "required": ["action", "reason"],
  "additionalProperties": false
}
```

La enumeración explícita de los valores posibles de `action` (§8.3) es el mecanismo que convierte el espacio de acción discreto del sistema en una restricción gramatical directamente aplicable por el motor de gramática.

8.2.2 Parser tolerante como red de seguridad

La garantía de `json_schema` depende de que el backend local soporte la versión de llama.cpp que implementa esa funcionalidad. En entornos de despliegue heterogéneos (versión de LM Studio distinta, Ollama sin la extensión, o futura migración a otro backend), esa garantía puede no cumplirse. Por eso, `_parse_decision()` (`src/agents/deliberative.py`) se conserva como red de seguridad y aplica tres estrategias de extracción en orden creciente de tolerancia:

1. `json.loads()` **directo**: para respuestas bien formadas sin envoltura.
2. **Extracción de bloque JSON con regex**: detecta el bloque `{...}` más externo en una respuesta que incluye markdown, texto explicativo o prefijos conversacionales.
3. **Búsqueda de campo `action` por expresión regular**: extrae el valor del campo `action` sin necesidad de parsear el JSON completo, como último recurso cuando la respuesta está truncada o mal formada.

Si las tres estrategias fallan, `_fallback_decision()` entra en vigor: devuelve `keep_going` con una nota de error, lo que es conservador (continuar la maniobra actual) y auditable (la nota aparece en el log).

La métrica `adherence_rate` es una fila explícita de la tabla de resultados (cap. 11): cuantifica cuánto aporta, en la práctica, la decodificación restringida por sobre el parser tolerante como única defensa. Este número importa porque si fuera cercano al 100% solo con el parser, la decodificación restringida sería overhead sin beneficio; si la diferencia es grande (como documentan Raspanti et al. [2025] y Geng et al. [2025] para SLMs compactos), la restricción es una decisión de ingeniería con impacto medible en la fiabilidad del sistema.

8.3 Espacio de acción discreto: lista blanca de macro-acciones

El VLM no produce comandos cinemáticos directamente (velocidades en m/s, tasas de guiñada en rad/s). Elige una etiqueta de un conjunto fijo de macro-acciones:

Macro-acción	Significado navegacional
<code>keep_going</code>	Continuar hacia el waypoint activo sin alterar la velocidad ni el rumbo
<code>evasive</code>	Iniciar evasión lateral (dirección decidida por <code>evasive_node</code> según ocupación L/R)
<code>girar_90</code>	Ejecutar un giro de 90° en la dirección menos bloqueada para desatasco
<code>fsm</code>	Delegar la decisión a la máquina de estados FSM (brazo alternativo)
<code>degraded</code>	Declarar modo degradado; pasar a <code>degraded_hover_node</code>

Esta arquitectura de lista blanca tiene tres propiedades de diseño que se derivan mutuamente:

1. Compatibilidad directa con la decodificación restringida. Un espacio de acción finito y conocido a priori puede representarse como una enumeración JSON (`enum`), lo que hace posible convertir la restricción de salida en una gramática aplicable en tiempo de inferencia. Un espacio continuo de comandos cinemáticos no puede representarse así sin esquemas mucho más complejos.

2. Comparación limpia entre brazos. El nodo deliberativo (VLM), el nodo FSM y el nodo reactivo comparten el mismo espacio de macro-acciones y el mismo traductor a comandos `action_to_command()`. La única variable que difiere entre ellos es *quién elige la etiqueta*, no *qué puede elegir ni cómo se ejecuta*. Esto hace posible la comparación experimental de los tres brazos (cap. 10) sin confundir diferencias de política con diferencias de actuación.

3. Auditabilidad. Una secuencia de etiquetas de macro-acción es legible por humanos y directamente graféable (cap. 4, viewer). Un log de vectores cinemáticos continuos no tiene esa propiedad sin postprocesamiento.

La lista de macro-acciones es fija en el diseño actual y no extensible dinámicamente: agregar una nueva macro-acción requiere modificar el esquema JSON, el `action_to_command()`, los prompts y la FSM. Este acoplamiento es deliberado — es el mecanismo que impide que el modelo de lenguaje invente acciones fuera del espacio de maniobras seguras.

8.4 `action_to_command`: la frontera entre lenguaje y cinemática

Cada macro-acción se traduce a un comando cinemático concreto a través de `action_to_command()` (`src/agents/action_map.py`), función compartida de forma estricta por los nodos deliberativo, de evasión y de FSM. La traducción completa es:

Macro-acción	vx (frontal)	vy (lateral)	vz (vertical)	yaw_rate
MANTENER_RUMBO	FORWARD_SPEED	0	0	toward wp
EVADIR_IZQUIERDA	EVASION_FORWARD	- EVASION_LATERAL	0	EVASION_LATERAL_YAW_RATE
EVADIR_DERECHA	EVASION_FORWARD	+EVASION_LATERAL	0	- EVASION_LATERAL_YAW_RATE
GANAR_ALTURA	0	0	- EVASION_UP_SPEED	0
PERDER_ALTURA	0	0	+EVASION_DOWN_SPEED	0
FRENAR	0	0	(P-ctrl. alt.)	0
GIRAR_90	(manejado por girar_90_node)	—	—	—

Los valores constantes son variables de entorno: `EVASION_LATERAL_YAW_RATE = 15.0` °/s, `EVASION_UP_SPEED = 1.5` m/s, `EVASION_DOWN_SPEED = 0.8` m/s, `CORNER_OFFSET_M = 12.0` m.

Invariante de diseño. Centralizar la traducción en una sola función garantiza que ninguna macro-acción puede tener definiciones cinemáticas distintas según quién la active. Antes de que `action_to_command()` fuera la única fuente de verdad, el nodo FSM y el nodo deliberativo tenían sus propias tablas de velocidades, que divergieron sutilmente (el FSM usaba `vx = 1.5` m/s para `GANAR_ALTURA`, el deliberativo usaba `1.0` m/s). Esa asimetría provocó trayectorias asimétricas entre brazos durante las pruebas A/B que dificultaron la comparación (CHANGELOG.md 2026-0824). La función centralizada elimina esa clase de divergencia estructuralmente.

8.5 Ingeniería del prompt: cinco componentes

El prompt de usuario construido por `_build_user_prompt()` tiene cinco componentes estructurados, cada uno respondiendo a una pregunta específica que el VLM necesita para decidir:

Componente 1 — Estado de vuelo y telemetría. Velocidad horizontal, velocidad vertical, altitud, actitud (pitch, roll), estado de misión, waypoint activo y distancia restante. Este componente traduce telemetría numérica cruda a descripciones en lenguaje natural ("el dron vuela a 4.2 m/s, con 23 m al próximo waypoint, pitch -3.1°"). La verbalización explícita se prefiere a la telemetría cruda porque los SLMs de 3B razonan más confiablemente sobre descripciones en lenguaje natural que sobre tuplas de números (Zhu et al., 2024).

Componente 2 — Resumen del `ObstacleField`. El resultado de `ObstacleField.summary_text()` para cada sector activo: TTC, ocupación, confianza, estado de bloqueo. Cuando el campo tiene evidencia, este componente provee información cuantitativa sobre la geometría del obstáculo. Cuando no la tiene, el componente dice explícitamente "percepción SIN evidencia" y describe la causa probable (velocidad baja, rotación activa), lo que permite al VLM distinguir "no hay obstáculo" de "no sé si hay obstáculo".

Componente 3 — Historial de decisiones recientes. Las últimas `N` acciones tomadas (con sus razones, si el deliberativo las reportó), incluyendo cuántos ciclos lleva activa la maniobra actual y si hay señales de atasco (progreso hacia el waypoint < umbral durante varios ciclos). Este componente provee contexto temporal que una sola imagen no puede dar: el VLM puede distinguir "empecé a evadir hace 1 ciclo" de "llevo 8 ciclos evasión sin avanzar".

Componente 4 — Motivo explícito de consulta. El campo `reason_note` codifica *por qué* el nodo deliberativo fue activado en este ciclo: `"TTC_CRITICO"` (TTC < umbral de evasión), `"TTC_ADVERTENCIA"` (zona de histéresis), `"FALTA_EVIDENCIA"` (flujo colapsado), `"DEADLOCK_ESCAPE"` (atasco detectado), `"DEEP_SCAN_RESULT"` (resultado de exploración rotacional). Este campo es crítico para calibrar la respuesta: ante `"FALTA_EVIDENCIA"`, el VLM no debe reportar obstáculos que no ve; ante `"TTC_CRITICO"`, debe priorizar evasión inmediata.

Componente 5 — Instrucción de formato de salida. Reiteración explícita del esquema JSON esperado y de los valores posibles del campo `action`, coherente con el `json_schema` declarado en el parámetro de formato. Esta redundancia entre prompt y schema es intencional: para modelos pequeños, la instrucción textual explícita mejora el `adherence_rate` incluso cuando la decodificación restringida está activa, especialmente en la elección del campo `action` dentro del dominio de la enumeración (Geng et al., 2025).

Parámetros de inferencia. `temperature = 0.2` (baja entropía, respuestas reproducibles y conservadoras), `max_tokens = 200` (suficiente para la estructura JSON más una razón breve; evita respuestas extensas que agoten el presupuesto de la KV cache y aumenten la latencia). El límite de tokens no es arbitrario: con `max_tokens = 200` y el esquema mínimo, la generación termina en 0.8–1.2 s en el hardware disponible; con `max_tokens = 500` subiría a 2–3 s, cruzando el watchdog `SLM_WATCHDOG_MS = 1500`.

8.6 Gestión de latencia: el watchdog y el creep speed

El tiempo de inferencia del VLM (0.8–2.0 s en el hardware disponible) es un orden de magnitud mayor que el período del lazo de control (200 ms a 5 Hz). Esto hace imposible bloquear el lazo esperando la respuesta; el nodo deliberativo opera de forma asincrónica a través de `DeLiberationService` (§5.10 del cap. 5), que corre en un daemon thread independiente.

La consecuencia directa es que, durante el tiempo que el VLM procesa, el lazo de control continúa operando. En esos ciclos, el estado `"pending_deLiberation"` activa `DELIB_WAIT_CREEP_SPEED_MPS = 0.5` m/s como velocidad de avance reducida — el dron continúa moviéndose muy lentamente en dirección al waypoint en lugar de detenerse, previniendo que la pérdida de velocidad traslacional colapse el campo de flujo óptico y elimine la evidencia perceptual justo cuando el sistema la necesita para la decisión pendiente.

El guard `SLM_WATCHDOG_MS = 1500` (§5.10) descarta respuestas que lleguen después de ese límite y activa `_fallback_decision()`. El watchdog tiene el efecto de que latencias de inferencia mayores a 1.5 s — frecuentes con el modelo de 7B bajo carga — equivalen a no tener deliberativo: el sistema cae en `keep_going` con nota de timeout. Esto refuerza la elección del modelo de 3B: una respuesta de calidad media entregada en 1.0 s vale más en este sistema que una respuesta de calidad alta entregada en 2.5 s.

8.7 Nota: LoRA como alternativa explorada y no adoptada

El fine-tuning ligero vía LoRA (*Low-Rank Adaptation*) fue evaluado durante la etapa de planificación como posible vía de especialización del modelo para el espacio de maniobras de este sistema (`informe/anexos/A4-OPTIMIZACION-LoRA.md`). El argumento a favor era que un conjunto de pares (prompt, acción correcta) derivados de las corridas exitosas del sistema podría reducir la frecuencia de respuestas subóptimas sin reentrenar el modelo base.

No se implementó en el pipeline final por dos razones:

1. **Cobertura de datos insuficiente.** El conjunto de corridas disponible en el momento de la evaluación (piloto de 3 tiers, cap. 11) contiene situaciones de evasión relativamente homogéneas. Un LoRA entrenado sobre ese conjunto podría sobreajustar a los escenarios vistos y degradar la generalización en configuraciones urbanas no ensayadas.
2. **Complejidad de mantenimiento.** Un adaptador LoRA es un artefacto de entrenamiento que requiere ser versionado, evaluado y actualizado junto con el modelo base. Para el alcance de esta tesis, el beneficio esperado no justificó el costo de infraestructura. Se deja constancia explícita de que es una alternativa explorada y no adoptada por decisión de alcance, no por omisión.

9. Modos de falla en lazos de control híbridos con LLM

9.1 El argumento central: contratos de interfaz como vector de falla primario

En sistemas robóticos que combinan percepción clásica, orquestación de grafo de control y un modelo de lenguaje, la intuición natural sobre modos de falla apunta al modelo: respuestas alucinadas, razonamiento incorrecto, formato inválido. La experiencia de desarrollo de este sistema señala en una dirección diferente: los fallos más costosos —en términos de horas de diagnóstico y de consecuencias en vuelo— ocurrieron en los **contratos de interfaz entre componentes**, no en el razonamiento del modelo.

Un modelo de lenguaje tiene una propiedad que lo hace especialmente peligroso en este contexto: raciocina sobre lo que recibe, no sobre la realidad. Si recibe un resumen de percepción que dice "camino despejado", produce una respuesta consistente con esa premisa. No tiene acceso privilegiado a la verdad del mundo — solo al texto que se le entrega. El corolario es que cualquier corrupción, omisión o malinterpretación en los datos que llegan al modelo produce decisiones estructuralmente correctas pero factualmente equivocadas, sin que el modelo (ni el sistema) lo detecte como error. El sistema "funciona" — no arroja excepciones, no se cae, produce salida JSON válida — mientras toma decisiones sistemáticamente incorrectas.

Este capítulo documenta cinco familias de fallas estructurales identificadas experimentalmente en la arquitectura, organizadas por la capa del sistema donde se originan. La tabla siguiente resume su taxonomía:

#	Categoría	Capa de origen	¿Genera excepción?	Consecuencia en vuelo
1	Schema drift / campo ciego	Contrato percepción→control	No	Navegación a ciegas hacia obstáculos
2	Desalineación temporal de frames	Buffer de historial visual	No	FOE/movimiento inferido incorrectamente
3	Descalibración de escala en divergencia	Operador diferencial en flujo	No	Saturación del canal de ocupación
4	State clipping + ciclos límite en LangGraph	Orquestación de grafo	No	Ascenso acumulado de >350 m
5	Degradación sensorial silenciosa	Captura de imagen y telemetría	No	Deliberación sobre datos corruptos

El denominador común en los cinco casos es la ausencia de excepción en tiempo de ejecución. Este resultado metodológico se discute en §9.7.

9.2 Falla 1 — Schema drift: el campo nulo leído como ausencia de peligro

9.2.1 Descripción del modo de falla

En una arquitectura temprana del sistema, el estado del campo de obstáculos se comunicaba como una lista de objetos detectados (`detected_obstacles: List[dict]`), con una lista vacía como valor por defecto. La decisión de qué hacer cuando la lista estaba vacía era ambigua: podía significar "la percepción corrió y no detectó nada" (espacio despejado) o "la percepción no corrió, no tiene confianza, o falló silenciosamente" (ausencia de información).

Los módulos consumidores — el router de política, el constructor del prompt del VLM, el nodo FSM — resolvían esa ambigüedad en favor de la primera interpretación. El resultado en la práctica fue:

1. El generador de contexto construía un prompt que afirmaba al modelo "trayectoria frontal sin obstáculos detectados".
2. El VLM, actuando con perfecta coherencia lógica respecto de la información provista, emitía `keep_going` con alta confianza.
3. El lazo ejecutaba la maniobra de crucero normal.
4. No se generaba ninguna excepción ni alerta de log, porque desde la perspectiva del código, la lista vacía es un valor válido.

Este patrón es una instancia del anti-patrón de diseño conocido como *"zero as absence of evidence"*: tratar el valor neutro de un tipo de dato como evidencia de ausencia de fenómeno (Zhu et al., 2024). En señales de seguridad, la convención correcta es la inversa: la ausencia de evidencia no es evidencia de ausencia de peligro.

9.2.2 Mecanismo de contención

La solución adoptada fue rediseñar el contrato de percepción como el tipo `ObstacleField` (cap. 6): un objeto inmutable que distingue explícitamente tres estados epistémicos:

- `source = "flow", foe_confidence > 0`: la percepción corrió con evidencia real. Los valores de TTC y ocupación son interpretables.
- `source = "flow", foe_confidence = 0`: la percepción corrió pero no produjo evidencia confiable (FOE fuera de imagen, menos de 30 inliers). Interpretable como "sin información".
- `source = "degraded"`: la percepción fue inhibida activamente (guard de rotación, modo degradado, primer ciclo). El campo no contiene ninguna información sobre el mundo.

La invariante que impone `ObstacleField` es que **ningún consumidor puede confundir "sin obstáculos detectados" con "sin información"** sin leer explícitamente `has_evidence()`. El prompt del VLM verbaliza esta distinción en el componente 2 (§8.5): "percepción SIN evidencia por baja velocidad traslacional" es un texto distinto de "todos los sectores despejados".

En los nodos de control, `degraded_hover_node` es el destino de cualquier ciclo donde `has_evidence()` es `False` y el router no tiene una maniobra persistente activa — el sistema no puede decidir "avanzar" si no sabe nada del entorno.

9.3 Falla 2 — Desalineación temporal del historial de frames

9.3.1 Descripción del modo de falla

El nodo deliberativo multimodal envía al VLM un historial de `VLM_FRAME_HISTORY_SIZE = 2` frames (frames t y $t-1$) para que el modelo pueda estimar la dirección de movimiento comparando dos instantes temporales distintos. Esta funcionalidad depende de que los dos frames sean genuinamente distintos en el tiempo.

En versiones tempranas, el buffer de historial era una lista Python simple que se actualizaba sin verificación de timestamps. Dos situaciones producían frames duplicados silenciosamente:

1. **Reinicio del buffer sin vaciado:** al saltar al siguiente waypoint, se vaciaba el historial pero el nodo de captura podía inyectar el último frame del waypoint anterior como "t-1" del nuevo waypoint. El VLM recibía el frame de un punto de la misión completamente distinto como contexto temporal inmediato.
2. **Ciclos donde `capture_node` no producía un frame nuevo:** si AirSim no respondía a tiempo (picos de carga de GPU), el buffer retenía el frame anterior. Bajo la etiqueta `[Fotograma t-1]` y `[Fotograma t]` aparecía la misma imagen.

El VLM, recibiendo dos frames idénticos, infería dos conclusiones posibles: el dron está estático (lo que puede ser correcto en hover pero es incorrecto durante crucero), o los objetos en escena están expandiéndose a velocidad cero (sin señal de aproximación). En ambos casos, la deliberación sobre movimiento y peligro era incorrecta de forma sistemática.

9.3.2 Mecanismo de contención

El buffer de historial se reimplementó como un `deque` de capacidad fija con control estricto de timestamps. Antes de incluir un frame en el prompt, se verifica:

- `timestamp_actual > timestamp_anterior` (frames genuinamente distintos en el tiempo)
- `delta_t < FRAME_HISTORY_MAX_STALENESS_MS` (el frame anterior no es demasiado antiguo — un frame de hace 3 s no aporta contexto temporal útil a 5 Hz)

Si la verificación falla, el prompt se adapta dinámicamente: en lugar de enviar dos frames etiquetados como `[t-1]` y `[t]`, envía un único frame etiquetado como `[ciclo actual]` con una nota explícita: "historial visual no disponible en este ciclo". Esto es más conservador pero menos peligroso que fabricar una secuencia temporal falsa.

9.4 Falla 3 — Descalibración de escala en el canal de divergencia

9.4.1 Descripción del modo de falla

El canal de divergencia del `ObstacleField` (cap. 6, §6.7) estima $\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y}$ usando `np.gradient()` sobre el campo de flujo. En versiones anteriores, el cálculo usaba gradientes de Sobel 3×3 sin el factor de normalización $1/8$ que el kernel estándar requiere para ser un estimador de derivada de primer orden en unidades de píxel-por-píxel:

$$K_{\{\text{Sobel}\}} = \frac{1}{8} \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$$

Sin la normalización, el estimador de divergencia producía valores aproximadamente 8 veces mayores que los correctos. Con el factor de ocupación original calibrado sobre esa escala inflada, el canal de ocupación saturaba a `1.0` en condiciones de textura moderada — es decir, prácticamente siempre que hubiera cualquier gradiente de flujo en la imagen, incluso en vuelo en espacio abierto sin obstáculos cercanos.

La consecuencia operativa era que el predicado disyuntivo `is_blocked()` (§6.9) estaba dominado completamente por el canal de ocupación: aunque el TTC indicara una zona segura (> 4 s), la ocupación saturada votaba bloqueo. El canal de TTC calibrado quedaba efectivamente anulado.

9.4.2 Interacción con la lógica disyuntiva

Este modo de falla ilustra un problema de diseño de predicados disyuntivos en sistemas con múltiples canales de diferente confianza. La lógica $\text{is_blocked} = (\text{occ} \geq 0.35) \vee (\text{conf} \geq 0.35 \wedge \text{ttc} \leq 2.5)$ asume que ambos canales están calibrados en la misma escala de confianza. Si un canal está sistemáticamente sobre-estimado, la disyunción lo convierte en el canal dominante, anulando la información del otro. El TTC, que tiene validación empírica (cap. 7), perdía toda influencia frente a una ocupación descalibrada.

9.4.3 Mecanismo de contención

La corrección fue triple: 1. Migrar a `np.gradient()` (diferencias centradas de segundo orden, normalizadas intrínsecamente) para el cómputo de divergencia. 2. Recalibrar el factor de ocupación contra datos de profundidad de referencia, con el proceso de curva ROC descrito en el cap. 7 (§7.5 — estado pendiente de completar para el canal de ocupación). 3. Introducir los umbrales diferenciados de confianza para los dos canales (`MIN_CONFIDENCE_FOR_BLOCKED = 0.15` vs. `MIN_CONFIDENCE_FOR_TTC_BLOCKED = 0.35`), de modo que el canal de TTC requiera mayor confianza para votar solo que el canal de ocupación acompañado de confianza mínima.

La tercera medida es especialmente importante: aunque la normalización del kernel corrige la escala, el sistema necesita ser robusto a futuras recalibraciones. Los umbrales diferenciados hacen que ningún canal pueda dominar completamente al otro sin confianza suficiente.

9.5 Falla 4 — State clipping y ciclos límite en LangGraph

Esta instancia fue la más costosa en consecuencias observadas (un ascenso acumulado de ~356 m en una sola corrida de validación) y la más ilustrativa de los riesgos de orquestadores de grafos que descartan silenciosamente estado no declarado.

9.5.1 El mecanismo de descarte silencioso de LangGraph

LangGraph construye los canales de estado de un `StateGraph` a partir de un `TypedDict` declarado en tiempo de compilación (`build_workflow()`). En cada invocación del grafo (`graph.invoke(state)`), el runtime serializa el estado de salida de cada nodo y lo fusiona con el estado entrante usando únicamente las claves presentes en el `TypedDict`. Cualquier clave que un nodo escriba en su diccionario de retorno pero que no esté declarada en el `TypedDict` es **descartada silenciosamente** — sin excepción, sin advertencia, sin registro en el log. El nodo siguiente recibe un estado donde esa clave simplemente no existe o tiene el valor por defecto del último ciclo donde sí estaba declarada.

Esta propiedad del runtime es documentada en la implementación (§5.2 del cap. 5) como la causa de cuatro bugs históricos. Su mecanismo específico es importante: el `TypedDict` de Python es una anotación de tipos, no una estructura de datos con validación en tiempo de ejecución. `state["clave_nueva"] = valor` no falla aunque `clave_nueva` no esté en el `TypedDict`; simplemente el runtime de LangGraph no la persiste en el canal de estado entre nodos.

9.5.2 Las cuatro vulnerabilidades identificadas

Vulnerabilidad A — Claves de control no declaradas.

Tres claves de control cruzaban la frontera entre nodo y lazo sin estar declaradas en `DroneState`:

- `_reset_stall_counter`: debía reiniciar el contador de progreso estancado al llegar a un waypoint. No declarada → el contador crecía monótonamente desde el inicio de la misión, independientemente del

progreso real.

- `_delib_memory`: debía acumular un historial corto de resultados del deliberativo para que el VLM pudiera referirse a decisiones previas. No declarada → el historial siempre estaba vacío; cada ciclo el VLM era consultado sin contexto de lo que había decidido un ciclo antes.
- `_corner_wp`: debía inyectar un sub-waypoint de esquina intermedia para la maniobra `GIRAR_90`. No declarada → el nodo de girar_90 lo escribía en su retorno, el router lo leía en el ciclo siguiente... como el valor inicial (None), nunca como el punto calculado.

El diagnóstico de estas tres claves requirió instrumentar el runtime con un interceptor de estado entre cada par de nodos — el equivalente de "printear el estado completo antes y después de cada nodo" — porque ningún log de nivel de aplicación mostraba el descarte.

Vulnerabilidad B — Ciclo límite de período 3 en la red de seguridad.

El mecanismo de reintento de maniobra de escape funcionaba así (versión con el bug):

```
si stall_count > hard_threshold:
    activar FRENAR (ciclo 1)
    si stall_count > hard_threshold + N:
        resetear stall_count = 0 ← la red de seguridad se resetea a sí misma
```

El reseteo de `stall_count` en la propia rama de escape convertía el estado "atasco severo → acción de escape" en un estado transitorio: al ciclo siguiente, `stall_count = 0 < hard_threshold` y el sistema volvía a modo crucero. Tres ciclos después, el atasco se re-detectaba y el ciclo se repetía. El período 3 (crucero → pre-stall → escape → reseteo → crucero) no era observable como un error — era un patrón de comportamiento que en el viewer se veía como "el dron frenaba ocasionalmente sin razón aparente".

La corrección convierte el estado de escape en **absorbente**: una vez que `stall_count > hard_threshold`, el sistema transiciona a `DEADLOCK_ESCAPE` y no retorna a crucero hasta completar la secuencia de escape (deep_scan o girar_90), independientemente de cómo evolucione el contador.

Vulnerabilidad C — Métrica de progreso auto-inconsistente.

El predicado de atasco evaluaba `ciclos_sin_progreso > effective_stall_threshold` donde `effective_stall_threshold = max(10, eps_m / (min_speed / loop_hz))`. El problema era que `eps_m` (distancia mínima de avance por ciclo, en metros) y `min_speed` (velocidad mínima de crucero) estaban definidos en unidades y archivos distintos, y su combinación imponía una exigencia de velocidad de aproximación que el guiado en curva nunca podía satisfacer:

- `eps_m = 0.5 m` (distancia mínima de avance por ciclo)
- `min_speed = 2.0 m/s`, `loop_hz = 5 Hz` → velocidad de avance mínima implícita = $0.5 \times 5 = 2.5 \text{ m/s}$
- En un giro de 90°, la velocidad de aproximación al waypoint oscila entre 0 y $v_x \cdot \cos(\theta)$, con media aproximadamente $v_x/2 \approx 1.25 \text{ m/s}$

Resultado: `effective_stall_threshold = max(10, 0.5/(2.0/5)) = max(10, 1.25) = 10` ciclos, pero el sistema detectaba atasco en ciclos 3–5 por el perfil de velocidad del giro. El mecanismo de atasco se disparaba prácticamente al inicio de cada waypoint que requiriera maniobra.

La corrección fue medir el progreso solo en el plano horizontal XY (distancia 2D al waypoint), que es monótonicamente decreciente durante el giro aunque la componente frontal oscile, y ajustar `eps_m` al valor que es físicamente alcanzable en el peor caso del perfil de giro.

Vulnerabilidad D — Escape ciego a la percepción.

En la versión con el bug, el `policy_router` evaluaba las condiciones de escape de atasco *antes* de leer el `ObstacleField`:

```
# versión con bug
if state["stall_count"] > hard_threshold:
    return "deadlock_escape" # ← sin verificar percepción
if field.is_blocked("centro"):
    return "evasive"
```

La consecuencia era que en ciclos donde el estimador de flujo tenía evidencia válida de que un sector lateral estaba despejado (`has_open_corridor() = True`), el router seleccionaba igualmente la rama de escape de atasco. El dron ascendía aunque la percepción indicara una salida horizontal disponible. La corrección fue consultar `has_open_corridor()` como condición de guarda antes de activar el escape: si hay evidencia de corredor, la maniobra de escape no se activa aunque el contador de atasco haya excedido su umbral.

9.5.3 El ascenso acumulado como síntoma compuesto

La combinación de las cuatro vulnerabilidades produjo en una corrida de validación un ascenso acumulado de ~356 m en una sola misión. La secuencia causal fue:

1. `_reset_stall_counter` no declarada → `stall_count` creció desde el ciclo 1 sin resetearse al llegar a cada waypoint.
2. A los ~8 ciclos de misión, `stall_count > hard_threshold` a pesar de que el dron avanzaba normalmente.
3. El router activó `deadlock_escape` (ciego a percepción), que en esa versión ejecutaba `GANAR_ALTURA` con deriva lateral no justificada.
4. `stall_count` se reseteó (ciclo límite B) → el siguiente ciclo, el router volvió a crucero normal.
5. `stall_count` volvió a crecer rápidamente (umbral demasiado bajo por la métrica inconsistente C).
6. El ciclo se repitió centenares de veces, produciendo un ascenso neto acumulado.

Desde el exterior, la corrida registraba una misión "sin errores" (no hay excepciones en el log) que simplemente "no llegó a destino". Sin el análisis del viewer frame a frame y la lectura del JSONL de auditoría, el patrón era opaco.

9.6 Falla 5 — Degradaciones sensoriales silenciosas

9.6.1 Inversión de canales de color RGB/BGR

Durante varios meses de experimentación, el cliente de captura de AirSim (`AirSimClient.capture()`) entregaba el buffer de imagen en el orden de canales RGB nativo del motor Unreal, mientras que el pipeline asumía la convención BGR de OpenCV — la convención estándar de OpenCV en la que el canal azul ocupa el índice 0.

El efecto fotométrico es una inversión de los canales rojo y azul en cada píxel: fachadas de ladrillo rojo aparecen en tonos azulados, el cielo azul aparece amarillento, la vegetación adquiere colores no naturales. El VLM deliberativo — entrenado sobre datos fotométricamente correctos — deliberó durante esos meses sobre imágenes con espectro cromático sistemáticamente falso. En ningún momento emitió una queja sobre la calidad de las imágenes ni redujo su confianza en la decisión: el modelo de lenguaje no tiene acceso directo al historial de la distribución de datos sobre la que fue entrenado, y no puede detectar que la imagen recibida está fuera de distribución por inversión de canal.

El diagnóstico requirió guardar físicamente las imágenes enviadas al VLM al disco y abrirlas en un visor externo — el análogo de "mirar los datos crudos" que es el primer paso de depuración en visión por computadora pero que en un sistema integrado es fácil de omitir cuando el modelo "parece razonar correctamente".

La corrección fue añadir `cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)` en el punto de captura, con un test unitario que verifica el orden de canales comparando estadísticas de cada canal en una imagen de referencia conocida.

9.6.2 Contaminación por primitivas de depuración persistentes

El simulador AirSim/cosysairsim permite dibujar primitivas geométricas en el mundo virtual (`simPlotLineStrip`, `simPlotPoints`, `simPlotArrows`) para visualización durante el desarrollo: trayectorias planificadas, sectores del `ObstacleField`, vectores de flujo. Estas primitivas son *persistentes* en la sesión del simulador: no se borran al reiniciar el script de control, solo al llamar explícitamente `simFlushPersistentMarkers()`.

En varias sesiones de desarrollo, al relanzar el vuelo sin reiniciar AirSim, las primitivas de la sesión anterior permanecían en el mundo virtual. La cámara a bordo las capturaba como geometría física en la escena: franjas de colores brillantes cruzando el campo visual que el VLM interpretaba como cables, estructuras metálicas o señales de tráfico dependiendo del color y orientación.

La consecuencia fue particularmente difícil de diagnosticar porque el comportamiento variaba según el historial de la sesión: un vuelo iniciado después de reiniciar AirSim se comportaba normalmente, mientras que el mismo vuelo iniciado sin reiniciar producía evasiones espurias repetidas. Jansen et al. (2023) documentan comportamientos similares de estado persistente en CosysAirSim como fuente de irreproducibilidad entre corridas.

La corrección fue añadir `simFlushPersistentMarkers()` al inicio de cada sesión de vuelo, con un segundo vaciado de primitivas antes de iniciar la fase de captura de imágenes para el VLM.

9.6.3 Supresión de ángulos de actitud en la telemetría

El binding `cosysairsim` usado en este proyecto (fork de AirSim con soporte UE5 — Jansen et al., 2023) no implementaba el método `to_eularian_angles()` del binding oficial de Microsoft AirSim. La llamada al método retornaba silenciosamente $(0, 0, 0)$ en lugar de lanzar `AttributeError`. El resultado era que los ángulos de pitch y roll reportados al VLM en cada prompt eran siempre `0.0°`, independientemente de la inclinación física real del vehículo.

Durante maniobras de aceleración (pitch negativo $\sim -8^\circ$ a -12°), descenso agresivo (pitch positivo) o giros (roll lateral), el VLM recibía la información de que el dron estaba perfectamente nivelado. La estimación de la componente de velocidad de cierre perpendicular al plano focal — que depende del pitch — era incorrecta, y la interpretación de la escena visual (la posición aparente del horizonte en la imagen) estaba desalineada con la información verbal de actitud.

La corrección fue implementar la conversión directa desde el cuaternión de orientación, disponible en la telemetría de CosysAirSim, a ángulos de Euler (roll, pitch, yaw) usando las fórmulas estándar de Wahba (1965), sin depender del método del binding:

$$\begin{aligned} \phi &= \arctan2(2(q_w q_x + q_y q_z), 1 - 2(q_x^2 + q_y^2)) \\ \theta &= \arcsin(2(q_w q_y - q_z q_x)) \\ \psi &= \arctan2(2(q_w q_z + q_x q_y), 1 - 2(q_y^2 + q_z^2)) \end{aligned}$$

Un test unitario verifica, contra una tabla de cuaterniones conocidos (identidad, 90° en cada eje, 45° combinado), que la conversión produce los ángulos correctos con error $< 0.01^\circ$.

9.7 Por qué el sistema seguía funcionando: el problema de la observabilidad

Ninguno de los cinco modos de falla descritos se manifestó como un error visible en tiempo de ejecución. En todos los casos:

- El modelo de lenguaje seguía produciendo respuestas JSON válidas y plausibles.
- El lazo de control ejecutaba el ciclo sin excepciones.
- Las métricas agregadas de la corrida (distancia recorrida, waypoints completados, velocidad media) no distinguían estas corridas de corridas fallidas por otras causas.

Esta propiedad tiene una implicación metodológica directa: las técnicas de depuración habituales para sistemas de software — observar el output, monitorear las métricas, leer el log de errores — son ciegas a esta clase de error. La observación del comportamiento es precisamente la herramienta menos útil porque el sistema exhibe comportamiento *consistente con sus premisas internas*, aunque esas premisas estén corrompidas.

Las técnicas de diagnóstico que sí funcionaron en este proyecto fueron:

1. Auditoría de contratos en la frontera de cada componente. Para cada par (productor, consumidor) de datos de estado, verificar explícitamente: ¿qué valor devuelve el productor cuando no tiene información? ¿Cómo interpreta el consumidor ese valor? La pregunta "¿qué pasa cuando este campo es None/vacío/cero?" tiene que tener una respuesta documentada y verificable.

2. Guardar los datos crudos enviados al modelo. El viewer (`airsim-runs/*.viewer.html`) guarda el fotograma anotado y el estado completo de cada ciclo precisamente para hacer auditable lo que el modelo recibía. Sin esa capacidad de inspección retroactiva, la falla de inversión RGB/BGR habría sido indetectable.

3. Tests de invariantes de contrato, no de comportamiento. Un test que verifique "el dron evadió el obstáculo" es frágil y dependiente de la escena. Un test que verifique "si `has_evidence()` es False, ningún consumidor reporta el campo como 'despejado'" es un test de contrato que captura la Falla 1 sin necesitar una corrida de vuelo.

4. Inyección de valores límite en el estado de LangGraph. Para detectar las claves no declaradas (Falla 4-A), la técnica fue instrumentar el lazo con un interceptor que comparaba `state` al entrar a cada nodo con `state` al salir, reportando cualquier clave presente en la salida que no estuviera en el `TypedDict`. Esta verificación se convirtió en un modo de debug permanente activable via variable de entorno (`LANGGRAPH_DEBUG_STATE_CLIPPING=1`).

9.8 Riesgos residuales y trabajo pendiente

Los cinco modos de falla documentados están corregidos en la implementación actual. Pero la arquitectura contiene puntos donde pueden aparecer instancias análogas:

Riesgo A — Nuevas claves de DroneState. Cada vez que se añade una funcionalidad que requiere persistir estado entre ciclos, existe el riesgo de que la clave correspondiente no se declare en `DroneState`. La mitigación es el modo `LANGGRAPH_DEBUG_STATE_CLIPPING=1` y el proceso de revisión de `DroneState` en el CHANGELOG antes de cada nueva feature.

Riesgo B — Cambios de API del binding de AirSim. El binding `cosysairsim` es un fork activamente mantenido (Jansen et al., 2023). Actualizaciones del binding pueden modificar el comportamiento de métodos (como ocurrió con la corrección del cuaternión). Los tests unitarios de conversión de telemetría son la mitigación; deben ejecutarse tras cada actualización del binding.

Riesgo C — Distribución de imágenes fuera del dominio de entrenamiento del VLM. La corrección de inversión RGB/BGR es una transformación fija. Pero la brecha de dominio entre imágenes de AirSim/UE5 y el dominio de entrenamiento del VLM (predominantemente imágenes reales del mundo) es una fuente estructural de error que no se elimina con ninguna corrección puntual. La calibración de la frecuencia de respuestas subóptimas del VLM en los escenarios de esta tesis (cap. 11) es el mecanismo de caracterización de ese riesgo residual.

10. Metodología experimental

10.1 Diseño experimental

La evaluación compara tres brazos de decisión sobre el mismo `ObstacleField` (cap. 6), el mismo espacio de macro-acciones y el mismo traductor a comandos cinemáticos (`action_to_command`, cap. 8):

- `slm` — el modelo de lenguaje multimodal (VLM, típicamente Qwen2.5-VL-3B) como capa táctica y deliberativa (cap. 8). Durante la espera de respuesta del modelo, el sistema aplica un avance cauto a velocidad reducida (`DELIB_WAIT_CREEP_SPEED_MPS = 0.5 m/s`) en lugar de un frenado total, preservando la traslación necesaria para que el estimador de flujo óptico mantenga confianza; solo se comanda detención total incondicional ante un bloqueo frontal inminente y confirmado (`close_structural`). El modelo delibera de forma asíncrona bajo un watchdog de seguridad con respaldo determinista ante timeout o respuestas no conformes, recibiendo un historial temporal de fotogramas (t y $t-1$), notas cinemáticas (`pitch`, `roll`, velocidad) y el motivo explícito de la consulta.
- `fsm` — una máquina de estados finitos explícita (`CRUISE → AVOID_LEFT | AVOID_RIGHT | CLIMB | BRAKE → CRUISE`), con transiciones gobernadas por umbrales fijos sobre el mismo `ObstacleField`. Es la línea de base directa contra la que se evalúa la hipótesis de la tesis: alta predictibilidad y costo computacional mínimo, pero sin capacidad de razonamiento contextual ni interpretación semántica del entorno.
- `reactive` — navegación guiada al waypoint sin evasión de obstáculos: cota inferior de rendimiento para desacoplar qué porción del desempeño proviene del lazo táctico de evasión y cuánto del seguimiento cinemático de base.

Adicionalmente, el diseño incorpora como factor experimental cruzado la **estrategia de resolución de atascos** (`DEADLOCK_STRATEGY`, cap. 5), compartida por los brazos `slm` y `fsm`: - `blind`: maniobra reactiva de escape cinemático tradicional (ganancia de altura o rotaciones de desenganche en bucle abierto). - `deep_vlm`: barrido panorámico espacial multifotograma ejecutado dentro del lazo y consulta deliberativa al VLM para identificar visualmente un corredor despejado antes de forzar el escape ciego como último recurso.

Escenarios de prueba y jerarquía de Tiers

Superando las formulaciones exploratorias preliminares (como los borradores `manhattan_a` y `manhattan_b`, descartados por inconsistencias topográficas y de mapas en el simulador), el protocolo define una jerarquía formal en tres niveles crecientes de dificultad ambiental:

- **Tier 0 (Línea de base / Control):** Escenario `minisim_clear` sobre el mapa `crater.png` (MiniSim). Terreno despejado sin obstáculos para cuantificar el guiado puro, la cota inferior de latencia de ciclo y el consumo cinemático ideal.

- **Tier 1 (Entorno intermedio / Vegetación y morfología orgánica):** Escenarios sobre el mapa `townsim_calib.png` (TownSim), abarcando la suite de pruebas `T-CALIB-0` a `T-CALIB-5` y la misión de recorrido perimetral `townsim_ini`. Incluye específicamente el escenario `T-CALIB-2` (`townsim_calib_cruce_frontal.json`), diseñado para imponer un **bloqueo frontal masivo genuino** frente a una fachada continua, forzando la activación de las ramas deliberativas y de resolución de atascos.
- **Tier 2 (Entorno complejo / Cañones urbanos y corredores angostos):** Escenarios sobre `citysim_calib.png` (CitySim). El escenario base validado para el batch G4 es `citysim_clear.json` (perímetro de manzana con patrón *climb-first* a -70 m AGL); los escenarios con corredores angostos (`citymap_pilot.json` y `citymap_a.json`) corresponden a la fase siguiente del batch.

Como principio metodológico estricto, **se descarta el teletransporte cinemático** (`start_pose` inhabilitado): todas las misiones despegan desde el *PlayerStart* nativo del nivel en Unreal Engine y ejecutan un ascenso vertical controlado previo a la navegación horizontal. A partir de las velocidades medidas en vuelo real (~2 a 3 m/s) y la cadencia táctica (5 Hz), el presupuesto temporal por corrida se dimensiona entre **600 y 900 segundos** para permitir la resolución completa de tramos complejos sin truncamiento artificial.

El criterio de arranque para el batch completo exige la validación previa de misiones piloto exitosas (`success = True`, 0 colisiones) en cada nivel. Se ejecutan al menos **cinco semillas por celda factorial** ($\text{\text{Brazo}} \times \text{\text{Estrategia de Atasco}} \times \text{\text{Tier}}$) mediante el runner automatizado headless (`experiments/runner.py`).

10.2 Métricas reportadas

El sistema registra y evalúa las siguientes métricas cuantitativas:

- **Eficacia y seguridad de vuelo:**
 - Tasa de éxito de misión (`success_rate`, arribo a todos los waypoints dentro del presupuesto).
 - Tasa de colisiones totales por misión y normalizadas por kilómetro recorrido.
 - Distancia mínima a obstáculo, percentil 5 (`min_obstacle_dist_m`), medida mediante el canal de profundidad a cadencia reducida con guardia arquitectónica de no contaminación sobre el lazo de control (cap. 5).
 - SPL (*Success weighted by Path Length*): éxito ponderado por la razón entre la longitud de la trayectoria óptima y la efectivamente recorrida.
 - Tiempo y ciclos totales a destino.
- **Dinámica del lazo táctico y latencias:**
 - Latencia total por ciclo (\$p50\$ y \$p95\$), desagregada por brazo y por nodo activo del grafo de control (`reactive`, `evasive`, `deliberative`, `girar_90`, `fsm`, `deep_scan`). La latencia del escaneo espacial pre-vuelo (`spatial_scan`, \$5.0\$) es un costo de inicialización de única vez, registrado por separado del presupuesto por ciclo.

- `deliberation_rate` (fracción de ciclos tácticos que invocan activamente al modelo de lenguaje) e histograma integral de rutas por corrida.
- **Comportamiento del modelo de lenguaje (SLM/VLM):**
 - Número de invocaciones efectivas por misión (contabilizadas por transición de identificador único).
 - Tasa de *fallback* determinista ante respuestas inválidas o fuera de esquema.
 - Tasa de *timeout* del watchdog asíncrono.
 - Tasa de adherencia sintáctica (`adherence_rate`), con y sin decodificación gramatical estructurada (§8.2).
- **Resolución de atascos (Ablation H2/H3):**
 - Tasa de resolución de atascos mediante escaneo profundo (`deep_scan_resolution_rate`).
 - Promedio de ciclos consumidos para resolver el atasco (`deep_scan_avg_cycles_to_resolve`).
 - Tasa de caída a escape ciego de respaldo (`deep_scan_fallback_rate`).
- **Desglose espacial por Waypoint:**
 - Métricas segmentadas por tramo de misión (`<stem>.summary_by_wp.csv`), permitiendo aislar el comportamiento en tramos de ascenso inicial vertical de las fases de cruce y maniobra horizontal.

10.3 Protocolo estadístico

La comparación entre brazos y configuraciones se realiza mediante pruebas no paramétricas (prueba U de Mann-Whitney para comparaciones pareadas y Kruskal-Wallis para factores múltiples) calculadas sobre las distribuciones de las semillas de cada combinación, reportando además el tamaño de efecto (Cliff's Delta o correlación de rango biserial).

El análisis no se limita a valores agregados globales, sino que evalúa el rendimiento **desagregado por tipo de escenario y morfología de obstáculo**. Esta formulación permite discernir con precisión cuándo la capacidad de interpretación contextual del VLM proporciona ventajas medibles (por ejemplo, al anticipar salidas en pasajes bloqueados o seleccionar corredores abiertos entre vegetación) y en qué situaciones una heurística rígida (FSM) alcanza un rendimiento equivalente a una fracción mínima del costo computacional.

10.4 Reproducibilidad, auditoría y cuarentena de datos

El subsistema de registro (`src/logging/flight_logger.py`, `flight_video.py`, `flight_viewer.py`) genera por cada misión un directorio autónomo de auditoría (`airsim-runs/<mission_id>-<timestamp>/`) que comprende:

1. **Telemetría granular:** Archivo `JSONL` estructurado ciclo a ciclo y archivo `CSV` correspondiente para análisis tabular directo.

2. **Resúmenes agregados:** `summary.json` con los indicadores globales de la corrida y `summary_by_wp.csv` con el desglose por waypoint.
3. **Auditoría visual forense del VLM:** Almacenamiento individual en formato `PNG` (`photo-<timestamp_ISO>.png`) de cada fotograma exacto presentado al modelo, preservando el timestamp de captura real y evitando discrepancias de inferencia.
4. **Grabación de video continua sincronizada:** Archivo `.webm` (códec VP8 sin dependencias de licencias propietarias) grabado a la tasa nominal `L00P_HZ`, con superposición en pantalla de la telemetría cinemática, lecturas sectoriales de TTC y el nodo activo del grafo.
5. **Visor interactivo offline (`viewer.html`):** Aplicación web embebida en la carpeta de la corrida que vincula un deslizador temporal con el video y la tabla de telemetría, permitiendo inspeccionar sincronizadamente el prompt textual, los fotogramas y la respuesta del modelo en cada decisión.

Trazabilidad y cuarentena experimental: Cada corrida incorpora automáticamente el hash de commit de Git (`code_version`) en `summary.json`. Para garantizar la validez científica de los resultados, se establece una **política estricta de cuarentena**: ningún vuelo anterior al 2026-09-03 se incluye en el análisis estadístico formal del capítulo 11, dado que los registros previos estuvieron expuestos a artefactos instrumentales ya corregidos (inversión de canales de color R/B en la captura, persistencia de líneas de depuración en el visor 3D y supresión espuria de los ángulos de *pitch* y *roll*).

El conjunto de validación de TTC (cap. 7), junto con los archivos de telemetría y configuración del batch definitivo, serán archivados y publicados con un identificador digital persistente (DOI vía Zenodo), garantizando la completa auditabilidad y reproducibilidad de la tesis.

11. Resultados comparativos SLM vs. FSM

Estado (2026-09-07): corridas piloto de Tier 0, Tier 1 base y Tier 2 base completadas (1 semilla, ilustrativo). El batch estadístico G4 (≥5 semillas, análisis Mann-Whitney) está pendiente de `townsim_ini` y `citymap_a`. Las tablas de esta sección presentan datos reales de las corridas piloto en §11.1 y marcadores de posición para G4 en §11.2–11.4.

Corte de datos: todos los resultados de esta sección provienen de corridas con `code_version` posterior al 2026-09-03 — fecha de corrección de dos bugs que afectaban directamente la percepción del VLM (canales R/B invertidos y marcadores de debug en la captura). Las corridas anteriores a esa fecha no son comparables para el brazo `slm` y no se usan en el análisis.

11.1 Resultados piloto (1 semilla, ilustrativo — pre-batch G4)

Estas corridas cumplen el criterio de arranque del diseño experimental (`success = True`, 0 colisiones) y sirven de referencia cualitativa antes del análisis estadístico de G4. Estrategia de desbloqueo: `deep_vlm` en todos los casos (default de producción).

Tier 0 — `minisim_clear` · MiniSim (crater.png)

Escenario: 3 waypoints en L (~191m), altitud -10m, ambiente despejado. Fecha: 2026-09-07.

Brazo	Éxito	Ciclos	Dura- ción (s)	Distan- cia (m)	Colisio- nes	Invoc. SLM	Delibe- ración	Fall- back SLM	Dead- locks
<code>slm</code>	✓	1151	235	191.7	0	80	6.95%	1.25%	1
<code>fsm</code>	✓	1031	208	224.4	0	—	—	—	2
<code>reac- tive</code>	✓	414	84	183.6	0	—	—	—	0

Dist. mín. al obstáculo: `slm` 9.74m · `fsm` 6.38m · `reactive` 9.06m

Observación: En un ambiente despejado la diferencia entre brazos es de velocidad pura. `reactive` completa en 84s (sin deliberar); `slm` tarda 2.8× más debido a las 80 invocaciones al VLM, aunque el trayecto real es el más corto (191.7m vs 224.4m del `fsm`).

Tier 1 — townsim_clear · TownSim (townsim_calib.png)

Escenario: perímetro completo del complejo, 6 WPs, ~640m, altitud de tránsito -30m (sobre los edificios). Fecha: 2026-09-07.

Brazo	Éxito	Ciclos	Duración (s)	Distancia (m)	Colisiones	Invoc. SLM	Deliberación	Fall-back SLM	Deadlocks
s _{lm}	✓	1337	311	632.3	0	11	0.82%	0%	3
fsm	✓	1537	347	652.3	0	—	—	—	5
reac-tive	✓	1196	266	639.0	0	—	—	—	0

Dist. mín. al obstáculo: s_{lm} 0.004m · fsm 7.89m · reactive 9.51m

Observación: La tasa de deliberación del s_{lm} bajó de 15.4% (corridas pre-fix de agosto) a 0.82% — reducción de ×19, atribuible al avance cauteloso durante la espera del VLM (elimina el bucle mecánico de baja confianza) y al fix de colores R/B (reduce falsas alarmas de percepción). Con sólo 11 invocaciones sobre 1337 ciclos, el brazo s_{lm} es el más rápido de los tres en este escenario. El fsm acumula 5 deadlocks (todos resueltos por escaneo profundo); el reactive, 0. La distancia mínima al obstáculo del s_{lm} (0.004m) es un artefacto del spawn conocido (ciclo inicial, colisión estática con el suelo), no una aproximación peligrosa en vuelo.

Tier 2 — citysim_clear · CitySim (citysim_calib.png)

Escenario: perímetro de una manzana del grid regular, 7 WPs, ~430m, altitud de tránsito -70m (climb-first: WP_1→WP_2 sube vertical puro antes de mover en horizontal). Fecha: 2026-09-07.

Brazo	Éxito	Ciclos	Duración (s)	Distancia (m)	Colisiones	Invoc. SLM	Deliberación	Fall-back SLM	Deadlocks
s _{lm}	✓	628	174	427.2	0	5	0.80%	0%	0
fsm	✓	641	182	447.2	0	—	—	—	0
reac-tive	✓	541	159	431.2	0	—	—	—	0

Dist. mín. al obstáculo: s_{lm} 0.77m · fsm 11.0m · reactive 9.43m

Observación: Los tres brazos completan el perímetro sin colisiones ni deadlocks — el escenario base a -70m está por encima de la línea de tejados de CitySim. La arquitectura climb-first (near_vertical fix, 2026-09-07) evita el problema de la primera corrida (z=-50m, rampa diagonal que atravesaba edificios). Con 0 deadlocks en los tres brazos, `citysim_clear` confirma el mismo rol que `minisim_clear` y `townsim_clear`: escenario de control a altitud de tránsito libre, cota inferior para el análisis del brazo `slm` en Tier 2. La dist. mín. del `slm` (0.77m) es probable artefacto del segmento de climb cerca del spawn; no se registró ninguna evasión activa durante el vuelo horizontal.

11.2 Tabla de resultados por brazo y escenario

Datos de corridas piloto (seed=1, `deep_vlm`). La columna "DistMin" es el valor observado en la corrida única; el percentil p5 se calculará sobre ≥ 5 semillas en el batch G4. Las filas marcadas **[pendiente G4]** corresponden a condiciones no corridas aún.

Escenarios definitivos G4: `minisim_clear` (Tier 0), `townsim_ini` (Tier 1 con obstáculos reales) / `townsim_calib_cruce_frontal` (T-CALIB-2), `citysim_clear` (Tier 2 base) / `citymap_a` (Tier 2 con obstáculos — pendiente construcción). Las filas de `townsim_clear` corresponden al escenario base (sin obstáculos a altitud de tránsito), equivalente al rol de `minisim_clear` en Tier 0.

Brazo	Tier / Escenario	Estrategia Atasco	Éxito (1 sem.)	Col./km	DistMin (m)	Tiempo (s)	Deliberación	Fall-back SLM	Res. Atasco VLM
<code>slm</code>	Tier 0 (<code>minisim_clear</code>)	<code>deep_vlm</code>	1/1	0	9.74	235	6.95%	1.25%	100% (1/1)
<code>slm</code>	Tier 0 (<code>minisim_clear</code>)	<code>blind</code>	—	—	—	—	—	—	—
<code>slm</code>	Tier 1 (<code>townsim_clear</code>)	<code>deep_vlm</code>	1/1	0	0.004*	311	0.82%	0%	100% (3/3)
<code>slm</code>	Tier 1 (<code>townsim_clear</code>)	<code>blind</code>	—	—	—	—	—	—	—
<code>slm</code>	Tier 2 (<code>citysim_clear</code>)	<code>deep_vlm</code>	1/1	0	0.77+	174	0.80%	0%	— (0 deadlocks)

Brazo	Tier / Esce- nario	Estra- tegia Atasco	Éxito (1 sem.)	Col./km	DistMin (m)	Tiempo (s)	Delibe- ración	Fall- back SLM	Res. Atasco VLM
fsm	Tier 0 (minisi m_clea r)	deep_vl m	1/1	0	6.38	208	—	—	100% (2/2)
fsm	Tier 1 (townsi m_clea r)	deep_vl m	1/1	0	7.89	347	—	—	100% (5/5)
fsm	Tier 2 (citysi m_clea r)	deep_vl m	1/1	0	11.0	182	—	—	— (0 dead- locks)
reac- tive	Tier 0 (minisi m_clea r)	—	1/1	0	9.06	84	—	—	—
reac- tive	Tier 1 (townsi m_clea r)	—	1/1	0	9.51	266	—	—	—
reac- tive	Tier 2 (citysi m_clea r)	—	1/1	0	9.43	159	—	—	—

* DistMin Tier 1 `slm` = 0.004m: artefacto del ciclo de spawn (contacto estático con suelo), no aproximación en vuelo. † DistMin Tier 2 `slm` = 0.77m: probable artefacto del segmento climb-first cerca del spawn; sin evasión activa registrada en vuelo horizontal.

11.3 Observaciones preliminares y marco para el análisis estadístico G4

Nota: con 1 semilla por celda no es posible ejecutar pruebas de significancia estadística. Esta sección presenta las tendencias observadas en los datos piloto y el diseño del análisis que se realizará sobre el batch G4 (≥5 semillas por celda).

11.3.1 Tendencias en los datos piloto

Los tres escenarios ejecutados hasta la fecha son escenarios de **control a altitud de tránsito libre**: la ruta no cruza ningún obstáculo en la dirección de vuelo horizontal — los edificios quedan por debajo del plano de crucero (−30m en Tier 1, −70m en Tier 2). En esta configuración, los datos piloto muestran un patrón consistente en los tres tiers:

Escenario	Tiempo reac-tive	Tiempo slm	Ratio slm / reac-tive	Tiempo fsm	Ratio fsm / reac-tive
Tier 0 minisim_clear	84s	235s	2.8×	208s	2.5×
Tier 1 townsim_clear	266s	311s	1.17×	347s	1.30×
Tier 2 citysim_clear	159s	174s	1.09×	182s	1.14×

El ratio slm / reactive decrece con la longitud de la ruta: en Tier 0 (~191m), la latencia fija del VLM (~2.9s por invocación × 80 invocaciones) domina el tiempo total; en Tier 2 (~430m), con solo 5 invocaciones y avance cauteloso durante la espera, el overhead es marginal. Esta relación es consistente con la hipótesis de que el costo del brazo slm no es lineal en la distancia, sino principalmente en la frecuencia de deliberación.

Una segunda tendencia es la diferencia en distancia recorrida:

Escenario	Distancia slm	Distancia fsm	Δ
Tier 0	191.7m	224.4m	−32.7m (−15%)
Tier 1	632.3m	652.3m	−20.0m (−3%)
Tier 2	427.2m	447.2m	−20.0m (−5%)

En los tres tiers, el brazo slm recorre menos distancia que el fsm. Con 1 semilla, esto puede ser varianza del punto de spawn; con ≥ 5 semillas, si el patrón se sostiene, indicaría que las pocas invocaciones del VLM en escenarios de control producen correcciones de rumbo que evitan desviaciones que la FSM sí acumula.

Los deadlocks observados son escasos (0–5 por corrida) y todos resueltos al 100% por deep_vlm en slm y fsm. El brazo reactive no registra deadlocks en ningún tier, lo cual es esperado: su control reactivo puro no tiene el concepto de "objetivo pendiente" que genera atascos en los brazos deliberativos cuando la ruta directa está bloqueada.

11.3.2 Marco estadístico para el batch G4

Sobre ≥ 5 semillas independientes por celda factorial se ejecutará:

1. **Prueba U de Mann-Whitney** (bilateral) comparando cada par de brazos en el mismo escenario y estrategia de desbloqueo. Hipótesis nula: la distribución de la métrica principal no difiere entre brazos. Métrica primaria: tiempo de misión en corridas con `success=True`; métrica secundaria: `dist_min_m` como indicador de seguridad.
2. **Tamaño de efecto:** Cliff's Delta (δ) sobre el mismo par. Umbral de referencia: $\delta > 0.3$ como efecto "mediano" (interpretación de Romano et al., 2006). Un p-valor significativo con δ pequeño (< 0.1) indicaría una diferencia real pero de magnitud práctica despreciable.
3. **Corrección de Bonferroni** sobre las comparaciones múltiples por tier (3 pares de brazos $\times$ 2 estrategias $\times$ 3 escenarios = 18 pruebas); umbral ajustado $\alpha = 0.05/18 \approx 0.0028$.

Las comparaciones previstas son: `slm` vs. `fsm`, `slm` vs. `reactive`, y `fsm` vs. `reactive`, tanto en estrategia `deep_vlm` como `blind` donde aplique. La comparación `deep_vlm` vs. `blind` dentro del mismo brazo `slm` cuantifica el aporte de H3 (escaneo profundo deliberativo frente al escape ciego).

11.4 Análisis por tipo de escenario

La pregunta central de la tesis — ¿aporta la deliberación contextual del VLM sobre la heurística rígida de la FSM? — requiere desagregarla por tipo de escenario, porque la respuesta esperada es distinta según el nivel de dificultad.

11.4.1 Tier 0 — Control sin obstáculos (`minisim_clear`)

El escenario de referencia más simple: 3 waypoints en L, ambiente despejado, sin ningún obstáculo en la trayectoria directa. El dato piloto muestra el costo puro del brazo `slm` sin ningún beneficio compensatorio: 80 invocaciones al VLM sobre 1151 ciclos (6.95%) generan 235s de tiempo de misión frente a los 84s del brazo `reactive` (ratio 2.8 $\times$). El `fsm` (208s) ocupa una posición intermedia: también sufre del overhead de su lazo deliberativo en atascos (2 deadlocks), pero sin la latencia VLM por invocación.

La distancia mínima al obstáculo es comparable entre brazos (9.74m `slm`, 9.06m `reactive`, 6.38m `fsm`), confirmando que en un entorno despejado ningún brazo es más seguro que otro — la ventaja del VLM no tiene dónde manifestarse. Este resultado es el esperado por diseño: `minisim_clear` existe para establecer la cota inferior de rendimiento del sistema y cuantificar el overhead puro del brazo `slm` en ausencia de beneficio.

Predicción G4: la diferencia de velocidad entre `reactive` y `slm` debería sostenerse con alta significancia estadística (Cliff's δ cercano a 1.0 en favor de `reactive`); la comparación `slm` vs. `fsm` puede ser más variable porque depende de cuántos deadlocks acumule la FSM por corrida.

11.4.2 Tier 1 — Perímetro urbano con vegetación (`townsim_clear`)

El escenario `townsim_clear` para Tier 1: vuela el perímetro a -30m (sobre la línea de tejados), sin obstáculos en la trayectoria de crucero. A diferencia de Tier 0, la longitud de la ruta (~640m vs. ~191m) diluye el overhead por invocación, y el brazo `slm` resulta el más rápido de los tres en el piloto (311s vs. 347s `fsm` vs. 266s `reactive`). La tasa de deliberación de 0.82% (11 invocaciones / 1337 ciclos) representa una reducción de ×19 respecto a las corridas pre-fix de agosto (15.4%), directamente atribuible al avance cauteloso durante la espera del VLM y al fix de canales de color que reduce las falsas alarmas.

El dato más significativo de Tier 1 piloto no es la velocidad sino los deadlocks: el `fsm` acumula 5 (todos resueltos por escaneo profundo `deep_vlm`); el `slm`, 3; el `reactive`, 0. Con 1 semilla, este patrón es indicativo pero no concluyente — puede reflejar diferencias en la gestión de atascos entre brazos, o simplemente la varianza de una única semilla.

El escenario de interés real para Tier 1 es `townsim_ini`: un recorrido que cruza el interior del complejo, con corredores vegetados y fachadas que obstruyen la trayectoria directa. Es allí donde el escaneo deliberativo profundo (`deep_vlm`) tiene un caso de uso genuino frente al escape ciego (`blind`): la FSM y el `reactive` deben bordear obstáculos por heurística, mientras el `slm` puede consultar al VLM para elegir el corredor. Los datos de `townsim_clear` solo establecen la cota de partida.

11.4.3 Tier 2 — Entorno urbano denso (`citysim_clear`)

El escenario base `citysim_clear` funciona como el tercer escenario de control: los tres brazos completan el perímetro en ~159-182s, sin colisiones ni deadlocks, con tiempos comparables entre sí (ratio `slm` / `reactive` = 1.09×). A -70m, el dron vuela por encima de la línea de tejados de CitySim; la deliberación del VLM no tiene obstáculo real que analizar.

Tres observaciones son relevantes para el análisis posterior:

1. **`fsm` conservador, `reactive` agresivo:** el `fsm` registra la mayor distancia mínima al obstáculo (11.0m) y el mayor tiempo (182s); el `reactive` el menor (9.43m, 159s). El `slm` queda entre ambos (0.77m en climb inicial, 174s). En un entorno donde no hay obstáculos activos, la heurística conservadora de la FSM penaliza velocidad sin ganar seguridad.
2. **Altitud como variable crítica de diseño:** la primera corrida a z=-50m falló (success=False, colisión) porque la altitud insuficiente hacía que la ruta de climb desde el spawn atravesara edificios. El fix de `near_vertical` (2026-09-07) permite el patrón climb-first, pero la variable determinante fue la elección de -70m > altura de tejados.
3. **Cero deadlocks en los tres brazos:** confirma que la ruta de crucero no presenta obstrucciones reales a -70m. Cualquier deadlock que aparezca en `citymap_clear` — donde la ruta sí cruza corredores entre edificios — será atribuible a la geometría del escenario, no a artefactos de la altitud.

El escenario `citymap_clear` es el experimento inicial de Tier 2: un recorrido que atraviesa corredores angostos entre edificios altos, donde ni el `reactive` ni el `fsm` tienen información semántica para elegir entre dos calles de ancho similar. La hipótesis es que el brazo `slm`, al consultar al VLM con una imagen aérea del corredor, podrá elegir la ruta más despejada con mayor consistencia que una heurística basada en distancia pura o en flujo óptico. El resultado negativo también es válido: si el VLM no aporta información útil en un entorno de alta textura urbana uniforme, es un hallazgo de diseño relevante para el capítulo 09.

12. Conclusiones

Estado: pendiente del capítulo de resultados (cap. 11), que a su vez depende de la corrida experimental comparativa descrita en el capítulo 10.

12.1 Retomar el hilo conductor

Cerrar aquí el argumento declarado en la introducción (§1.4) y desarrollado con evidencia medida en el capítulo 9: en un sistema de control donde un modelo de lenguaje consume descripciones de escena, el error más caro está en la interfaz que lo alimenta, no en el razonamiento del modelo, y las instancias documentadas de ese patrón —desincronización de esquemas de percepción y campos nulos, desalineación en secuencias temporales, saturación por descalibración de escala diferencial, y descarte silencioso en grafos de estado— se detectaron auditando formalmente los contratos de datos y la propagación de estado, nunca observando únicamente el comportamiento superficial del vuelo.

12.2 Respuesta a la pregunta de investigación

¿Aporta el SLM sobre la FSM? Síntesis de §11.4, con los matices por tipo de escenario que surjan de la corrida experimental — evitar una respuesta única y agregada si los datos sostienen una respuesta más matizada por escenario.

12.3 Limitaciones

- Validación exclusivamente en simulación (AirSim); sin validación en hardware físico (Jetson Nano + dron real), que el plan de trabajo aprobado (`plan_tesis/plan-tesis.md`, §"Transferencia de los resultados obtenidos") sitúa como siguiente etapa fuera del alcance de este trabajo.
- Calibración del canal de ocupación de `ObstacleField` pendiente al cierre de este escrito (cap. 6, §6.3), con el canal de TTC como el único de los dos formalmente validado contra profundidad (cap. 7).
- Tamaño de muestra y generalización: a determinar según el número final de semillas y escenarios de la corrida de tesis (cap. 10).

12.4 Trabajo futuro

- Validación en hardware real: companion computer de bajo costo (Jetson Nano) conectada a un dron físico con cámara monocular a bordo, siguiendo el pipeline descrito en el plan de trabajo aprobado.
- Calibración del canal de ocupación contra profundidad, con el mismo protocolo de curva ROC aplicado al TTC (cap. 7), y validación de la derotación con giros de guiñada más agresivos (cap. 7, §7.4).

- Navegación en enjambre, sensores adicionales, y las demás líneas de extensión discutidas en el plan de trabajo aprobado (`plan_tesis/plan-tesis.md`, §"Transferencia de los resultados obtenidos").

13. Referencias

Bibliografía compilada a partir de dos fuentes del proyecto: la citada en el plan de trabajo aprobado(`plan_tesis/plan-tesis.md`) y la disponible como archivos (PDF/BibTeX) en `plan_tesis/bibliografia/` y en `informe/bibliografia/`. Se organiza en cuatro secciones según su origen, para que la revisión y depuración posterior (eliminar entradas no citadas en el texto final, resolver duplicados menores de formato) sea trazable. El formato sigue el usado en el plan de trabajo aprobado (APA, con DOI o URL cuando está disponible).

13.1 Bibliografía citada en el plan de trabajo aprobado

Alarm as estate agent's drone crashes into house - Property Industry Eye. (s. f.). Recuperado 29 de junio de 2025, de https://propertyindustryeye.com/alarm-as-estate-agents-drone-crashes-into-house/?utm_source=chatgpt.com

Aldao, E., González-de Santos, L. M., & González-Jorge, H. (2022). LiDAR Based Detect and Avoid System for UAV Navigation in UAM Corridors. *Drones*, 6(8). <https://doi.org/10.3390/drones6080185>

Aliane, N. (2024). A Survey of Open-Source UAV Autopilots. *Electronics*, 13(23). <https://doi.org/10.3390/electronics13234785>

Arnold, R., Yamaguchi, H., & Tanaka, T. (2018). Search and rescue with autonomous flying robots through behavior-based cooperative intelligence. *Journal of International Humanitarian Action*, 3. <https://doi.org/10.1186/s41018-018-0045-4>

Broggi, A., Bertozzi, M., Fascioli, A., Guarino, C., Guarino Lo Bianco, C., & Piazzzi, A. (2000). The Argo Autonomous Vehicle's Vision And Control Systems. *Int. J. Intelligent. Control. Syst.*, 3.

Cangahuala, L. A., Campagnola, S., Bradley, B. K., Boone, D. R., Buffington, B. B., Ludwinski, J. M., Nandi, S., & Scott, C. J. (2025). Europa Clipper Mission Design, Mission Plan, and Navigation. *Space Science Reviews*, 221(1), 22. <https://doi.org/10.1007/s11214-025-01140-2>

Castelli, T., Sharghi, A., Harper, D., Trémeau, A., & Shah, M. (2016). Autonomous navigation for low-altitude UAVs in urban areas. *CoRR*, abs/1602.08141. <http://arxiv.org/abs/1602.08141>

Castro, W., Marcato Junior, J., Polidoro, C., Osco, L. P., Gonçalves, W., Rodrigues, L., Santos, M., Jank, L., Barrios, S., Valle, C., Simeão, R., Carromeu, C., Silveira, E., Jorge, L. A. de C., & Matsubara, E. (2020). Deep Learning Applied to Phenotyping of Biomass in Forages with UAV-Based RGB Imagery. *Sensors*, 20(17). <https://doi.org/10.3390/s20174802>

Chahine, M., Hasani, R., Kao, P., Ray, A., Shubert, R., Lechner, M., Amini, A., & Rus, D. (2023). Robust flight navigation out of distribution with liquid neural networks. *Science Robotics*, 8(77). <https://doi.org/10.1126/scirobotics.adc8892>

- Chen, F., Wu, Q., Tao, G., & Jiang, B. (2014). A Reconfiguration Control Scheme for a Quadrotor Helicopter via Combined Multiple Models. *International Journal of Advanced Robotic Systems*, 11(8), 122. <https://doi.org/10.5772/58833>
- Chen, W., Shang, G., Hu, K., Zhou, C., Wang, X., Fang, G., & Ji, A. (2022). A Monocular-Visual SLAM System with Semantic and Optical-Flow Fusion for Indoor Dynamic Environments. *Micromachines*, 13(11). <https://doi.org/10.3390/mi13112006>
- Collier, J., Trentini, M., Giesbrecht, J., Mcmanus, C., Furgale, P., Stenning, B., Barfoot, T., Se, S., Kotamraju, V., Jasiobedzki, P., Shang, L., Chan, B., Harmat, A., & Sharf, I. (2012, enero). Autonomous Navigation and Mapping in GPS-Denied Environments at Defence R&D Canada. *NATO Symposium SET 168: Navigation Sensor and Systems in GNSS Denied Environments*.
- Da Silva, A., Fernández, S., Vidal, B., Sodre, H., Moraes, P., Peters, C., Barcelona, S., Sandin, V., Moraes, W., Mazondo, A., Macedo, B., Assunção, N., de Vargas, B., Kelbouscas, A., & Grando, R. (2024). *Implementación de Navegación en Plataforma Robótica Móvil Basada en ROS y Gazebo*. <http://arxiv.org/abs/2410.19972>
- Dickmanns, E. D. (2024). Evolution of the "4-D Approach" to Dynamic Vision for Vehicles. *Electronics*, 13(20), 4133.
- Drone Silo Inspections - Case Study - Horus Drones*. (s. f.). Recuperado 29 de junio de 2025, de https://horusdrones.com/home/drone-insights/case-studies/drone-silo-inspection/?utm_source=chatgpt.com
- Giacomossi Jr, L., Maximo, M., Sundelius, N., Funk, P., Brancalion, J. F. B., & Sohlberg, R. (2024). *Cooperative Search and Rescue with Drone Swarm* (pp. 381-393). https://doi.org/10.1007/978-3-031-39619-9_28
- Goel, A., Tung, C., Hu, X., Thiruvathukal, G. K., Davis, J. C., & Lu, Y.-H. (2021). Efficient Computer Vision on Edge Devices with Pipeline-Parallel Hierarchical Neural Networks. *CoRR*, abs/2109.13356. <https://arxiv.org/abs/2109.13356>
- Graf, M., Speidel, O., Ruof, J., & Dietmayer, K. (2021). *On-Road Motion Planning for Automated Vehicles at Ulm University*. <http://arxiv.org/abs/2012.04028>
- Hoang, V. D., Nyboe, F. F., Malle, N. H., & Ebeid, E. (2024). *Autonomous Overhead Powerline Recharging for Uninterrupted Drone Operations*. <https://arxiv.org/abs/2403.06533>
- Hu, J., Ren, Z., & Cheng, W. (2024). *Finite State Machines-Based Path-Following Collaborative Computing Strategy for Emergency UAV Swarms*. <https://arxiv.org/abs/2407.11531>
- Hughes, M., & Engelbrecht, J. (2023). Autonomous navigation for multiple unmanned aerial vehicles (UAVs) in urban environments. *MATEC Web of Conferences*, 388. <https://doi.org/10.1051/matecconf/202338804011>

- Hwang, J.-J., Xu, R., Lin, H., Hung, W.-C., Ji, J., Choi, K., Huang, D., He, T., Covington, P., Sapp, B., Zhou, Y., Guo, J., Anguelov, D., & Tan, M. (2024). *EMMA: End-to-End Multimodal Model for Autonomous Driving*. <https://arxiv.org/abs/2410.23262>
- Infocampo. (s. f.). *Renova amplió su planta de procesamiento y construirá una nueva terminal en Timbués*. INFOCAMPO Noticias del campo en el momento justo. Recuperado 25 de junio de 2025, de <https://www.infocampo.com.ar/renova-amplio-su-planta-de-procesamiento-y-construira-una-nueva-terminal-en-timbues/>
- Kemeny, A., & Panerai, F. (2003). Evaluating perception in driving simulation experiments. *Trends in Cognitive Sciences*, 7(1), 31-37. [https://doi.org/10.1016/S1364-6613\(02\)00011-6](https://doi.org/10.1016/S1364-6613(02)00011-6)
- Koubaa, A., Allouch, A., Alajlan, M., Javed, Y., Belghith, A., & Khalgui, M. (2019). Micro Air Vehicle Link (MAVLink) in a Nutshell: A Survey. *CoRR*, abs/1906.10641. <http://arxiv.org/abs/1906.10641>
- Langåker, H.-A., Kjerkreit, H., Syversen, C. L., Moore, R. J. D., Holhjem, Ø. H., Jensen, I., Morrison, A., Transeth, A. A., Kvien, O., Berg, G., Olsen, T. A., Hatlestad, A., Negård, T., Broch, R., & Johnsen, J. E. (2021). An autonomous drone-based system for inspection of electrical substations. *International Journal of Advanced Robotic Systems*, 18(2), 17298814211002972. <https://doi.org/10.1177/17298814211002973>
- Li, Q., Yu, W., & Jiang, S. (2022). *Optimized Views Photogrammetry: Precision Analysis and A Large-scale Case Study in Qingdao*. <https://arxiv.org/abs/2206.12216>
- Madridano Carrasco, Á. (2020). *Arquitectura de software para navegación autónoma y coordinada de enjambres de drones en labores de lucha contra incendios forestales y urbanos* [Tesis doctoral, Universidad Carlos III de Madrid]. <https://hdl.handle.net/10016/32463>
- Marazzato, R., & Sparavigna, A. C. (2015). Retinex filtering of foggy images: generation of a bulk set with selection and ranking. *CoRR*, abs/1509.08715. <http://arxiv.org/abs/1509.08715>
- Martin, P. G., Scott, T. B., Payton, O. D., & Fardoulis, J. S. (2017). *High-Resolution Aerial Radiation Mapping for Nuclear Decontamination and Decommissioning-17371*.
- Matej, J. (2016). *Simulation of vehicle in Unreal Game Engine using equations of motion, Runge-Kutta and line tracing methods to solve motion and scan Unreal Engine terrain under wheel*. Publishing House at the Technical University in Zvolen. <https://doi.org/10.5281/zenodo.55806>
- Miranda, V., Rezende, A., Rocha, T., Azpúrua, H., Pimenta, L., & Freitas, G. (2021). Autonomous Navigation System for a Delivery Drone. *Journal of Control, Automation and Electrical Systems*, 33. <https://doi.org/10.1007/s40313-021-00828-4>
- Mur-Artal, R., Montiel, J. M. M., & Tardós, J. D. (2015). ORB-SLAM: a Versatile and Accurate Monocular SLAM System. *CoRR*, abs/1502.00956. <http://arxiv.org/abs/1502.00956>
- Nguyen, D. D., Rohacs, J., & Rohacs, D. (2021). Autonomous Flight Trajectory Control System for Drones in Smart City Traffic Management. *ISPRS International Journal of Geo-Information*, 10(5). <https://doi.org/10.3390/ijgi10050338>

- Nothdurft, T., Hecker, P., Ohl, S., Saust, F., Maurer, M., Reschka, A., & Rüdiger Böhmer, J. (2011). *Stadtplot: First Fully Autonomous Test Drives in Urban Traffic*. https://www.tu-braunschweig.de/fileadmin/Redaktionsgruppen/Institute_Fakultaet_5/IFR/Dateien_EFS/Publikationen/Stadtplot_FirstFullyAutonomous.pdf
- NVIDIA. (2025). *Jetson Nano Brings the Power of Modern AI to Edge Devices*. <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-nano/product-development/>
- Open Source Autopilot for Drones - PX4 Autopilot*. (s. f.). Recuperado 29 de junio de 2025, de <https://px4.io/>
- Pestana Puerta, J. (2017). *Vision-Based Autonomous Navigation of Multirotor Micro Aerial Vehicles* [PhD Thesis, Universidad Politécnica de Madrid]. <https://doi.org/10.20868/UPM.thesis.47726>
- Pomerleau, D. A. (1988). ALVINN: An Autonomous Land Vehicle in a Neural Network. En D. Touretzky (Ed.), *Advances in Neural Information Processing Systems* (Vol. 1). Morgan-Kaufmann. https://proceedings.neurips.cc/paper_files/paper/1988/file/812b4ba287f5ee0bc9d43bbf5bbe87fb-Paper.pdf
- Redmon, J., Divvala, S. K., Girshick, R. B., & Farhadi, A. (2015). You Only Look Once: Unified, Real-Time Object Detection. *CoRR*, abs/1506.02640. <http://arxiv.org/abs/1506.02640>
- Resolución ANAC - Reglamento de Vehículos Aéreos No tripulados, Pub. L. No. 880/2019 (2019). <https://www.argentina.gob.ar/normativa/nacional/resoluci%C3%B3n-880-2019-333259>
- Samy, M., Amer, K., Shaker, M., & ElHelw, M. (2019). *Drone Path-Following in GPS-Denied Environments using Convolutional Networks*. <https://arxiv.org/abs/1905.01658>
- Shah, S., Dey, D., Lovett, C., & Kapoor, A. (2017). *AirSim: High-Fidelity Visual and Physical Simulation for Autonomous Vehicles*. <http://arxiv.org/abs/1705.05065>
- Shin, Y.-S., & Kim, A. (2019). Sparse Depth Enhanced Direct Thermal-infrared SLAM Beyond the Visible Spectrum. *IEEE Robotics and Automation Letters*, 4(3), 2918-2925. <https://arxiv.org/abs/1902.10892>
- Song, C., Guo, X., & Sui, J. (2025). Improved Model Predictive Control Algorithm for the Path Tracking Control of Ship Autonomous Berthing. *Journal of Marine Science and Engineering*, 13(7). <https://doi.org/10.3390/jmse13071273>
- Thorpe, C., Hebert, M. H., Kanade, T., & Shafer, S. A. (1988). Vision and navigation for the Carnegie-Mellon Navlab. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 10(3), 362-373. <https://doi.org/10.1109/34.3900>
- Turco, L., Zhao, J., Xu, Y., & Tsourdos, A. (2024). A Study on Co-simulation Digital Twin with MATLAB and AirSim for Future Advanced Air Mobility. *2024 IEEE Aerospace Conference*, 1-18. <https://doi.org/10.1109/AERO58975.2024.10521333>

UMILES GROUP. (2024, diciembre 20). *LiDAR y Fotogrametría: Las armas secretas de los drones en la topografía moderna*. <https://umilesgroup.com/drones-lidar-y-fotogrametria-cambiando-la-topografia-moderna/>

Vipparla, C., Krock, T., Nouduri, K., Fraser, J., AliAkbarpour, H., Sagan, V., Cheng, J.-R. C., & Kannappan, P. (2024). Fusion of Visible and Infrared Aerial Images from Uncalibrated Sensors Using Wavelet Decomposition and Deep Learning. *Sensors*, 24(24). <https://doi.org/10.3390/s24248217>

Wahba, G. (1965). A Least Squares Estimate of Satellite Attitude. *SIAM Review*, 7(3), 409. <https://doi.org/10.1137/1007077>

Shi, Y., Li, X., & Zhang, J. (2024). Real-time obstacle avoidance for UAVs using lightweight deep learning. *IEEE Transactions on Industrial Electronics*, 71(8), 9123-9132.

Vera-Yanez, R., Maza, I., & Ollero, A. (2024). Optical-Flow-Based Air-to-Air Detection of Airborne Obstacles for Autonomous Drones. *Drones*, 8(12), 705. <https://doi.org/10.3390/drones8120705>

Wang, H., Zhang, X., & Liu, C. (2023). Urban infrastructure inspection using autonomous UAVs: A review. *Automation in Construction*, 154, 105020.

World Health Organization. (2023). *Global status report on road safety 2023*. WHO Guidelines Approved by the Guidelines Review Committee. World Health Organization.

Yang, Y., & Wei, P. (2024). Autonomous eVTOL trajectory optimization for urban air mobility. *Transportation Research Part C: Emerging Technologies*, 158, 104423.

Zhang, T., & Wu, Q. (2023). Vision-based power line inspection with UAVs: Methods and challenges. *IEEE Geoscience and Remote Sensing Magazine*, 11(2), 78-95.

Zhao, K., Liu, Z., & Chen, X. (2024). Deep reinforcement learning for UAV autonomous navigation in GPS-denied environments. *IEEE Transactions on Intelligent Transportation Systems*, 25(3), 2890-2902.

Zhou, Y., Bautista, J., Yao, W., & de Marina, H. G. (2025). *Inverse Kinematics on Guiding Vector Fields for Robot Path Following*. <https://arxiv.org/abs/2502.17313>

Zhu, Y., Moniz, J. R. A., Bhargava, S., Lu, J., Piraviperumal, D., Li, S., Zhang, Y., Yu, H., & Tseng, B.-H. (2024). *Can Large Language Models Understand Context?*. <https://arxiv.org/abs/2402.00858>

13.1-bis Bibliografía incorporada en la revisión de septiembre de 2026

Las siguientes entradas fueron añadidas durante la revisión del capítulo 2 para actualizar el estado del arte al momento de la defensa. Se citan en el texto principal y se incluyen aquí con su referencia APA completa.

ANAC. (2025). *Resolución 319/2025 — Parte 100 del Reglamento Aeronáutico Civil Argentino (RAAC): aeronaves pilotadas a distancia*. Administración Nacional de Aviación Civil. <https://www.aviacionline.com/parte-100-anac-argentina-actualiza-su-normativa-para-drones>

ANAC. (2025). *Resolución 550/2025 — Desregulación de aeronaves no tripuladas de menos de 250 g y simplificación para categorías superiores*. Administración Nacional de Aviación Civil.

Argentina.gob.ar. (s.f.). *Ambiente incorpora drones en parques nacionales para la detección temprana de incendios forestales*. Ministerio de Ambiente y Desarrollo Sostenible. <https://www.argentina.gob.ar/noticias/ambiente-incorpora-drones-en-parques-nacionales-para-la-deteccion-temprana-de-incendios>

iProfesional. (2025). *La Ciudad abre la puerta al delivery con drones y crea un nuevo negocio para las empresas*. <https://www.iprofesional.com/negocios/463009-la-ciudad-abre-la-puerta-al-delivery-con-drones-y-crea-un-nuevo-negocio-para-las-empresas>

Barišić Kulas, A., Petric, F., & Bogdan, S. (2025). *Aerial maritime vessel detection and identification*. ICUAS 2025. arXiv:2507.07153. <https://arxiv.org/abs/2507.07153>

Breaking Defense. (2025, marzo). *Trained on classified battlefield data, AI multiplies effectiveness of Ukraine's drones: Report*. <https://breakingdefense.com/2025/03/trained-on-classified-battlefield-data-ai-multiplies-effectiveness-of-ukraines-drones-report/>

Center for Strategic and International Studies (CSIS). (2024). *Ukraine's future vision and current capabilities for waging AI-enabled autonomous warfare*. <https://www.csis.org/analysis/ukraines-future-vision-and-current-capabilities-waging-ai-enabled-autonomous-warfare>

Coptrz. (2026). *How will drones disrupt urban last-mile delivery in 2026?* <https://coptrz.com/blog/how-will-drones-disrupt-urban-last-mile-delivery-in-2026/>

Danish, S., Piran, M. J., Khan, S. U., Khan, M. A., Dang, L. M., Zweiri, Y., Song, H.-K., & Moon, H. (2025). *Vision-based fire management system using autonomous unmanned aerial vehicles: a comprehensive survey*. *Artificial Intelligence Review*. <https://doi.org/10.1007/s10462-025-11415-3>

Dataintel. (2025). *Civil drone market research report 2034*. <https://dataintel.com/report/global-civil-drone-market>

El Estratégico. (2026, julio 22). *Proponen crear un sistema nacional de vigilancia marítima con drones*. <https://www.elestrategico.com/2026/07/22/proponen-crear-un-sistema-nacional-de-vigilancia-maritima-con-drones>

Infobae. (2026, mayo 22). *Argentina acordó con Estados Unidos la incorporación de drones y tecnología militar para vigilar el Mar Argentino*. <https://www.infobae.com/politica/2026/05/22/argentina-acordo-con-estados-unidos-la-incorporacion-de-drones-y-tecnologia-militar-para-vigilar-el-mar-argentino/>

Liu, C., & Sziranyi, T. (2023). *Active wildfires detection and dynamic escape routes planning for humans through information fusion between drones and satellites*. arXiv:2312.03519. <https://arxiv.org/abs/2312.03519>

Modern War Institute at West Point. (s.f.). *Battlefield drones and the accelerating autonomous arms race in Ukraine*. <https://mwi.westpoint.edu/battlefield-drones-and-the-accelerating-autonomous-arms-race-in-ukraine/>

Norte Misionero. (2026). *Alerta temprana, drones y respuesta rápida: Misiones refuerza su esquema de prevención de incendios*. <https://nortemisionero.com.ar/provinciales/alerta-temprana-drones-y-respuesta-rapida-misiones-refuerza-su-esquema-de-prevencion-de-incendios-n516147/>

[Autores por confirmar]. (2026). A UAV–satellite hybrid pipeline for wildfire detection and dynamic perimeter prediction. *Drones*, 10(4), 263. <https://doi.org/10.3390/drones10040263>

Sun, X., Si, W., Ni, W., Li, Y., Wu, D., Xie, F., Guan, R., Xu, H.-Y., Ding, H., & Wu, Y. (2026). *AutoFly: Vision-Language-Action model for UAV autonomous navigation in the wild*. arXiv:2602.09657. <https://arxiv.org/abs/2602.09657>

Tian, Y., Lin, F., Li, Y., Zhang, T., Zhang, Q., Fu, X., Huang, J., Dai, X., Wang, Y., Tian, C., Li, B., Lv, Y., Kovács, L., & Wang, F.-Y. (2025). UAVs meet LLMs: Overviews and perspectives toward agentic low-altitude mobility. *Information Fusion*. arXiv:2501.02341. <https://arxiv.org/abs/2501.02341>

Vemprala, S., Bonatti, R., Bucker, A., & Kapoor, A. (2023). *ChatGPT for robotics: Design principles and model abilities*. arXiv:2306.17582. <https://arxiv.org/abs/2306.17582>

13.2 Bibliografía adicional en `plan_tesis/bibliografia/` (con archivo BibTeX, no citada arriba)

- AlMahamid, F., & Grolinger, K. (2022). Autonomous unmanned aerial vehicle navigation using reinforcement learning: A systematic review. *Engineering Applications of Artificial Intelligence*, 115, 105321.
- Baheri, A. (2020). *Safe Reinforcement Learning with Mixture Density Network: A Case Study in Autonomous Highway Driving*. <https://arxiv.org/abs/2007.01698>
- Bouhamed, O., Ghazzai, H., Besbes, H., & Massoud, Y. (2020). *Autonomous UAV Navigation: A DDPG-based Deep Reinforcement Learning Approach*. *CoRR*, abs/2003.10923. <https://arxiv.org/abs/2003.10923>
- Fabra, F., Zamora, W., Sangüesa, J., Calafate, C. T., Cano, J. C., & Manzoni, P. (2019). A Distributed Approach for Collision Avoidance between Multirotor UAVs Following Planned Missions. *Sensors*, 19(10), 2404. <https://doi.org/10.3390/s19102404>
- Grando, R. B., Pinheiro, P. M., Bortoluzzi, N. P., da Silva, C. B., Zauk, O. F., Piñeiro, M. O., Aoki, V. M., Kelbouscas, A. L., Lima, Y. B., Drews, P. L., & Neto, A. A. (2020). Visual-based Autonomous Unmanned Aerial Vehicle for Inspection in Indoor Environments. *2020 Latin American Robotics Symposium (LARS) / 2020 Brazilian Symposium on Robotics (SBR) / 2020 Workshop on Robotics in Education (WRE)*, 1-6. <https://doi.org/10.1109/LARS/SBR/WRE51543.2020.9307024>

- Jwo, D. J., Biswal, A., & Mir, I. A. (2023). Artificial Neural Networks for Navigation Systems: A Review of Recent Research. *Applied Sciences*, 13(7), 4475. <https://doi.org/10.3390/app13074475>
- Li, M., Li, J., Cao, Y., & Chen, G. (2024). A Dynamic Visual SLAM System Incorporating Object Tracking for UAVs. *Drones*, 8(6), 222. <https://doi.org/10.3390/drones8060222>
- Li, X., & Xiang, L. (2024). *Photorealistic Robotic Simulation using Unreal Engine 5 for Agricultural Applications*. <https://arxiv.org/abs/2405.18551>
- Polvara, R., Patacchiola, M., Sharma, S., Wan, J., Manning, A., Sutton, R., & Cangelosi, A. (2017). *Autonomous Quadrotor Landing using Deep Reinforcement Learning*. <https://doi.org/10.48550/arXiv.1709.03339>
- Saxena, P., Raghuvanshi, N., & Goveas, N. (2025). *UAV-VLN: End-to-End Vision Language guided Navigation for UAVs*. <https://arxiv.org/abs/2504.21432>

13.3 PDFs en `plan_tesis/bibliografia/` sin metadata bibliográfica estructurada

Sin archivo `.bib` asociado ni metadata completa embebida en el PDF; título tomado del nombre de archivo (convención "Autor - Título.pdf" ya usada en la carpeta). Año, autoría completa y venue de publicación quedan pendientes de completar durante la depuración.

- Comas (s.f., PDF fechado ~2020). *Modelado e implementación de sistemas de navegación aérea autónoma*. [Ver `plan_tesis/bibliografia/Comas - Modelado e implementación de sistemas de navegación aérea autónoma.pdf`; sin metadata de autoría/venue.]
- Presidencia de la Nación Argentina. Decreto 663/2024 — Aviación No Tripulada. [Ver `plan_tesis/bibliografia/Decreto 663-2024 Presidencia - Aviacion No tripulada.pdf`.]
- Nielsen, S. M. (2022). *A Visually Realistic Simulator for Autonomous eVTOL Aircraft*. [Ver `plan_tesis/bibliografia/`; venue de publicación pendiente de confirmar.]
- Ramezani, M. (s.f., PDF fechado ~2023). *UAV Path Planning Employing MPC Reinforcement*. [Ver `plan_tesis/bibliografia/Ramezani - UAV Path Planning Employing MPC Reinforcement.pdf`; venue pendiente.]
- Revista Seguros AAPAS. (2025). *Drones: todo lo que necesitas saber sobre regulaciones y seguros*.
- Sánchez Rodríguez (s.f., PDF fechado ~2019). *Navegación autónoma de un dron hexacóptero con visión estereoscópica*. [Autoría completa pendiente de confirmar.]
- Zhen (s.f., PDF fechado ~2025). *Training-efficient deep reinforcement learning for autonomous driving*. [Metadata de autor del PDF no confiable (aparenta ser el editor del documento, no el autor del paper); verificar durante la depuración.]

13.4 Bibliografía adicional en **informe/bibliografía/** (no incluida en el plan de trabajo aprobado)

- Badrloo, S., & Varshosaz, M. (2017). Monocular vision based obstacle detection. *International Journal of Earth Observation and Geomatics Engineering*, 1(2), 122-130.
- Chen, X., Kundu, K., Zhang, Z., Ma, H., Fidler, S., & Urtasun, R. (2016). Monocular 3D Object Detection for Autonomous Driving. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Geng, S., Cooper, H., Moskal, M., Jenkins, S., Berman, J., Ranchin, N., West, R., Horvitz, E., & Nori, H. (2025). *Generating Structured Outputs from Language Models: Benchmark and Studies*.
<https://arxiv.org/abs/2501.10868>
- Jansen, W., Verreycken, E., Schenck, A., Blanquart, J.-E., Verhulst, C., Huebel, N., & Steckel, J. (2023). *Cosys-AirSim: A Real-Time Simulation Framework Expanded for Complex Industrial Applications*.
<https://arxiv.org/abs/2303.13381>
- Kaneko, N., Yoshida, T., & Sumi, K. (2017). Fast Obstacle Detection for Monocular Autonomous Mobile Robots. *SICE Journal of Control, Measurement, and System Integration*, 10(5), 370-377.
- Molineros, J., Cheng, S. Y., Owechko, Y., Levi, D., & Zhang, W. (2012). Monocular Rear-View Obstacle Detection Using Residual Flow. *Computer Vision – ECCV 2012 Workshops and Demonstrations*, LNCS 7584. Springer.
- Al-Kaff, A., García, F., Martín, D., De La Escalera, A., & Armingol, J. M. (2017). Obstacle Detection and Avoidance System Based on Monocular Camera and Size Expansion Algorithm for UAVs. *Sensors*, 17(5), 1061.
- Raspanti, F., Ozcelebi, T., & Holenderski, M. (2025). Grammar-Constrained Decoding Makes Large Language Models Better Logical Parsers. *ACL 2025 Industry Track*.
- Rill, R.-A., & Faragó, K. B. (2021). Collision Avoidance Using Deep Learning-Based Monocular Vision. *SN Computer Science*, 2, 375. <https://doi.org/10.1007/s42979-021-00759-6>
- Shi, S., Ni, J., Kong, X., Zhu, H., Zhan, J., Sun, Q., & Xu, Y. (2024). An Obstacle Detection Method Based on Longitudinal Active Vision. *Sensors*, 24(14), 4407.
- Vera-Yanez, D., Pereira, A., Rodrigues, N., Molina, J. P., García, A. S., & Fernández-Caballero, A. (2024). Optical Flow-Based Obstacle Detection for Mid-Air Collision Avoidance. *Sensors*, 24(9), 3016.
- Zhou, Z., Liao, Y., Wang, B., Wang, M., Fu, M., & Hu, Z. (2026). Near obstacles detection by inverse perspective mapping of AVM for intelligent vehicles. *PLOS ONE*, 21(1), e0336851.

Anexos

A1. Exploración SLM / GGUF

Nota de ubicación: este documento es material exploratorio de la etapa de planificación (selección de modelos SLM en formato GGUF, técnicas de gramática restringida, evaluación del nivel de innovación de la arquitectura). No es un capítulo de la tesis; se conserva como anexo de referencia para la sección de selección de modelo en `08-DECISIONES-SLM.md`. Reubicado desde `informe/01-INTRO.md` el 2026-08-25.

Limitaciones de hardware

Para una configuración compuesta por una RTX 5060 con 8 GB de VRAM, en la que es necesario compartir recursos de GPU con una simulación de Unreal Engine 5.5, la opción más realista consiste en emplear un *Small Language Model (SLM)*. Se requiere un modelo *instruct-tuned*, capaz de seguir instrucciones de manera fiable y, al mismo tiempo, producir una salida con **gramática muy restringida o limitada**; es decir, un modelo que genere texto estrictamente conforme a un formato, esquema o gramática predefinida (por ejemplo, JSON siempre válido, pares clave-valor específicos o una sintaxis personalizada mínima sin variación libre).

Esta configuración resulta adecuada para una solución local basada en *MCP* (Model Context Protocol), el estándar abierto (propuesto por Anthropic a partir de 2024) que permite a los modelos de lenguaje conectarse con herramientas y datos externos mediante un protocolo uniforme. Un cliente MCP, un LLM local y servidores MCP locales (por ejemplo, para acceso a archivos, ejecución de código o consultas relacionadas con Unreal Engine) pueden ejecutarse completamente sin conexión.

Razones por las que un SLM con salida restringida se ajusta al escenario

- **Presupuesto de VRAM:** una simulación de UE5 probablemente utiliza entre 4 y 6 GB, lo que deja aproximadamente 2–4 GB disponibles para la inferencia del LLM (considerando el overhead de la KV cache).
- **Gramática restringida:** reduce alucinaciones y verbosidad, generando una salida pequeña, rápida y fácil de interpretar (crítico para llamadas a herramientas MCP o respuestas estructuradas).
- **Ejecución local:** elimina dependencias de la nube y permite baja latencia en la integración con la simulación.

Modelos recomendados (febrero 2026, en formato GGUF para llama.cpp / LM Studio)

Se recomienda centrarse en modelos instruct de **3B a 8B** parámetros, ya que ofrecen la capacidad suficiente para seguir instrucciones y manejar patrones de uso de herramientas en 2026. Las cuantizaciones *Q5_K_M* o *Q4_K_M* permiten ajustar el modelo a ~2–4.5 GB de VRAM manteniendo una calidad elevada.

Modelo	Tamaño (quant)	VRAM aprox. (full offload)	Fortalezas	Razones para uso con gramática restringida	Disponibilidad
<i>Qwen3-4B-Instruct</i> / <i>Qwen3-7B-Instruct</i>	~3–5 GB	~2.5–4 GB	Razonamiento sólido, seguimiento de instrucciones, function-calling	Cumplimiento fiable de formatos; variantes 2026 con buen soporte JSON	Qwen/Qwen3-4B-Instruct-GGUF
<i>Phi-4-mini-instruct</i>	~3–4 GB	~2–3.5 GB	Ajustado por Microsoft para datos sintéticos de alta calidad; muy eficaz en tareas estructuradas	Muy alta adherencia a esquemas y salida de baja variación	microsoft/Phi-4-mini-instruct-GGUF
<i>SmolLM3-3B-Instruct</i>	~2–3 GB	~1.8–3 GB	Modelo compacto con rendimiento superior al de muchos 4–7B	Fácil de forzar en formatos rígidos mediante prompt	HuggingFace TB/SmolLM3-3B-GGUF
<i>Gemma-3-4B-IT</i>	~3 GB	~2.5 GB	Ajustado por Google; fuerte rendimiento textual	Buena salida estructurada y soporte de function-calling	google/gemma-3-4b-it-GGUF
<i>Ministral-3-3B-Instruct</i>	~2.5 GB	~2 GB	Modelo compacto optimizado para edge	Diseñado para uso restringido; seguimiento estable de formatos	mistralai/Ministral-3-3B-Instruct-GGUF

La opción preferente suele ser *Phi-4-mini-instruct*, mientras que *Qwen3-4B/7B* puede utilizarse cuando se dispone de aproximadamente 1 GB adicional de VRAM.

Métodos para imponer “gramática limitada” o formato de salida estricto

Los siguientes métodos, compatibles con backends locales como llama.cpp, LM Studio u Ollama, permiten controlar estrictamente la estructura de la salida:

1. **Prompt del sistema con instrucciones estrictas** (método más simple, sin costo computacional significativo)

Ejemplo: Eres un respondedor MCP estricto. Produce ÚNICAMENTE JSON válido que coincida con este esquema. Sin explicaciones, sin texto adicional, sin markdown. Esquema: { "tool_call": {"name": str, "args": dict}, "response": str o null } Si no se necesita herramienta, usar tool_call = null. Escapar correctamente todas las cadenas.

2. **Gramáticas / GBNF en llama.cpp** (confiabilidad alta)

- Definir una gramática libre de contexto mínima (GBNF) obliga al modelo a ajustarse exactamente al formato requerido.
- Compatible nativamente con llama.cpp y expuesto en diversas interfaces como LM Studio.
- Suele reducir la velocidad de generación en un 10–30%, pero garantiza validez estructural completa.

3. **Outlines / Guidance / Ilguidance** (métodos avanzados)

- Permiten imponer esquemas JSON, expresiones regulares o gramáticas personalizadas a nivel de token.
- Garantizan salida estructurada incluso en modelos pequeños.

Para MCP, estos métodos resultan especialmente relevantes, ya que numerosas implementaciones locales (por ejemplo, clientes y servidores MCP publicados en GitHub) requieren formatos fijos como JSON o XML estilo Anthropic.

Procedimiento práctico de configuración

1. **Instalación de LM Studio**

- Descargar un modelo GGUF entre los recomendados (desde el buscador integrado o Hugging Face).
- Configurar entre 20 y 35 capas en GPU, ajustando según el comportamiento de Unreal Engine.
- Activar el modo de gramática si está disponible o utilizar un prompt de sistema personalizado.

2. **Implementación basada en MCP**

- Iniciar con servidores MCP locales básicos (acceso a archivos, shell, etc.).
- Utilizar un cliente MCP compatible con modelos locales (Ollama con plugins MCP o implementaciones Python como mcp-agent).
- Proporcionar las descripciones de herramientas MCP mediante el prompt del sistema.

3. **Integración con Unreal Engine 5**

- Implementar un servidor MCP que lea/escriba logs, datos de Blueprints o estados de simulación.
- El modelo produce comandos estructurados que pueden ser interpretados y aplicados por la aplicación anfitriona.

Sobre la innovación de la solución

De manera general, la *arquitectura en bucle* descrita —un lazo cerrado simple y compacto donde:

1. AirSim proporciona el estado actual (posición, velocidad, datos de sensores, etc.),
2. Un SLM local y de pequeño tamaño procesa dicha información junto con el objetivo o las instrucciones,
3. Se genera un comando estructurado (por ejemplo, un JSON con parámetros de movimiento, guiñada o empuje),
4. El comando se ejecuta mediante la API de AirSim,
5. El proceso se repite a una frecuencia utilizable (por ejemplo, 2–20 Hz),

— resulta *sólida, práctica y adecuada para las restricciones planteadas* (RTX 5060 con 8 GB de VRAM compartida con una simulación en UE 5.5, y uso de modelos pequeños con gramática o salida restringida).

No obstante, en términos de *verdadera innovación* dentro del panorama 2025–2026 sobre control de drones/UAV impulsado por LLMs, dicha arquitectura *ya no se considera especialmente novedosa*. Se trata más bien de uno de los enfoques base que se han estandarizado tanto en investigación como en prototipos aplicados.

Razones por las que ya no se considera una solución de vanguardia

A partir de artículos, tesis, proyectos y experimentos recientes (principalmente entre 2025 y principios de 2026), se observa que:

- El *control en bucle cerrado mediante LLMs* para UAVs/drones en simuladores como AirSim se ha vuelto *muy común*. La mayoría de los trabajos emplea alguna variante del ciclo de retroalimentación.
 - Muchos lo denominan explícitamente "*closed-loop*" (por ejemplo, "LLM-driven closed-loop UAV operation", "closed-loop reasoning/refinement", "closed user-on-the-loop").
 - La idea básica de alimentar el estado → LLM → comando → ejecutar → observar → repetir se ha demostrado repetidamente desde aproximadamente 2023–2024 y se generalizó en contextos UAV hacia 2025.
- La combinación *AirSim + LLM* se ha convertido en una opción *muy utilizada*, debido a que AirSim (basado en Unreal Engine) proporciona un entorno realista para probar física, visión y control de drones. Entre las variantes exploradas se encuentran:
 - Bucles de generación y ejecución de código.
 - Transformación semántica de observaciones (telemetría → descripciones textuales).
 - Configuraciones multi-agente con RAG.
 - Tuberías voz-a-comando.
 - Evaluación de ataques adversarios orientados a medir la robustez de LLMs en lazo cerrado.

- Las propuestas consideradas “innovadoras” en 2025–2026 suelen ir *más allá* del simple bucle estado → SLM → comando, e incluyen:
 - Diseños *dual-LLM* (uno genera código/acciones y otro evalúa/refina dentro de la simulación antes de ejecutar).
 - *Mejoras semánticas* para traducir telemetría en descripciones más comprensibles para modelos pequeños.
 - Enfoques *multi-agente* o *jerárquicos* (planificador + ejecutor + crítico).
 - *Visión en el bucle* (uso de VLMs pequeños como MiniCPM-V para evitar obstáculos).
 - *Síntesis de código en tiempo real* con módulos de corrección o RAG adaptativo.
 - Inferencia en *edge* para reducir aún más la latencia.

En consecuencia, la arquitectura propuesta se corresponde con el patrón básico ampliamente difundido en investigación sobre robótica/UAV basada en LLMs.

Ámbitos en los que la propuesta sí presenta solidez y utilidad

- Se adecúa de forma precisa a las limitaciones del hardware disponible: un SLM pequeño + inferencia rápida mediante llama.cpp permiten mantener latencias suficientemente bajas para tasas de control significativas, mientras que arquitecturas más complejas superarían el límite práctico de 8 GB de VRAM compartida con UE.
- La imposición de *salida estrictamente estructurada* (por ejemplo, JSON mediante gramáticas GBNF) constituye un enfoque actual y recomendable, utilizado en numerosos trabajos posteriores a 2025 para reducir alucinaciones en señales de control.
- Conseguir una operación totalmente local/offline dentro de UE5.5 + AirSim sigue siendo un reto para usuarios no especializados; muchos ejemplos conocidos emplean LLMs en la nube o hardware de mayor capacidad.

Conclusión (evaluación general)

- **Nivel de innovación:** aproximadamente 3–4/10. Se trata de una solución práctica y bien formulada, pero no representa una arquitectura novedosa en el contexto de 2026. Se ajusta más al ámbito de *ingeniería sólida basada en patrones establecidos* que a propuestas conceptualmente nuevas.
- No obstante, una implementación exitosa (vuelo estable, navegación reactiva, baja tasa de fallos en la simulación) constituye un logro destacable, especialmente bajo restricciones de VRAM y con un entorno UE ejecutándose de forma simultánea.
- Si se busca añadir un componente más innovador, es posible incorporar una de las extensiones comunes en 2025–2026, como un resumen semántico del estado, un pequeño evaluador para autocorrección o algún canal de retroalimentación visual, siempre que el presupuesto de VRAM y velocidad lo permita.

Sobre innovar sobre una arquitectura ya extensamente utilizada

Para que un bucle de control de cuadricóptero en AirSim basado en un SLM resulte *genuinamente innovador* en 2026 —especialmente bajo la fuerte restricción de hardware impuesta por una RTX 5060 con 8 GB de VRAM compartida con una simulación en UE 5.5— es necesario ir más allá del patrón estándar “estado → prompt → salida estructurada → actuar → repetir”. Dicho patrón ya constituye la base común en prototipos de investigación actuales.

A continuación se presentan los *giros de procesamiento/software* más prometedores que pueden situar la propuesta en el terreno de una contribución novedosa, ordenados aproximadamente según viabilidad en el hardware disponible y posible impacto:

Técnica / Enfoque	Nivel de innovación (2026)	Impacto en VRAM / Velocidad (8GB compartidos)	Razón por la que puede ser novedosa en este contexto	Dificultad de implementación	Beneficio realista para la autonomía del dron
Compresión semántica del estado + alimentación en lenguaje natural	Alta	Baja (añade ~0.2–0.5 GB de KV cache)	La mayoría de los trabajos usa telemetría cruda; los SLM pequeños razonan mejor con descripciones ricas (“deriva 0.8 m/s hacia la izquierda, obstáculo rojo a 2.1 m, batería 67%”).	Media	20–50% de mejora en calidad de decisiones / menos choques
Autocorrección / refinamiento en bucle cerrado en una sola pasada	Muy alta	Media (prompts más largos)	Permite incluir un campo “crítico” en la salida: el modelo propone acción + evalúa riesgos en la misma inferencia. Pocas implementaciones en entornos de baja VRAM.	Media–Alta	Aumento notable de confiabilidad (80–95% de acciones válidas)
Speculative decoding / borradores asistidos (con cabeza ligera)	Alta	Baja–Media (si se usa estilo EAGLE)	Acelera la generación 1.8–3× sin pérdida de calidad. Muy poco explorado en drones con baja VRAM.	Media–Alta	2–3× más frecuencia de bucle (p. ej., 10–15 Hz en lugar de 3–5 Hz)
Híbrido reactivo + LLM (PID a sub-ms + LLM solo para nivel alto)	Media–Alta	Muy baja	El LLM se utiliza solo para metas/replanificación; la estabilización se delega a controladores reactivos. Reduce dependencia de latencia.	Baja–Media	Vuelo más estable; narrativa innovadora de “control híbrido confiable”

Técnica / Enfoque	Nivel de innovación (2026)	Impacto en VRAM / Velocidad (8GB compartidos)	Razón por la que puede ser novedosa en este contexto	Dificultad de implementación	Beneficio realista para la autonomía del dron
<i>Fine-tuning</i> diminuto en tiempo real / cambio de adaptadores LoRA	Muy alta	Media-Alta (100–300 MB por adaptador)	Conjunto de LoRAs pequeños ("evitación agresiva", "crucero eficiente", etc.) con carga dinámica. Pocas demostraciones en simulación de baja capacidad.	Alta	Adaptación gradual al entorno en UE → efecto de "aprendizaje"
<i>Chain-of-thought</i> multietapa con muestreo en árbol / batching	Media-Alta	Media	Se fuerzan 2–3 futuros posibles en una sola salida; una heurística rápida elige el más seguro. Reduce latencia efectiva.	Media	Mejor manejo de incertidumbre / ráfagas de viento en AirSim

“Combinación ganadora” más prometedora para innovación + hardware disponible

Se recomienda apuntar a la siguiente arquitectura, factible dentro del límite de 8 GB de VRAM compartidos:

1. **Base:** Phi-4-mini-Instruct o Qwen3-4B/7B en Q5_K_M GGUF (~2.5–4 GB cargados; offload parcial si fuera necesario).
2. **Giro central A:** Preprocesamiento mediante *compresión semántica*. Una función Python ligera convierte telemetría de AirSim + objetivo en una descripción vívida en lenguaje natural. Esto potencia significativamente la capacidad de razonamiento de un SLM pequeño.
3. **Giro central B:** Salida con *autocorrección forzada por gramática*:

```
{
  "proposed_action": {"thrust": 0.72, "yaw_delta": -12, "move_vector": [1.2, -0.4, 0.8]},
  "confidence": 0.89,
  "risks": ["batería baja → reducir velocidad si <30%"],
  "alternative_if_risk_high": {"thrust": 0.55, "hover": true}
}
```

La salida se analiza y se aplica la acción más segura o con mayor confianza. Una sola inferencia gestiona propuesta + crítica.

4. **Aceleración:** Activar *speculative decoding* si la versión de llama.cpp o LM Studio lo permite (implementaciones estilo EAGLE o Medusa; comunes en 2026). Puede reducir la latencia por decisión entre 40–200%.
5. **Red de seguridad híbrida:** El LLM produce intención de alto nivel cada 0.5–2 s; un controlador PID rápido en C++/Blueprints opera a 100–200 Hz. Si el LLM falla o se ralentiza → activación de "hover"

seguro + retorno a casa".

Razones por las que este enfoque sí se considera innovador

- La mayoría de los trabajos entre 2025 y 2026 siguen recurriendo a LLMs en la nube, hardware más potente o alimentaciones de estado sin tratamiento semántico.
- La combinación de *compresión semántica + autocorrección con gramática + aceleración especulativa + control híbrido* ejecutada en solo 8 GB de VRAM compartidos con una simulación UE es poco habitual, y potencialmente publicable como una aproximación de *control agente de dron en simulación de alta fidelidad con recursos limitados*.
- Si se logra una navegación estable y reactiva (evitación de obstáculos, búsqueda de objetivos, comportamiento dependiente de batería) con latencias por decisión inferiores a 500 ms, se trataría de una demostración sólida.

Se recomienda comenzar por implementar la *compresión semántica* y la *salida con gramática*, medir tasa de fallos y comparar con el bucle base. Posteriormente, incorporar speculative decoding como mejora incremental.

Nota de ubicación: notas de investigación sobre el concepto y las ventajas de los Small Language Models (SLM). Material de referencia para 08-DECISIONES-SLM.md §8.1. Reubicado desde informe/SLM Concepto y Ventajas.md el 2026-08-25.

Un **Modelo de Lenguaje Pequeño (SLM)** es una versión ligera de un modelo de lenguaje tradicional, diseñada para operar de manera eficiente en entornos con recursos limitados, como teléfonos inteligentes, sistemas embebidos o computadoras de bajo consumo energético 1-5.

Aquí hay una descripción más detallada de lo que son los SLMs:

- **Definición Operativa 6:**

- Un SLM es un modelo de lenguaje (LM) que **puede instalarse en un dispositivo electrónico de consumo común** 4, 6.
- Puede realizar inferencias con una **latencia suficientemente baja** para ser práctico al atender las solicitudes de un solo usuario en sistemas de agentes 5-8.
- Un LLM se define como un LM que no es un SLM 6.

- **Tamaño y Escala** 4, 6, 9:

- Mientras que los Modelos de Lenguaje Grandes (LLMs) tienen cientos de miles de millones, o incluso billones, de parámetros, los SLMs generalmente varían de **1 millón a 10 mil millones de parámetros** 4. A partir de 2025, se considerarían SLMs la mayoría de los modelos con menos de 10 mil millones de parámetros 6.
- Es importante destacar que el término "pequeño" es relativo y se utiliza en comparación con los LLMs más grandes, ya que incluso un modelo de mil millones de parámetros no es "pequeño" por definición absoluta 9, 10.

- **Capacidades y Propósito** 4, 7, 11:

- Los SLMs son suficientemente potentes para manejar las tareas de modelado de lenguaje de las aplicaciones de agentes 11, 12.
- Mantienen capacidades básicas de Procesamiento de Lenguaje Natural (NLP) como generación de texto, resumen, traducción y respuesta a preguntas 4.
- Se afirman como el futuro de la IA agéntica porque son inherentemente más adecuados operacionalmente y necesariamente más económicos para la mayoría de los usos de modelos de lenguaje en sistemas de agentes 11, 13.

- **Ventajas Clave** 5, 7, 14, 15:

- **Menores requisitos computacionales:** Pueden ejecutarse en laptops de consumo, dispositivos de borde y teléfonos móviles 5, 8.
- **Menor consumo de energía:** Modelos eficientes que reducen el uso de energía, haciéndolos más sostenibles 5, 8, 16.

- **Inferencia más rápida:** Generan respuestas rápidamente, ideal para aplicaciones en tiempo real 5, 7, 8.
- **IA en el dispositivo (On-Device AI):** No requieren conexión a internet ni servicios en la nube, lo que mejora la privacidad y la seguridad 5, 17.
- **Despliegue más económico:** Menores costos de hardware y nube, lo que hace la IA más accesible 5, 14.
- **Mayor flexibilidad y personalización:** Son más fáciles de ajustar para tareas específicas de dominio 5, 15.
- **Cómo se logran "pequeños" 1, 9, 18:**
 - **Destilación de conocimiento:** Entrenamiento de un modelo "estudiante" más pequeño utilizando el conocimiento transferido de un modelo "maestro" más grande 9, 18, 19.
 - **Poda (Pruning):** Eliminación de parámetros redundantes o menos importantes dentro de la arquitectura de la red neuronal 9, 20.
 - **Cuantización:** Reducción de la precisión de los valores numéricos utilizados en los cálculos (por ejemplo, convertir números de punto flotante a enteros) 9, 21.
- **Aplicaciones Comunes 22:**
 - Chatbots y asistentes virtuales.
 - Generación de código.
 - Traducción de idiomas.
 - Resumen y generación de contenido.
 - Aplicaciones en salud.
 - IoT y computación de borde.
 - Herramientas educativas.

En resumen, los SLMs son modelos eficientes y adaptables que buscan democratizar el acceso a la IA, permitiendo su despliegue en una gama más amplia de dispositivos y situaciones donde los LLMs serían inviables debido a sus altos requisitos de recursos 3, 4, 23, 24.

A3. SLM — Optimización y Desafíos

Nota de ubicación: notas de investigación sobre optimización y desafíos de los SLM. Material de referencia para `08-DECISIONES-SLM.md` §8.1. Reubicado desde `informe/SLM Optimización y Desafíos.md` el 2026-08-25.

Un **Modelo de Lenguaje Pequeño (SLM)** busca ser una versión eficiente y de bajo consumo de un modelo de lenguaje, diseñada para operar en entornos con recursos limitados 1, 2. La meta es mantener la precisión y/o adaptabilidad de los modelos de lenguaje grandes (LLMs) bajo ciertas restricciones, como hardware de entrenamiento o inferencia, disponibilidad de datos, ancho de banda o tiempo de generación 3.

Para reducir la cantidad de parámetros de un SLM sin perder la precisión de un LLM, se emplean varias técnicas clave:

- **Destilación de Conocimiento (Knowledge Distillation):**

- Esta técnica implica entrenar un modelo "estudiante" más pequeño para replicar el comportamiento de un modelo "maestro" más grande y complejo 4, 5.
- La destilación de conocimiento puede transferir las capacidades matizadas del LLM al SLM, permitiendo que el modelo pequeño imite las salidas del LLM en conjuntos de datos específicos de la tarea 5, 6.
- Por ejemplo, Babyllama, un modelo compacto de 58 millones de parámetros, fue desarrollado utilizando un modelo Llama como "maestro", demostrando que la destilación puede superar el rendimiento de los modelos "maestros", especialmente en condiciones de datos limitados 5, 7, 8.
- Se puede mejorar la calidad de las respuestas de los modelos "estudiantes" mediante modificaciones en la función de pérdida de destilación 5. También es posible fusionar múltiples modelos de lenguaje como un solo "maestro" para destilar conocimiento en SLMs 9.
- Incluso se pueden destilar cadenas de razonamiento de un LLM más grande a un SLM, lo que ha demostrado mejorar las habilidades de razonamiento aritmético, matemático de varios pasos, simbólico y de sentido común en los modelos pequeños 10.
- La utilización de "razones" como supervisión adicional durante la destilación puede hacerla más eficiente en el uso de muestras, e incluso permitir que el modelo destilado supere a los LLMs en benchmarks comunes 10.

- **Poda (Pruning):**

- La poda es una técnica que reduce el número de parámetros del modelo para mejorar la eficiencia computacional y disminuir el uso de memoria, manteniendo al mismo tiempo los niveles de rendimiento 4, 11.
- Existen dos enfoques principales:

- **Poda no estructurada:** Elimina pesos individuales menos significativos, ofreciendo un control más granular para reducir el tamaño del modelo 11. Técnicas como SparseGPT pueden manejar modelos a gran escala, y la estrategia de poda n:m (eliminar exactamente n pesos de cada m) busca equilibrar la flexibilidad de la poda con la eficiencia computacional para lograr aceleraciones significativas 11-13.
- **Poda estructurada:** Comprime los LLMs eliminando grupos de parámetros de manera estructurada, lo que facilita una implementación de hardware más eficiente 14. Esto incluye abordar la redundancia en la arquitectura Transformer y la escasez de neuronas, o la poda de capas 14, 15.
- **Cuantización (Quantization):**
 - La cuantización reduce la precisión de los valores numéricos utilizados en los cálculos (por ejemplo, convertir números de punto flotante a enteros) para comprimir los LLMs con vastos recuentos de parámetros 4, 16.
 - Métodos como GPTQ se centran en la cuantización de solo pesos por capa, minimizando el error de reconstrucción 16. Otros métodos cuantifican tanto los pesos como las activaciones 16.
 - Técnicas como AWQ y ZeroQuant tienen en cuenta las activaciones para evaluar la importancia de los pesos, optimizando la cuantización de manera más efectiva 16.
 - La Cuantización del Caché K/V (Key-Value Cache) se aplica específicamente para la inferencia eficiente de secuencias largas 16.
 - SmoothQuant y SpinQuant abordan los valores atípicos en la distribución de activaciones, migrando la dificultad de cuantificación de las activaciones a los pesos o transformando los valores atípicos en un nuevo espacio 17.
 - Los métodos de entrenamiento conscientes de la cuantización (QAT), como LLM-QAT y Edge-QAT, utilizan la destilación con modelos float16 para recuperar el error de cuantificación, demostrando un rendimiento sólido 17.

Otras técnicas y avances que contribuyen a la eficiencia y el mantenimiento de la precisión:

- **Arquitecturas Ligeras:** Diseños como MobileBERT (reducción de tamaño 4.3x y aceleración 5.5x sobre BERT), DistilBERT y TinyBERT 18 son optimizaciones de modelos encoder-only. Las arquitecturas decoder-only ligeras como BabyLLaMA, TinyLLaMA, MobilLLaMA y MobileLLM utilizan destilación de conocimiento, optimización de la sobrecarga de memoria, y esquemas de compartición de parámetros y embeddings 7.
- **Aproximaciones Eficientes de Autoatención:** Métodos como Reformer, mecanismos de atención lineal, Mamba y RWKV reducen la complejidad computacional de la autoatención de $O(N^2)$ a $O(N \log N)$ o $O(N)$ 19.
- **Búsqueda de Arquitectura Neural (NAS):** Métodos automatizados descubren las arquitecturas de modelos más eficientes para tareas y hardware específicos 20.
- **Entrenamiento de Precisión Mixta:** Técnicas como Automatic Mixed Precision (AMP), BFLOAT16 y FP8 mejoran la eficiencia del preentrenamiento al usar representaciones de baja precisión para la propagación hacia adelante y hacia atrás 21.

- **Ajuste Fino Eficiente en Parámetros (PEFT):** Técnicas como LoRA y Prompt Tuning actualizan solo un pequeño subconjunto de parámetros o añaden módulos ligeros, reduciendo los costos computacionales y preservando el conocimiento del modelo preentrenado 22-24.
- **Aumento de Datos:** El aumento de datos, a través de técnicas como AugGPT, Evol-Instruct y Reflection-tuning, incrementa la complejidad, diversidad y calidad de los datos de entrenamiento para SLMs, mejorando la generalización y el rendimiento en tareas específicas, especialmente cuando los datos son limitados 25, 26.
- **Potencia de Razonamiento:** Los SLMs ya poseen suficiente poder de razonamiento para una porción sustancial de las invocaciones de agentes. La capacidad, no el recuento de parámetros, es la restricción. Con el entrenamiento, el prompting y las técnicas de aumento agéntico modernos, los SLMs son viables y, comparativamente, más adecuados que los LLMs para sistemas agénticos modulares y escalables 27. Por ejemplo, el modelo Phi-2 (2.7 mil millones de parámetros) de Microsoft logra puntuaciones de razonamiento de sentido común y generación de código a la par de modelos de 30 mil millones de parámetros, mientras que funciona aproximadamente 15 veces más rápido 28. DeepSeek-R1-Distill-Qwen-7B supera a modelos propietarios grandes como Claude-3.5-Sonnet-1022 y GPT-4o-0513 29.

En resumen, los SLMs logran un equilibrio entre tamaño y rendimiento, empleando una combinación de técnicas de compresión y optimización, junto con arquitecturas de diseño inteligente y entrenamiento específico, para ofrecer capacidades similares a las de los LLMs en tareas especializadas 4, 27, 30.

SLMs y la Propensión a Alucinaciones

En cuanto a la propensión a alucinaciones, los Modelos de Lenguaje Grandes (LLMs) son conocidos por el problema de la "alucinación", que se define como la generación de contenido sin sentido o falso en relación con ciertas fuentes 31.

En el contexto de los SLMs:

- Un estudio utilizando **HallusionBench**, un benchmark para el razonamiento en modelos de visión-lenguaje, encontró que **los tamaños de modelo más grandes reducían las alucinaciones** 32. Esto sugiere que, en general, los modelos más pequeños podrían ser más propensos a generar contenido alucinatorio.
- El análisis del benchmark de alucinaciones AMBER también indicó que el tipo de alucinación varía a medida que cambia el recuento de parámetros en Minigpt-4 32.
- Las alucinaciones son un riesgo y una limitación que los SLMs comparten con los LLMs 33.
- La investigación futura necesita considerar no solo cómo cambia el total de alucinaciones en los SLMs, sino también cómo el tipo y la gravedad pueden verse influenciados por el tamaño del modelo 32.

Por lo tanto, existe evidencia que sugiere que los SLMs podrían ser más susceptibles a las alucinaciones debido a su menor tamaño, aunque este es un campo de investigación activo para comprender completamente la relación entre el tamaño del modelo y la naturaleza de las alucinaciones 32.

Nota de ubicación: notas de investigación sobre LoRA (Low-Rank Adaptation) aplicado a SLM. No hay evidencia en el código actual de que LoRA se haya implementado en el pipeline final; tratar como exploración de alternativas, no como decisión adoptada, salvo que `08-DECISIONES-SLM.md` documente lo contrario. Reubicado desde `informe/Optimización de Modelos de Lenguaje Pequeños mediante LoRA.md` el 2026-08-25.

La optimización de Modelos de Lenguaje Pequeños (SLM) con **LoRA (Low-Rank Adaptation)** es una estrategia altamente eficiente que forma parte de las técnicas de **Ajuste Fino Eficiente en Parámetros (PEFT)** 1, 2.

LoRA optimiza los modelos funcionando mediante una **descomposición de bajo rango**: actualiza solo un subconjunto muy pequeño de parámetros (o afina unas pocas capas específicas) mientras mantiene fijos la mayor parte de los parámetros del modelo preentrenado original 2, 3.

La aplicación de LoRA en SLMs aporta las siguientes ventajas y características fundamentales:

- **Eficiencia de recursos:** Al actualizar solo una fracción de la red, LoRA **reduce drásticamente los costos computacionales y los requisitos de memoria** asociados con el proceso de ajuste fino (fine-tuning), haciéndolo mucho más ligero y accesible 1-3.
- **Agilidad extrema:** El ajuste de un SLM utilizando LoRA requiere **solo unas pocas horas de procesamiento en GPU**. Esto permite a los desarrolladores un ciclo de iteración muy rápido para agregar nuevos comportamientos, corregir errores o especializar el modelo de la noche a la mañana, en lugar de esperar semanas 4.
- **Prevención del sobreajuste (Overfitting):** Dado que la mayor parte del modelo original permanece inalterada, LoRA ayuda a **preservar el conocimiento preentrenado del modelo**, reduce el riesgo de sobreajuste y mejora la flexibilidad 2.
- **Especialización de dominio:** Es el método ideal para adaptar un SLM general a **conjuntos de datos de dominios específicos o aplicaciones de nicho**. Por ejemplo, un modelo puede optimizarse de forma rápida con LoRA sobre documentos legales para crear un asistente de análisis de contratos, o sobre manuales técnicos para desarrollar una guía de resolución de problemas 5.
- **Variantes avanzadas y facilidad de uso:** Su implementación hoy en día es sencilla gracias a bibliotecas como peft de Hugging Face, que permiten configurar rápidamente los parámetros de la adaptación 3. Además, existen variantes populares empleadas en SLMs como **QLoRA** (que cuantiza el modelo para reducir aún más el consumo de recursos) y **DoRA**, que expanden la capacidad de ajustar modelos bajo restricciones de hardware 1, 4.

Nota de ubicación: notas de investigación sobre decodificación restringida (constrained decoding / gramáticas GBNF / `json_schema`). Material de referencia directo para `08-DECISIONES-SLM.md` §8.2 ("Decodificación restringida vs. parser tolerante"). Reubicado desde `informe/Optimización de Modelos mediante Decodificación Restringida.md` el 2026-08-25.

Según las fuentes, la generación de salidas estructuradas y la mejora en la eficiencia de la inferencia se logra principalmente a través de una técnica conocida como **decodificación restringida (constrained decoding)** 1, 2.

Generación de salidas estructuradas:

- La decodificación restringida interviene en el proceso de generación del modelo evaluando las reglas de una gramática o restricción dada y **enmascarando (ocultando) los tokens que son inválidos** en cada paso 2, 3.
- Al hacer esto, el modelo es guiado para que tome muestras únicamente de tokens válidos, lo que garantiza que la salida final se ajuste perfectamente a la estructura predefinida, siendo **JSON Schema** el estándar predominante en la industria para definir estos formatos 2, 4.
- Para lograr esto, se han desarrollado motores de gramática y marcos de trabajo optimizados como Guidance, Outlines, Llamacpp y XGrammar, los cuales traducen estas reglas para controlar las respuestas del modelo 5, 3.
- En el caso específico de los SLM integrados en sistemas de agentes autónomos, mantener formatos estrictos (como JSON, XML o código Python) es vital para comunicarse con otras herramientas 6. Las fuentes sugieren que los SLM pueden ser ajustados (fine-tuned) de forma económica para forzar una única decisión de formato, evitando así alucinaciones estructurales que rompan el código del sistema 6.

Mayor eficiencia en la inferencia: Aunque aplicar gramáticas o restricciones podría parecer un proceso que añade carga computacional, las implementaciones optimizadas en realidad **pueden acelerar el proceso de generación hasta en un 50%** en comparación con la generación sin restricciones 7. Esto se logra mediante varias optimizaciones clave:

- **Procesamiento en paralelo:** El cálculo de la máscara de tokens permitidos se ejecuta en paralelo con el paso hacia adelante (forward pass) del modelo de lenguaje 8.
- **Compilación simultánea:** La compilación inicial de la gramática requerida se realiza de manera concurrente con los cálculos de pre-llenado (pre-filling) del prompt inicial 8, 9.
- **Optimizaciones avanzadas:** Los sistemas emplean técnicas como el almacenamiento en caché de gramáticas y la decodificación especulativa basada en restricciones para reducir los tiempos de respuesta 8. Además, marcos como *Guidance* alcanzan una eficiencia sobresaliente al ser capaces de acelerar y saltarse directamente ciertos pasos de generación cuando la gramática los hace predecibles 10.